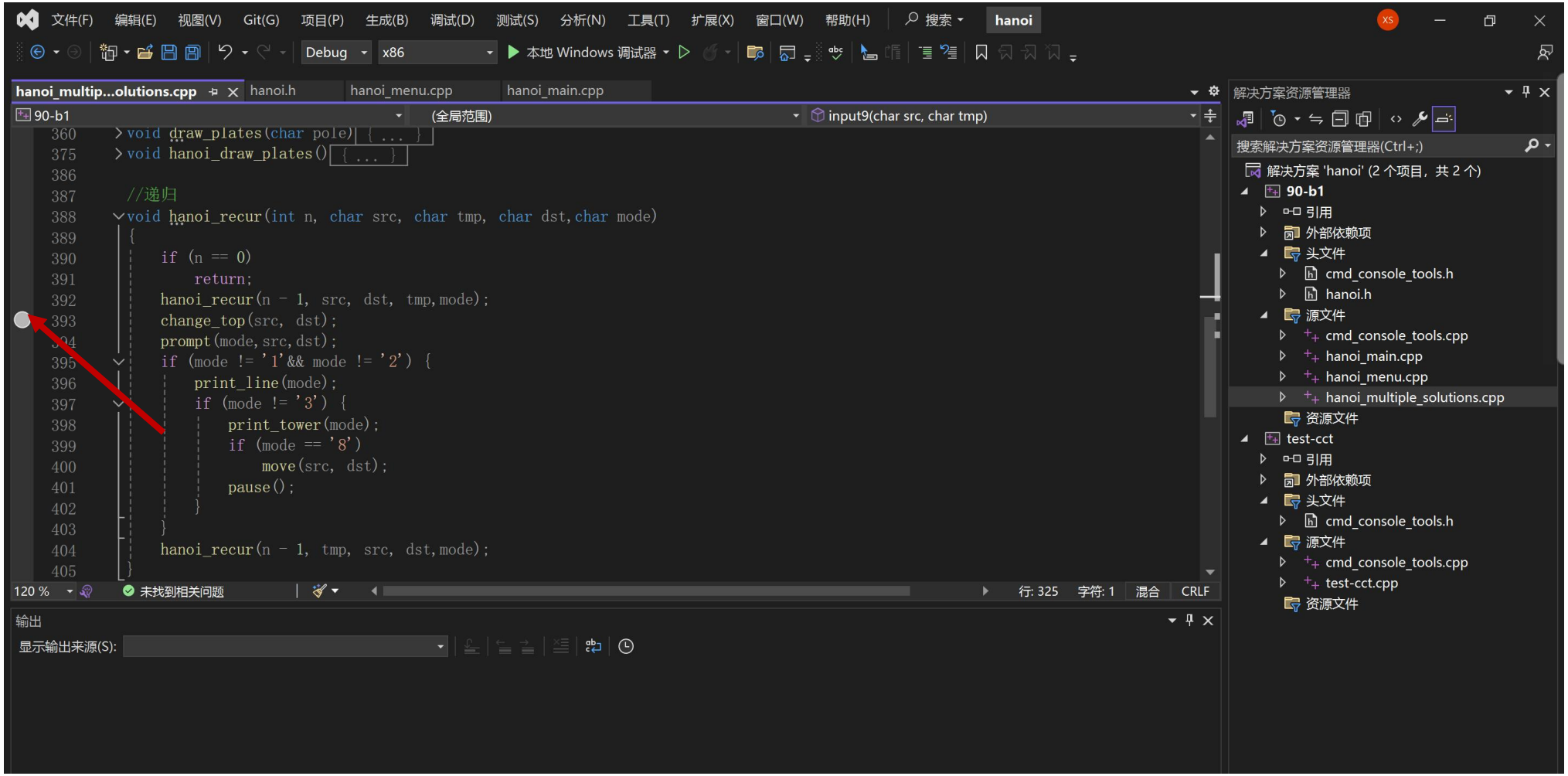
VS调试工具的使用报告

2350222 信12 盛曦

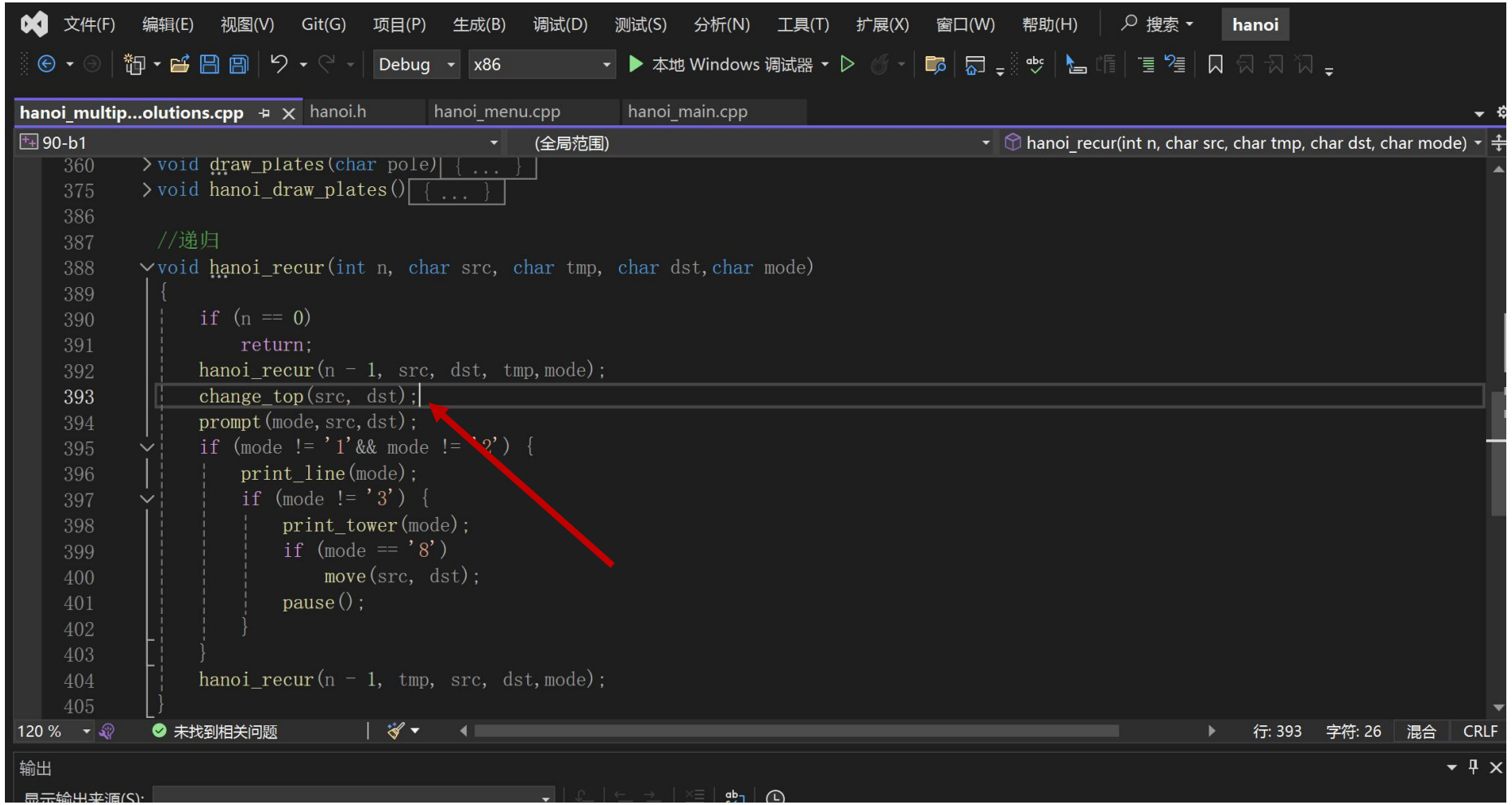
0：断点

- 0-1设断点
- 以hanoi程序中，希望在进入change_top函数前设断点为例
- 方法一：左键单击该行的最左侧即可设置断点



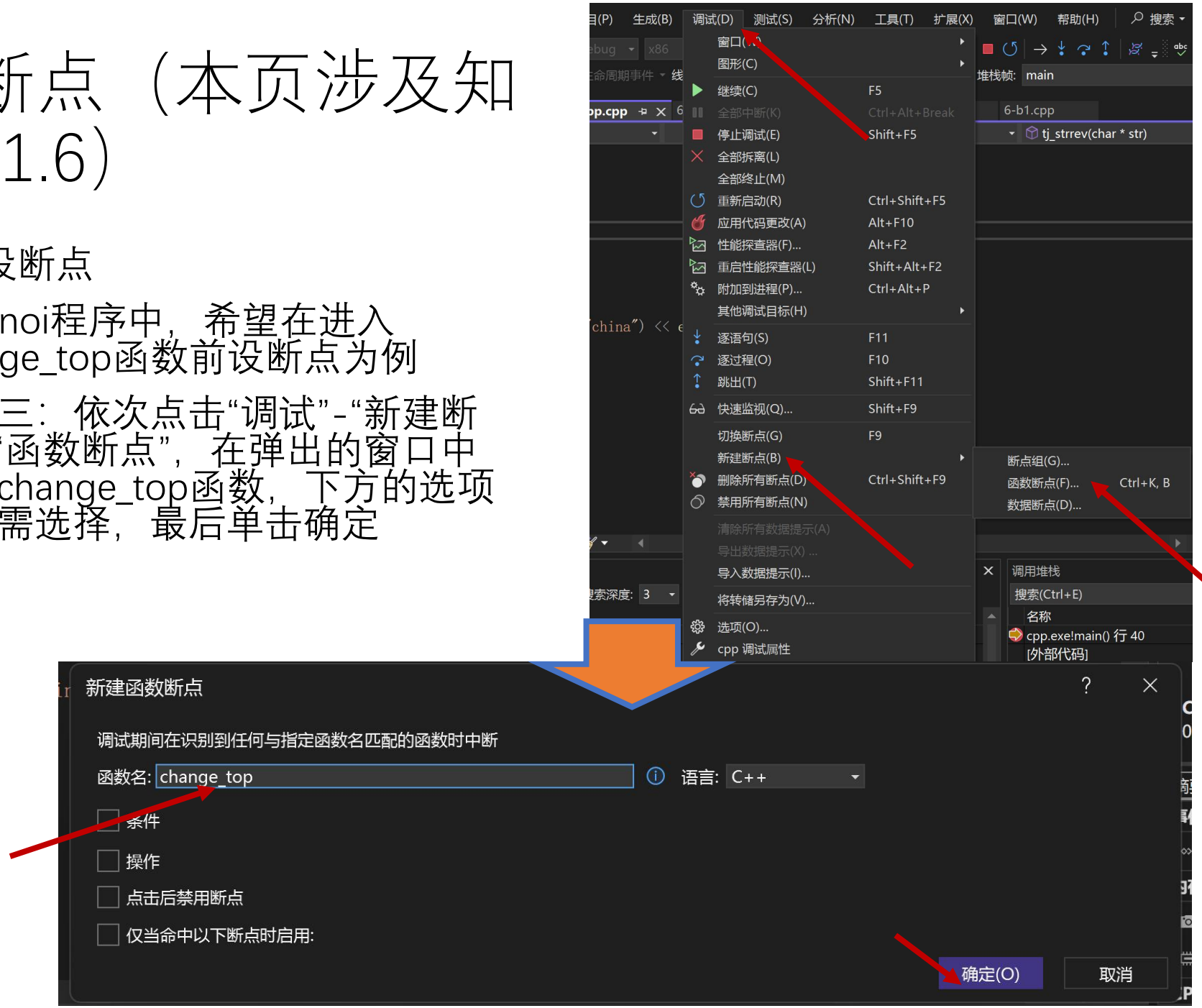
0：断点

- 0-1设断点
- 以hanoi程序中，希望在进入change_top函数前设断点为例
- 方法二：左键单击该行的任意位置，使光标在该行，按快捷键F9 （需要预先functionlock，按住Fn再按esc，使esc键上的灯亮起，下略）



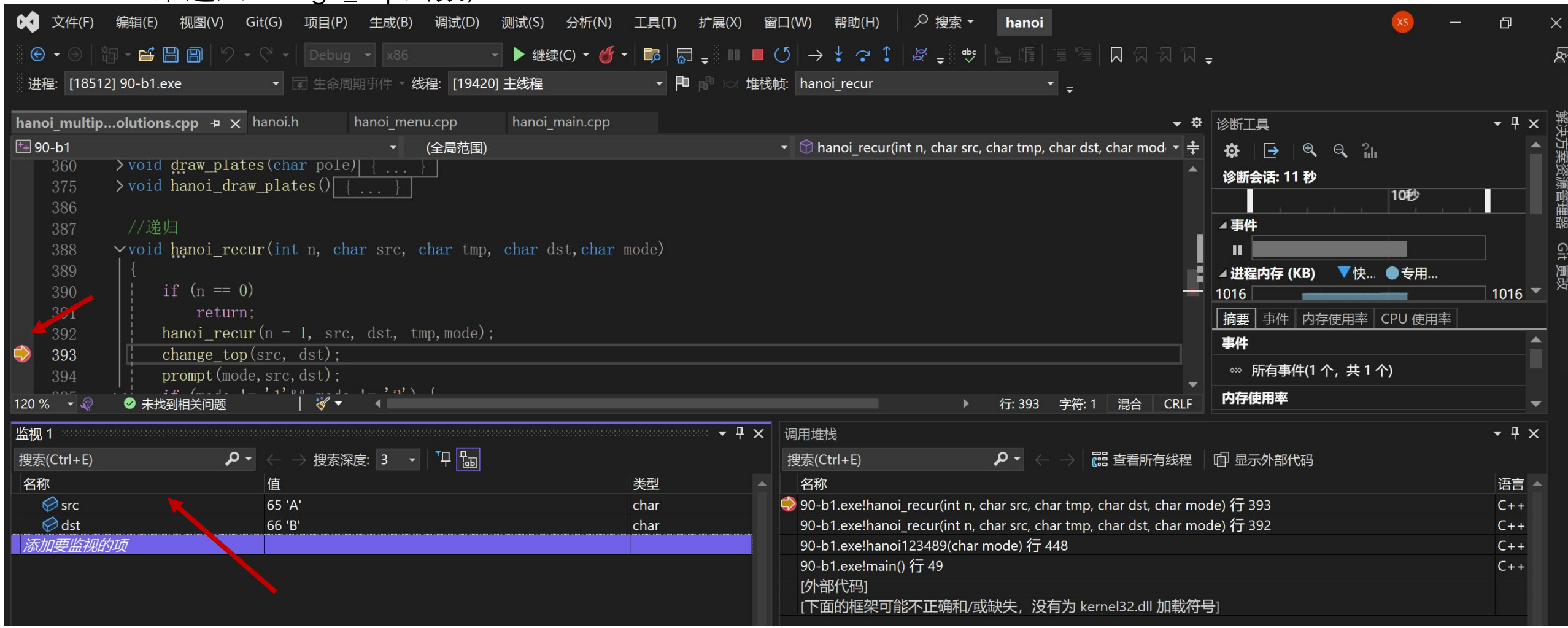
0：断点（本页涉及知识点1.6）

- 0-1设断点
- 以hanoi程序中，希望在进入change_top函数前设断点为例
- 方法三：依次点击“调试”-“新建断点”-“函数断点”，在弹出的窗口中输入change_top函数，下方的选项可按需选择，最后单击确定



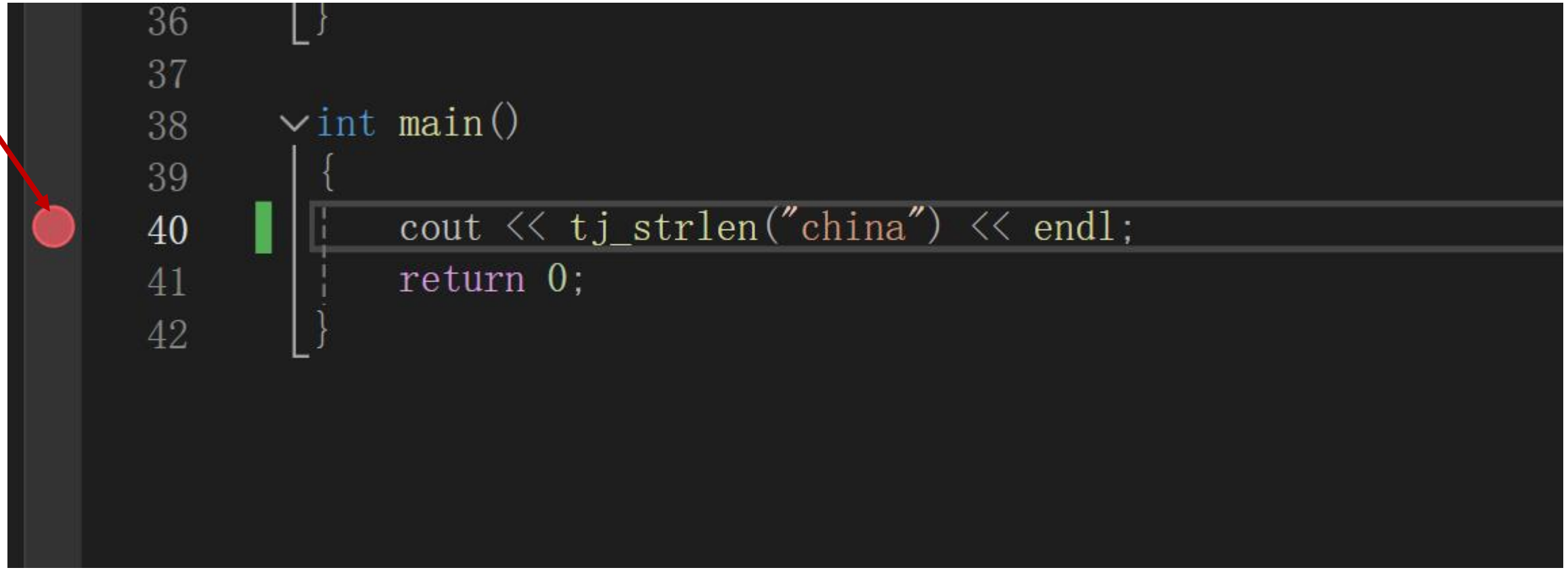
0：断点

- 0-1设断点
- 以hanoi程序中，希望在进入change_top函数前设断点为例
- 注意：设置断点后开始调试/继续调试，程序会在开始运行这一行的**前一刻**间断。（本例中，尚未进入change_top函数）



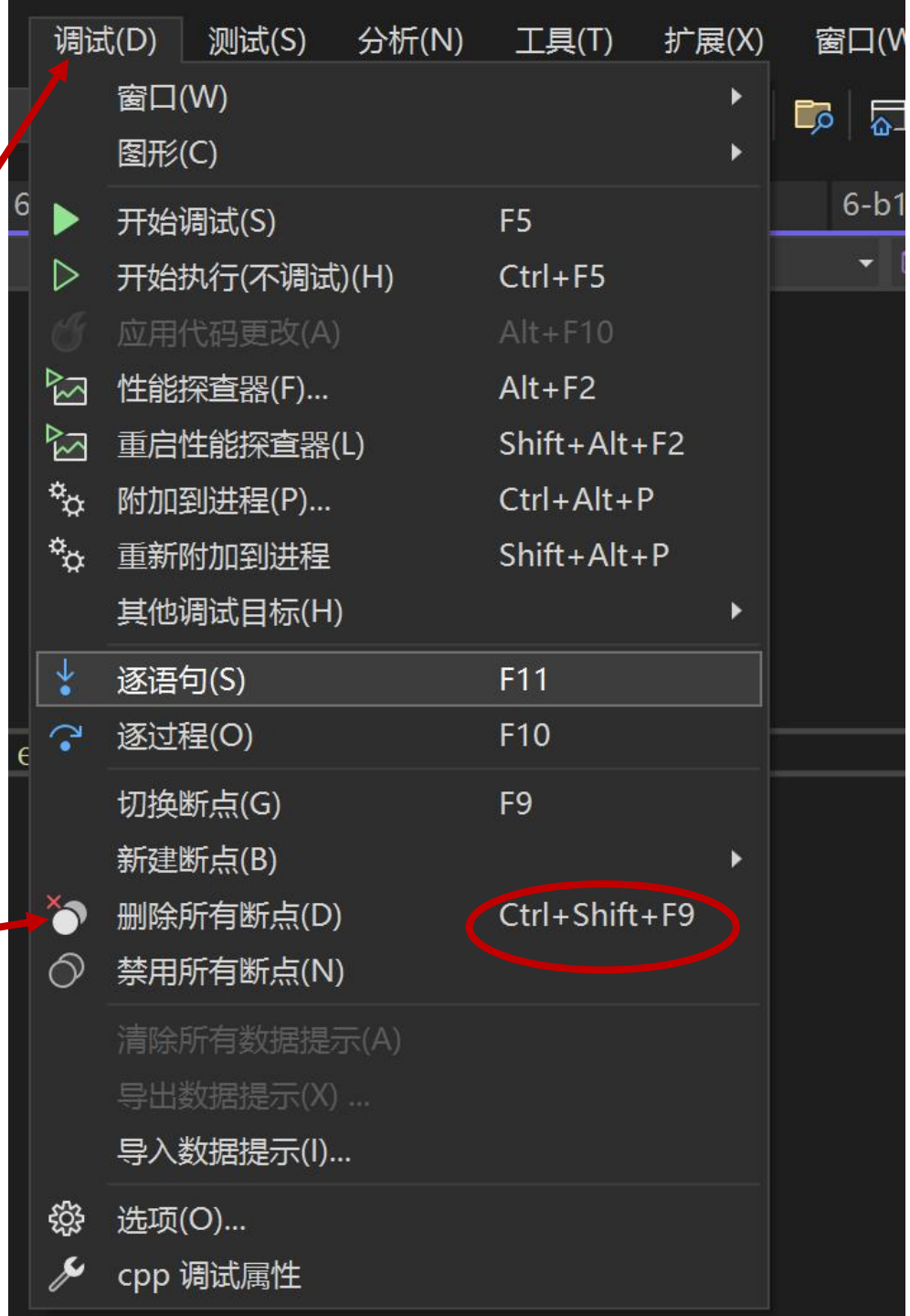
0：断点

- 0-2删除断点
- 方法一：左键单击红色圆点即可删除该处断点



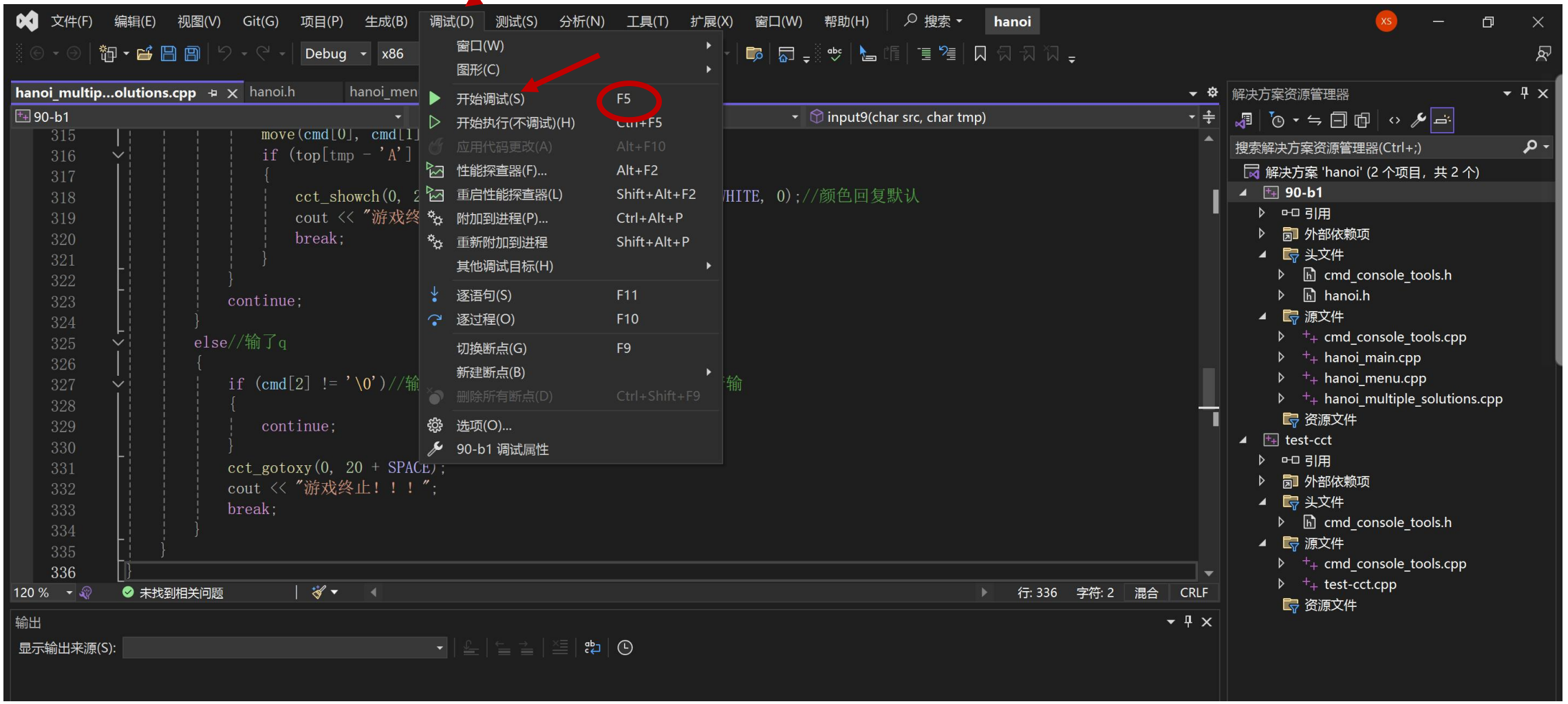
0：断点

- 0-2删除断点
- 方法二：“调试”>“删除所有断点”
- 方法三：快捷键ctrl+shift+F9



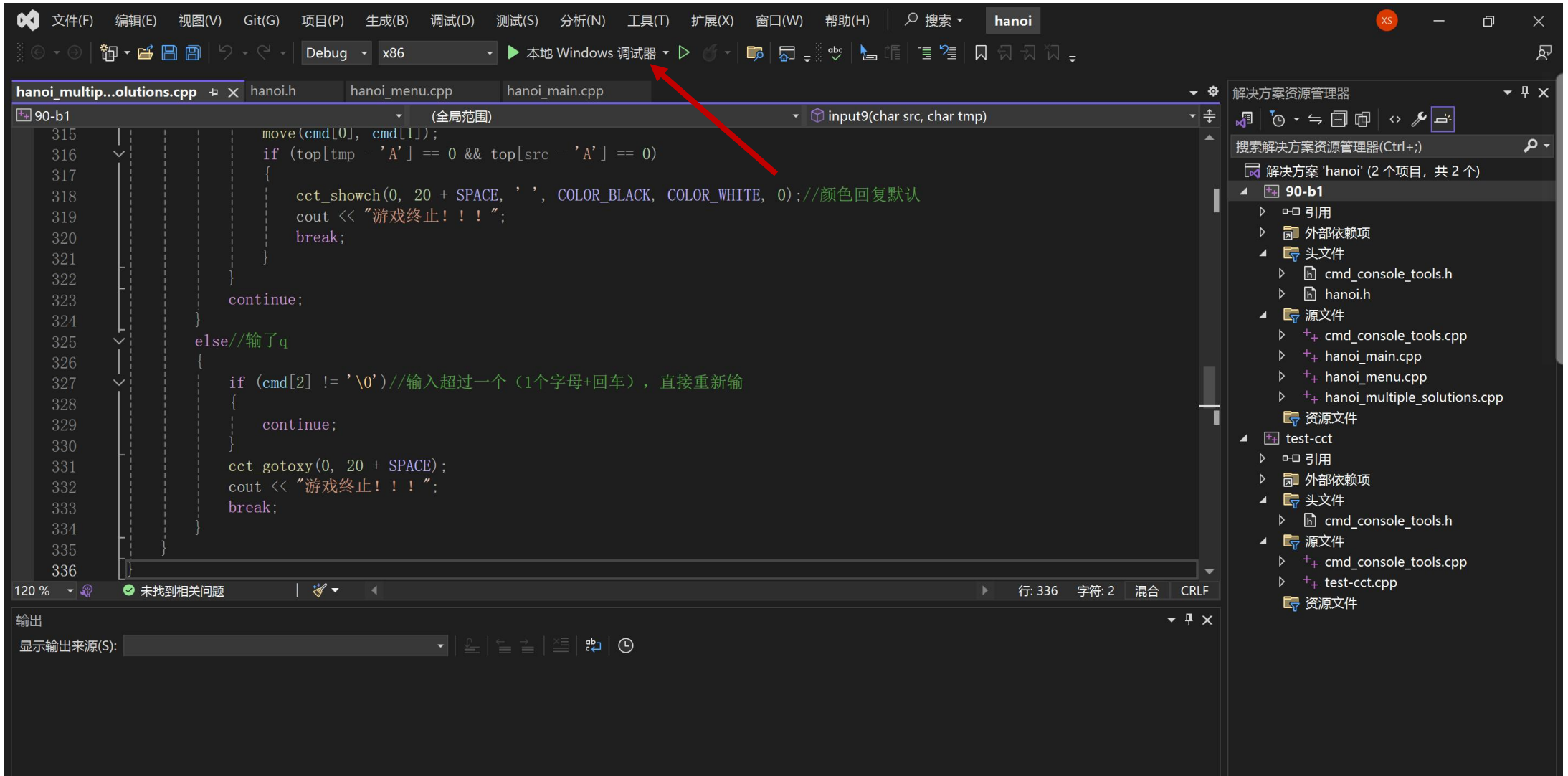
1：开始调试（本页涉及知识点1.1）

- 1-1 开始调试
- 方法一：依次单击最上方菜单栏的“调试”-“开始调试”
- 方法二：快捷键F5



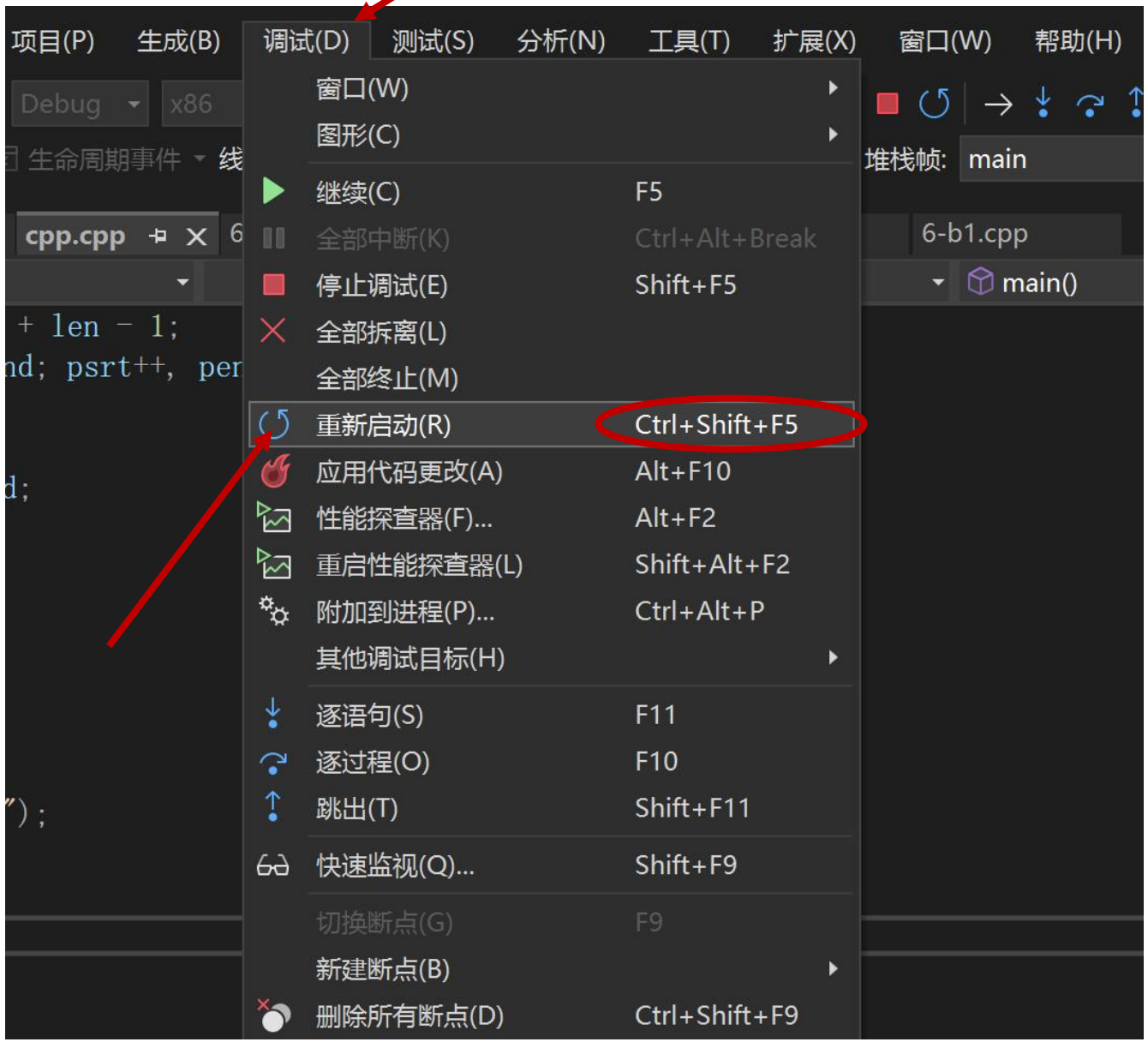
1：开始调试（本页涉及知识点1.1）

- 1-1 开始调试
- 方法三：单击绿色三角“本地Windows调试器”



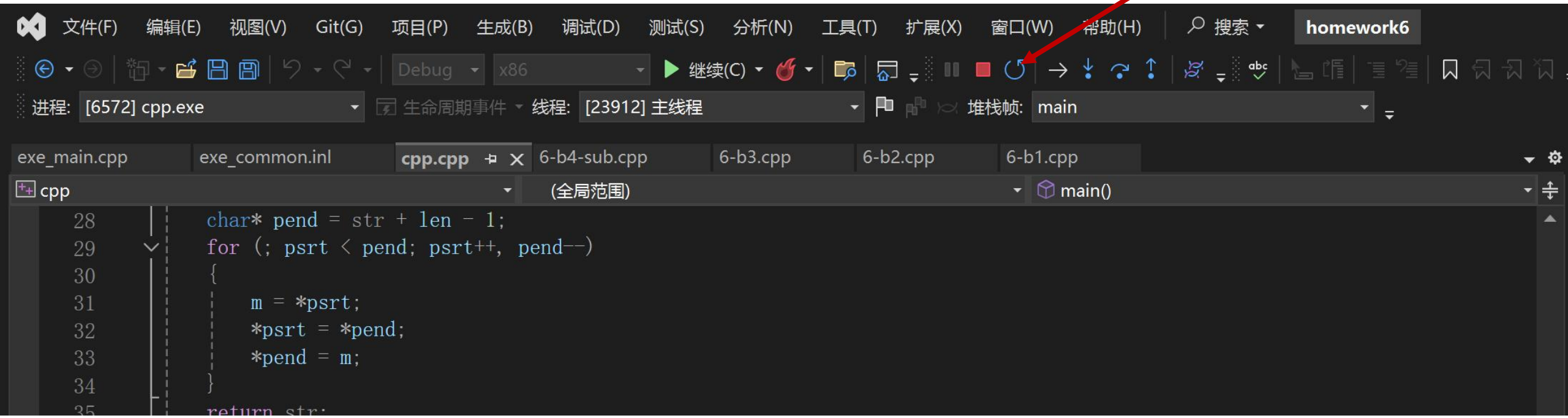
1：开始调试（本页涉及知识点1.1）

- 1-2 重新开始调试：在调试未完成时直接重新启动调试，重置一切变化
- 方法一：“调试”>“重新启动”
- 方法二：快捷键ctrl+shift+F5



1：开始调试（本页涉及知识点1.1）

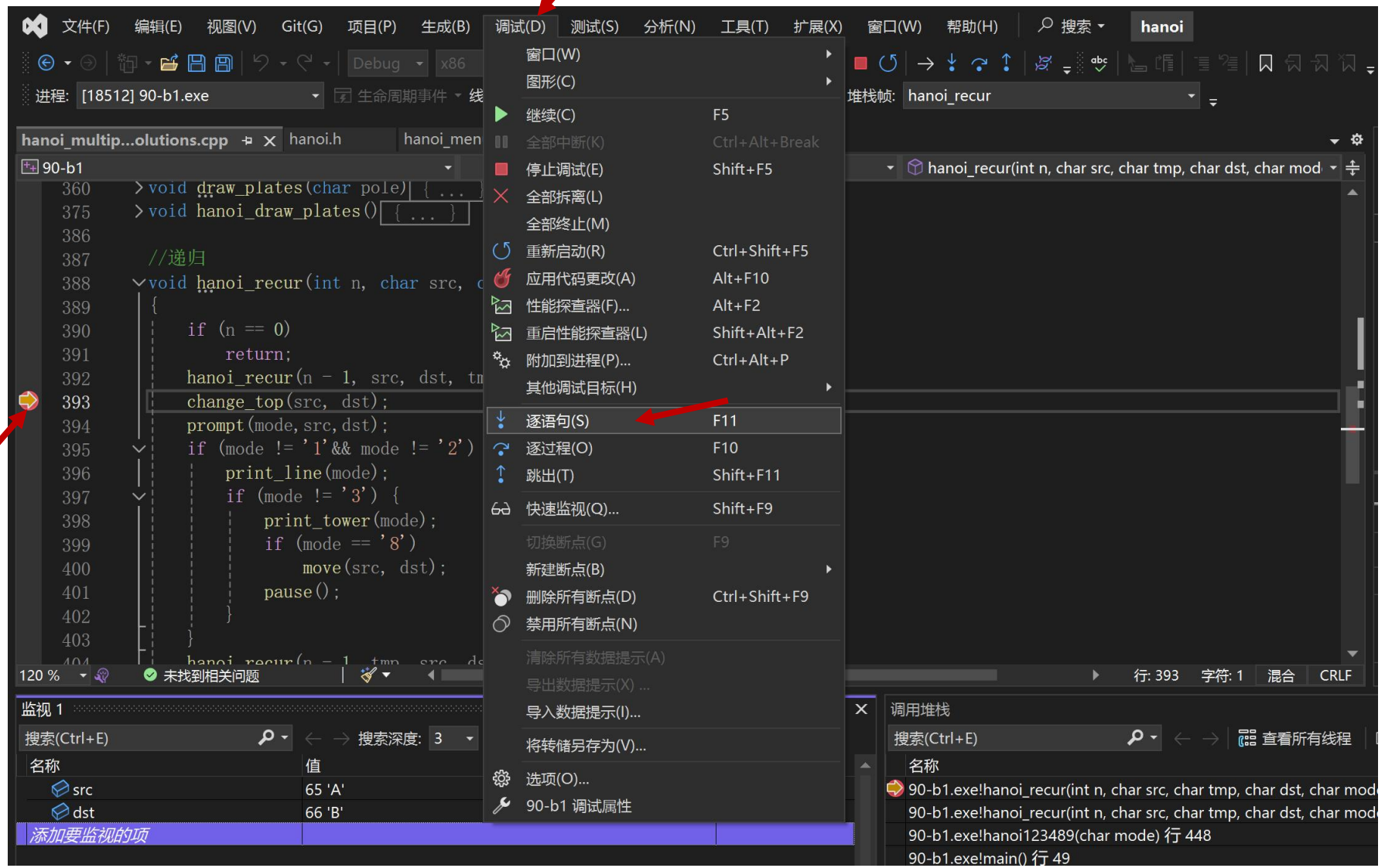
- 1-2 重新开始调试：在调试未完成时直接重新启动调试，重置一切变化
- 方法三：单击蓝色回转箭头



2.1：逐句执行（本页涉及知识点1.2）

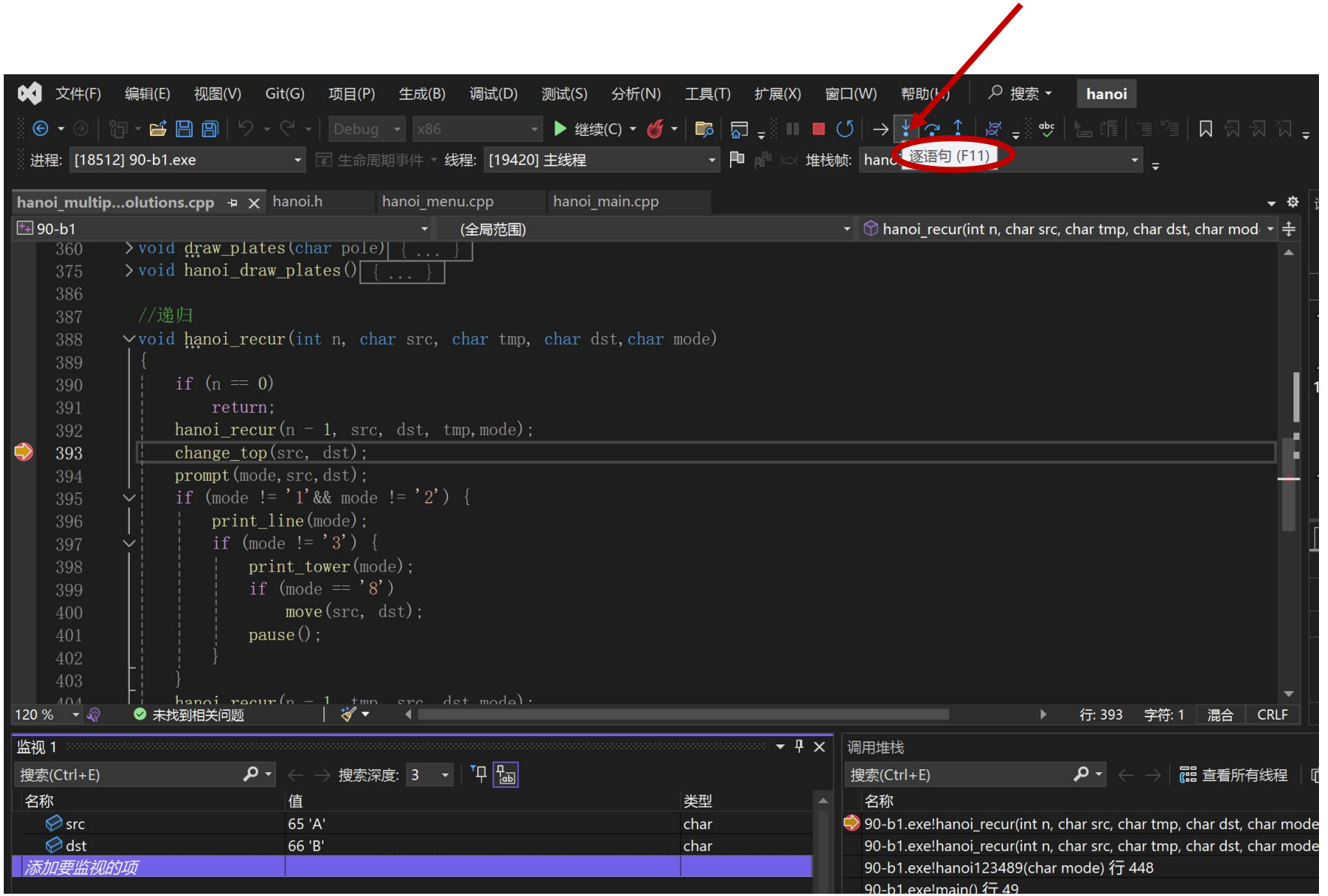
- 以hanoi程序中，希望进入change_top函数并逐句执行为例
- 2.1-1 设置断点（见0）
- 2.1-2 开始调试（见1）
- 2.1-3逐句执行
- 方法一：依次单击菜单栏“调试”-“逐语句”

黄色箭头表示当前将要执行的语句



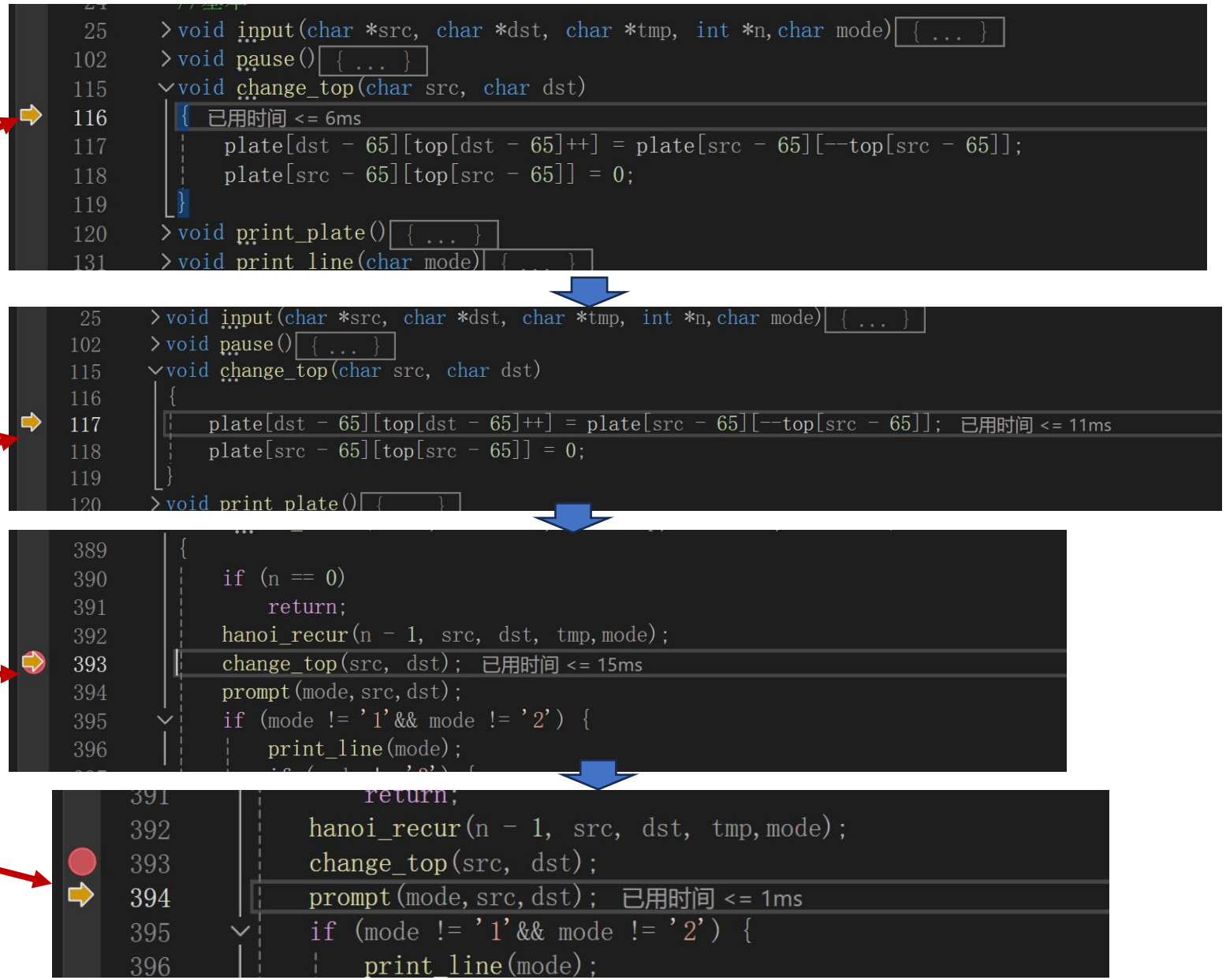
2.1：逐句执行（本页涉及知识点1.2）

- 以hanoi程序中，希望进入change_top函数并逐句执行为例
- 2.1-1 设置断点（见0）
- 2.1-2 开始调试（见1）
- 2.1-3逐句执行
- 方法二：单击蓝色向下箭头
- 方法三：快捷键F11



2.2：自定义函数内部逐句执行（本页涉及知识点1.2，1.6）

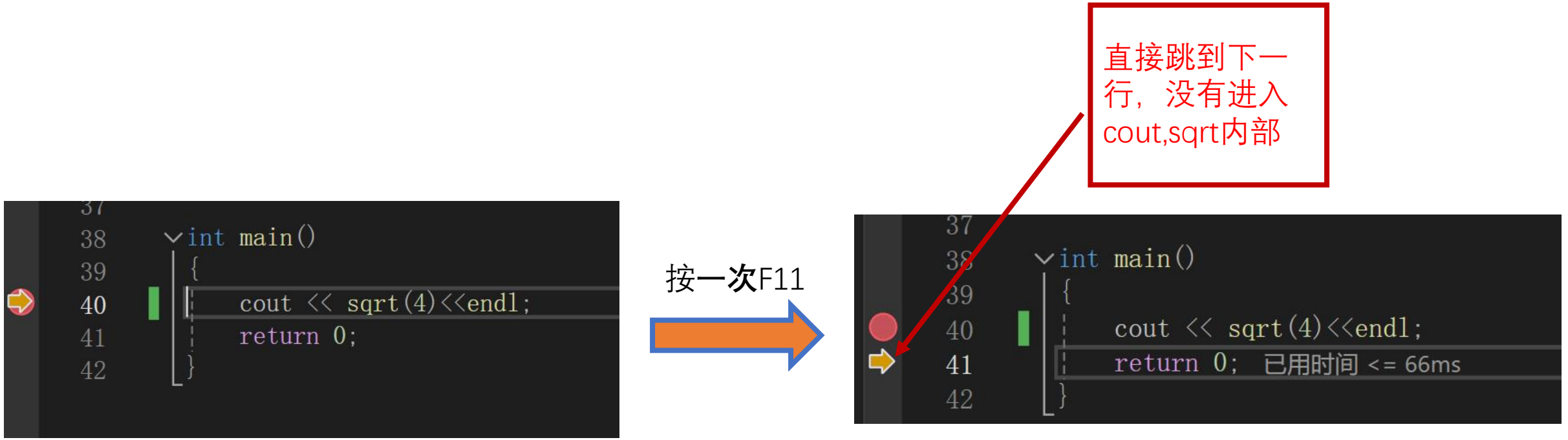
- 以hanoi程序中，希望进入change_top函数并逐句执行为例
- 可以看到，F11逐句执行后，黄色箭头已到达change_top内部，即断点处语句（调用change_top）的下一行。
- 若一直F11逐句执行，程序将依次执行change_top函数内部的每一行（按行执行，哪怕单{、单}也会停留）
- 最后“返回现场”,回到调用处，继续执行下一行（不会重新进入change_top）



2.2：自定义函数内部逐句执行（本页涉及知识点1.2，1.3，1.6）

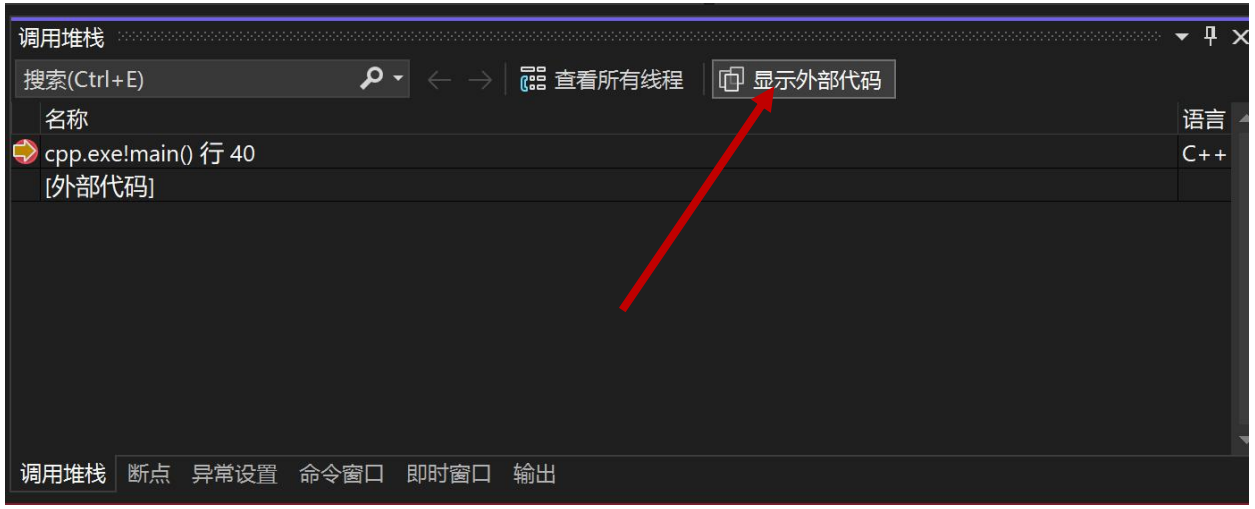
注意：默认情况下，调试器会跳过非用户代码，即

- 遇到自定义函数，进入（如change_top）
- 遇到系统类、系统函数，不进入(如cout、sqrt)

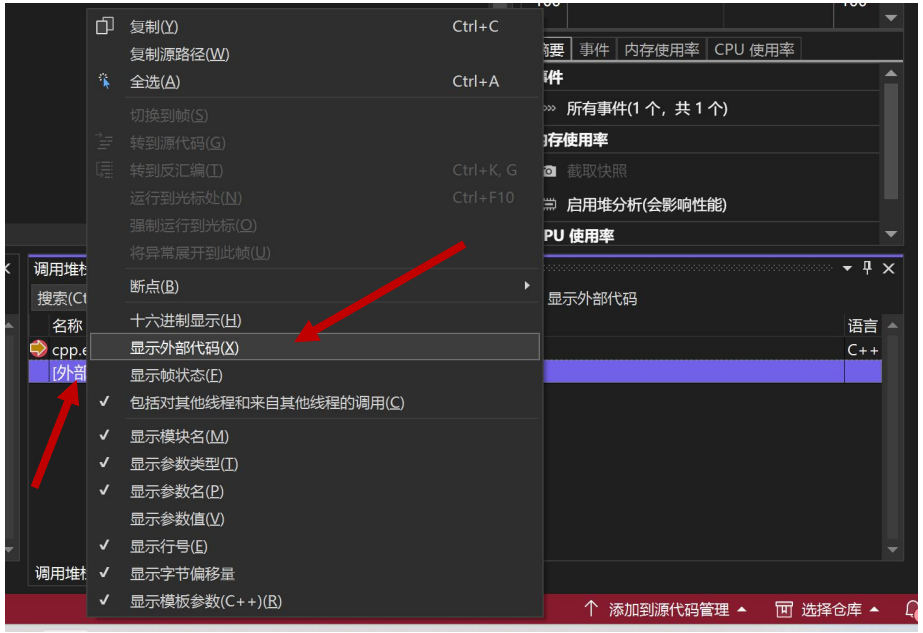


2.3： 系统类/函数内的执行（本页涉及知识点1.2）

“调用堆栈”窗口处一般不显示外部代码，可以单击“显示外部代码”或右击“[外部代码]”后，单击“显示外部代码”



or

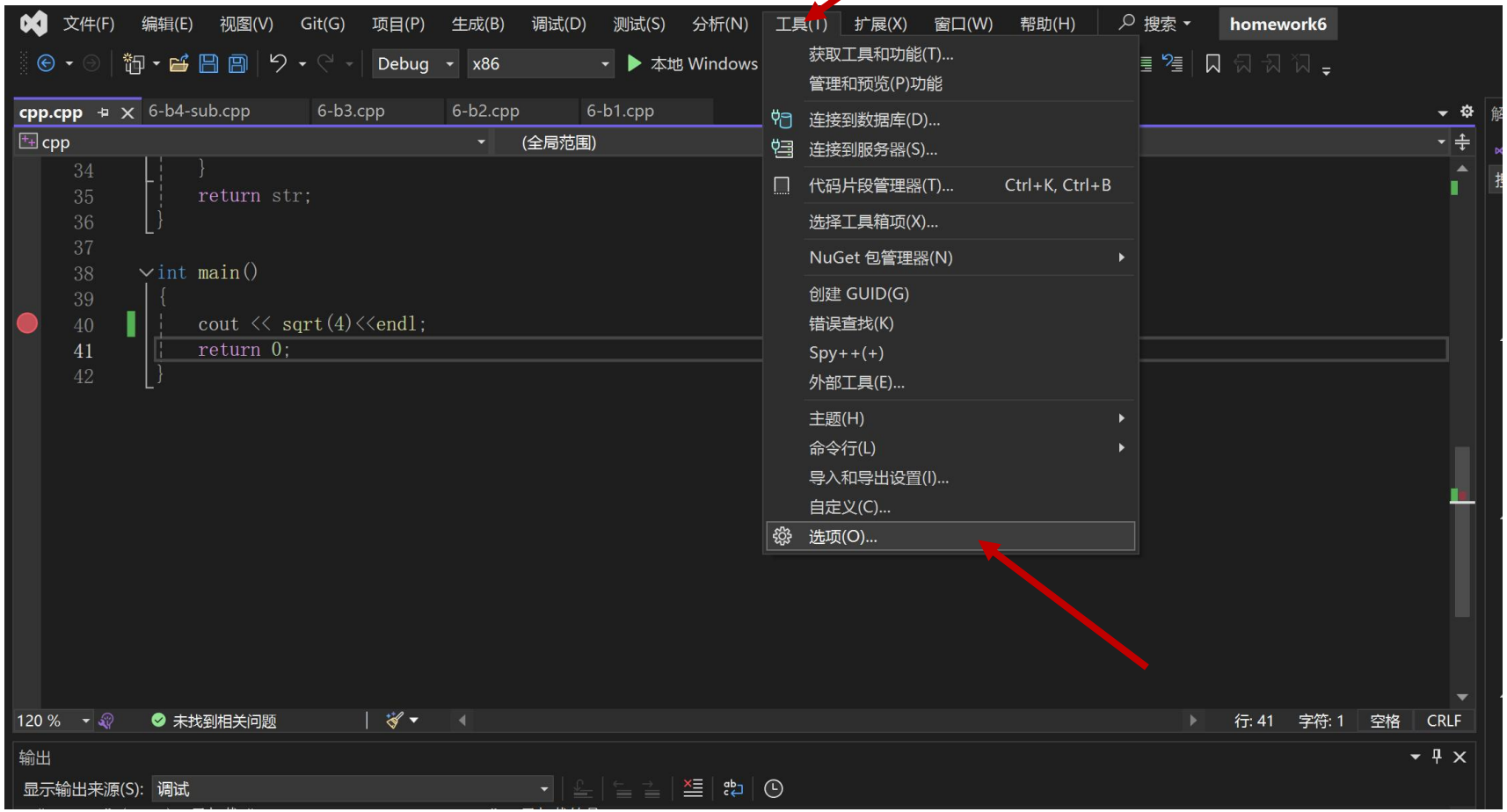




2.3： 系统类/函数内的执行（本页涉及知识点1.2）

为进到系统函数内部，需要选择启用外部代码

2.3-1依次单击“工具”-“选项”

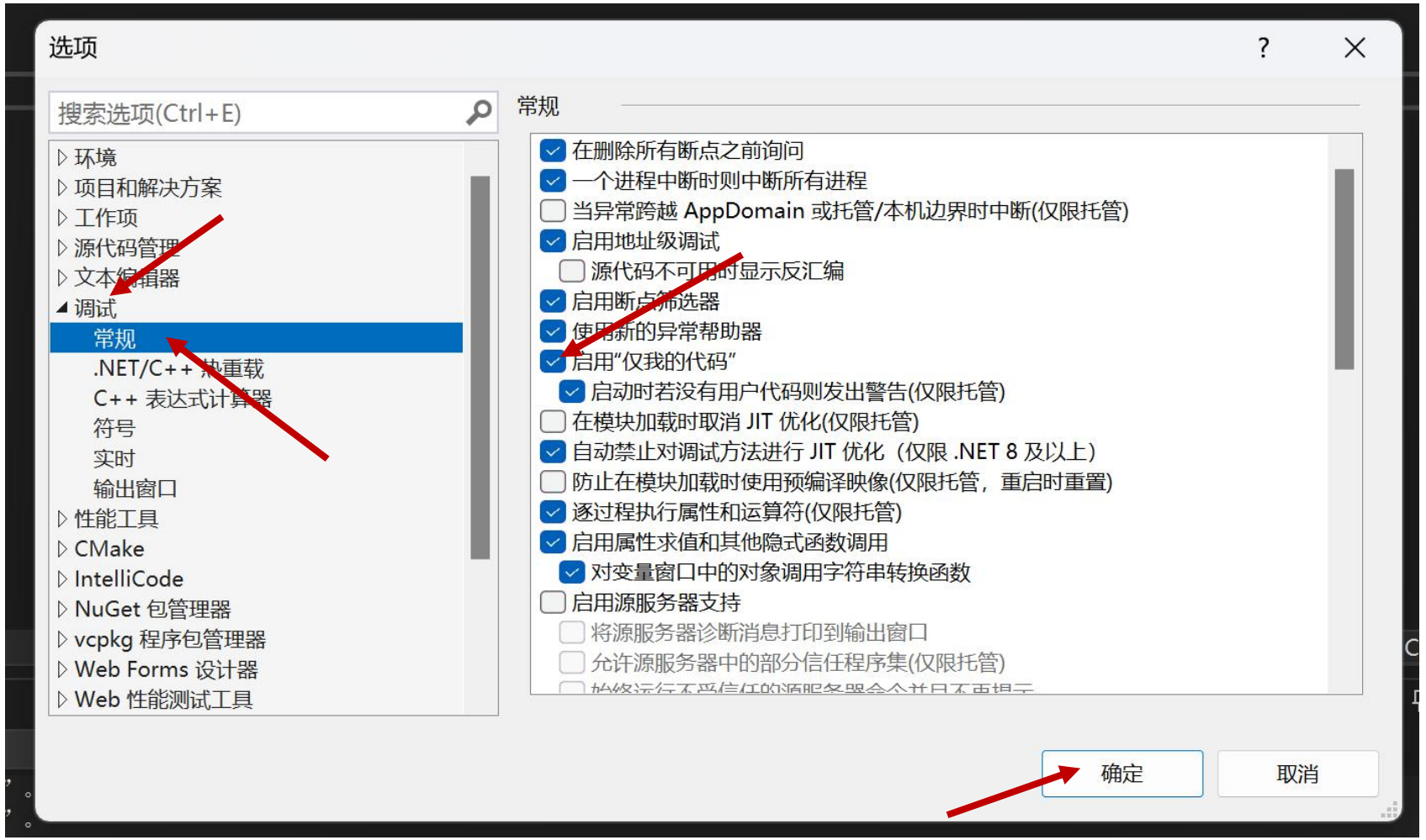


2.3：系统类/函数内的执行（本页涉及知识点1.2）

为进到系统函数内部，需要选择启用外部代码

2.3-2在弹出的窗口中，依次单击“调试”-“常规”

2.3-3取消勾选“启用‘仅我的代码’”,并单击“确定”



2.3：系统类/函数内的执行（本页涉及知识点1.2）

可以进入sqrt等系统函数，但是并没有进入cout内

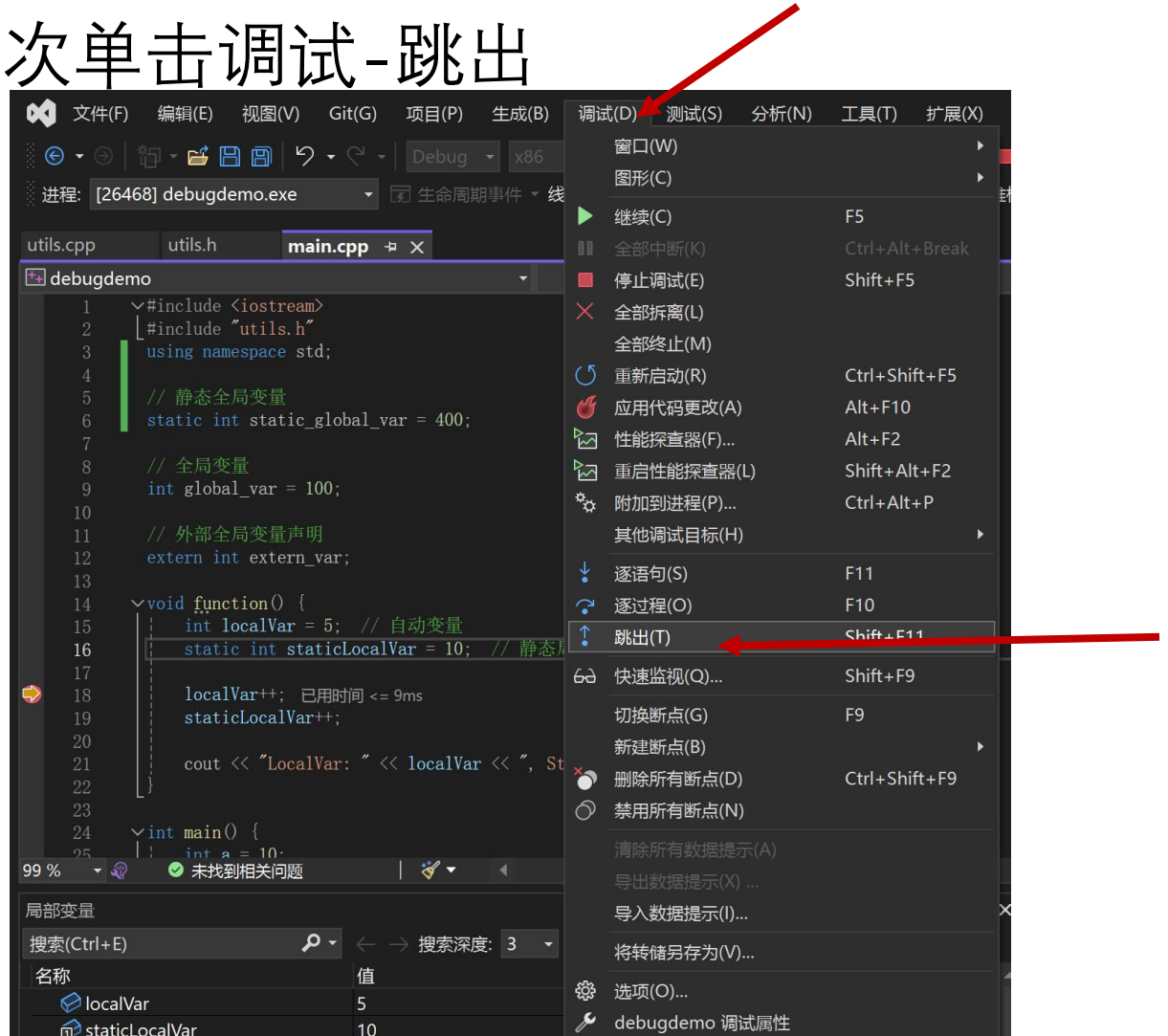
```
37
38  int main()
39  {
40      cout << sqrt(4)<<endl;
41      return 0;
42  }
```

按一次F11

```
672  // 3-arg hypot() is hand-crafted
673  _GENERIC_MATH2(pow)
674  _GENERIC_MATH1(sqrt)   已用时间 <= 20ms
675  _GENERIC_MATH1(erf)
676  _GENERIC_MATH1(erfc)
```

3.1跳出当前所在函数（本页涉及知识点1.4）

- 3.1进入tj_strlen内部，需要跳出
- 方法一：依次单击调试-跳出



3.1跳出当前所在函数（本页涉及知识点1.4）

- 3.1进入tj_strlen内部，需要跳出
- 方法二：单击蓝色向上箭头、
- 方法三：快捷键shift+F11



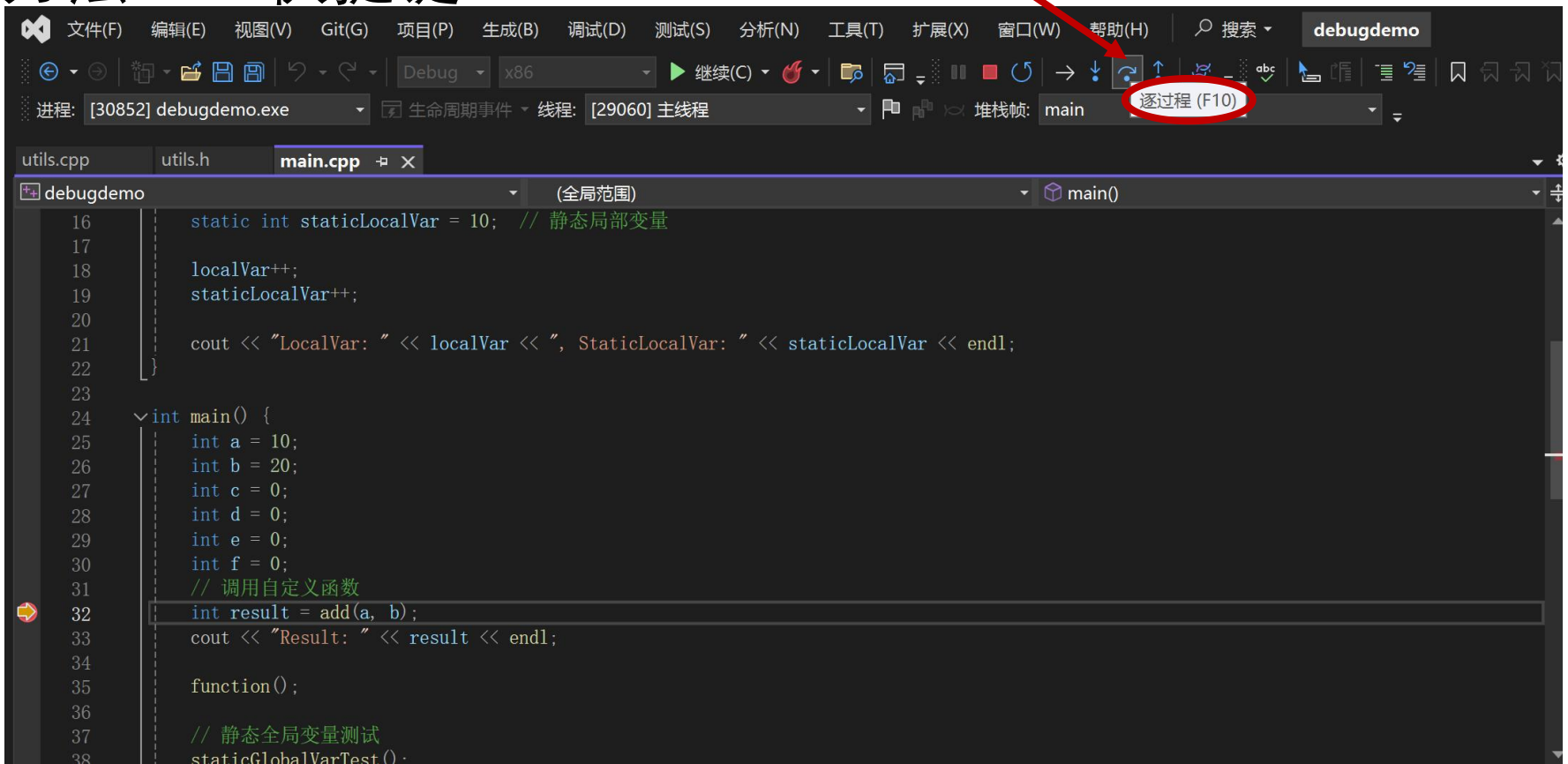
3.1跳出当前所在函数（本页涉及知识点1.4）

- 3.2成功跳出tj_strlen函数，返回现场



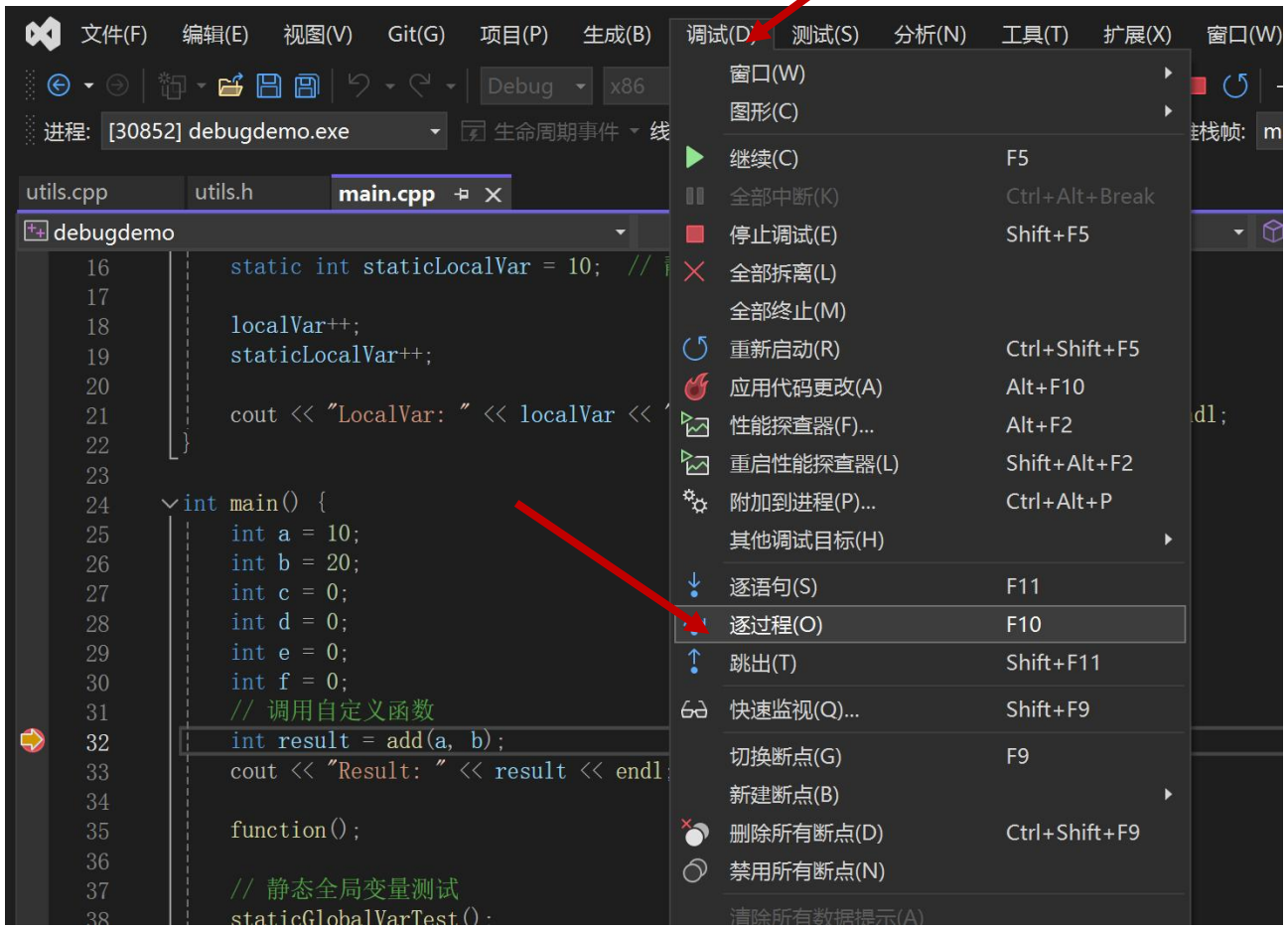
3.2跳过整个函数（本页涉及1.3,1.5）

- E.g.跳过add函数
- 方法一：单击蓝色箭头
- 方法二：快捷键F10



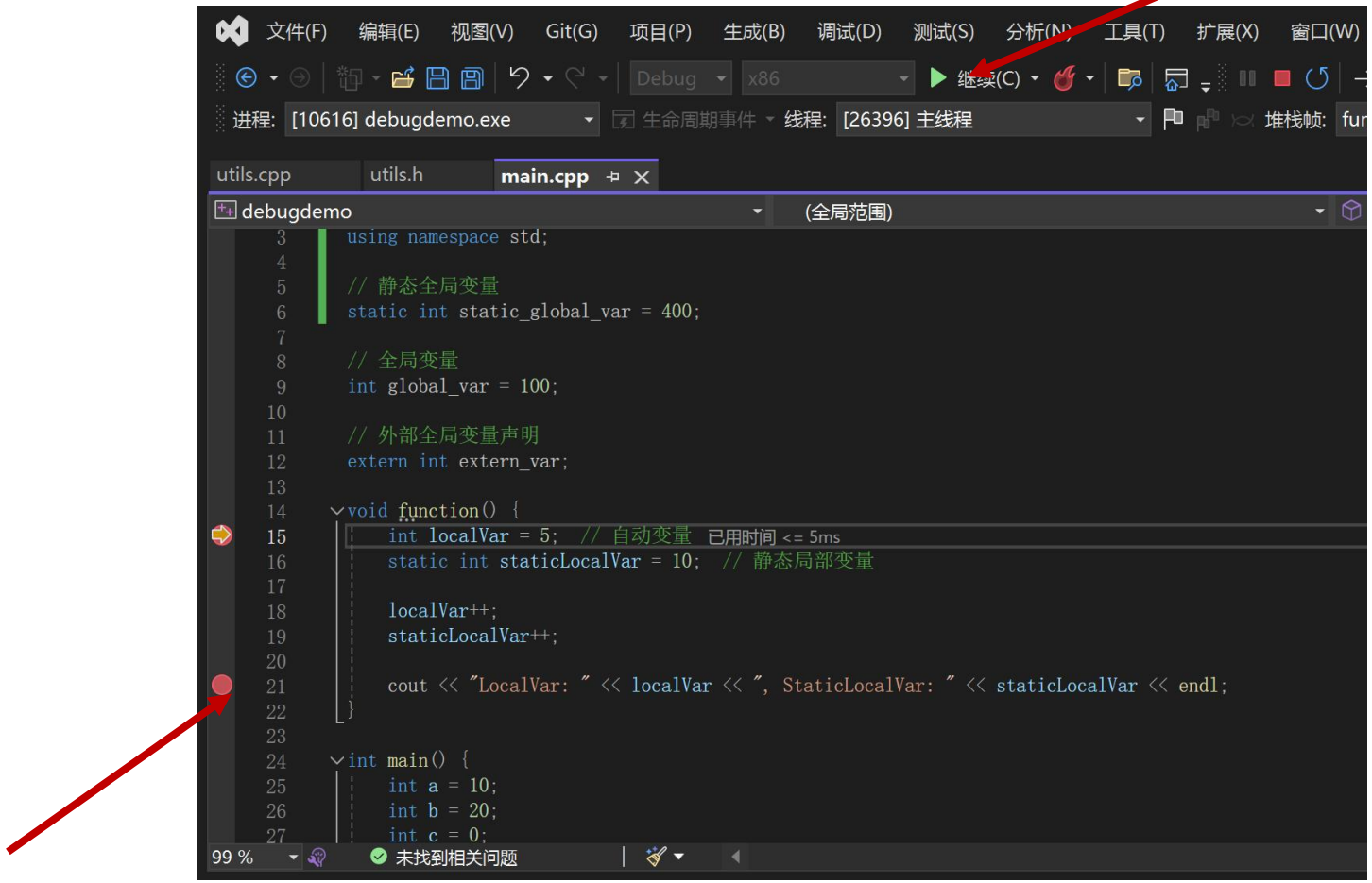
3.2跳过整个函数（本页涉及1.3,1.5）

- E.g.跳过add函数
- 方法三：调试>逐过程



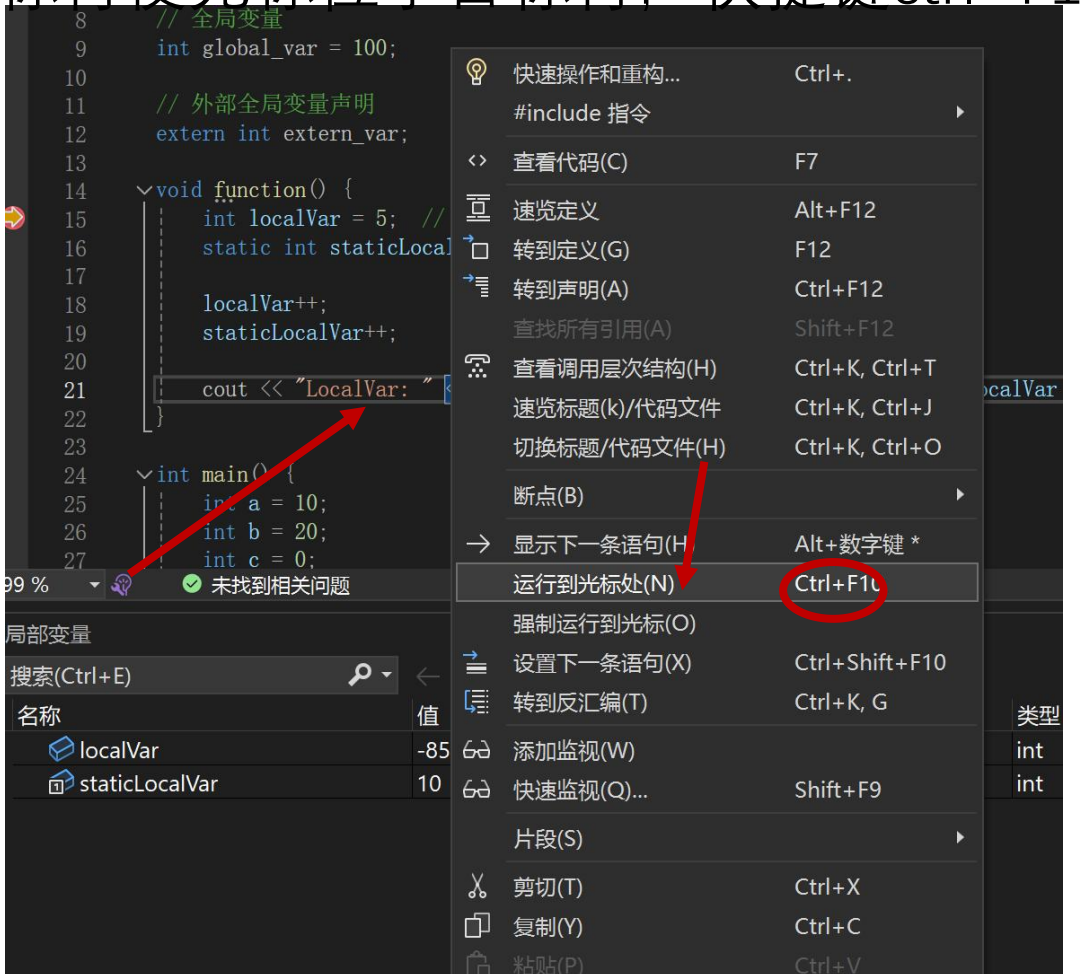
3.3跳到指定位置（本页涉及1.3，1.4，1.5,1.6）

- E.g.:从function的第一行跳到最后一行前
- 方法一：在目标处设断点后，点击“继续”



3.3跳到指定位置（本页涉及1.3，1.4，1.5,1.6）

- E.g.:从function的第一行跳到最后一行前
- 方法二：右键单击目标行的任意位置，选择“运行到光标处”
- 方法三：点击目标行使光标位于目标行，快捷键ctrl+F10



3.3跳到指定位置（本页涉及1.3，1.4，1.5,1.6）

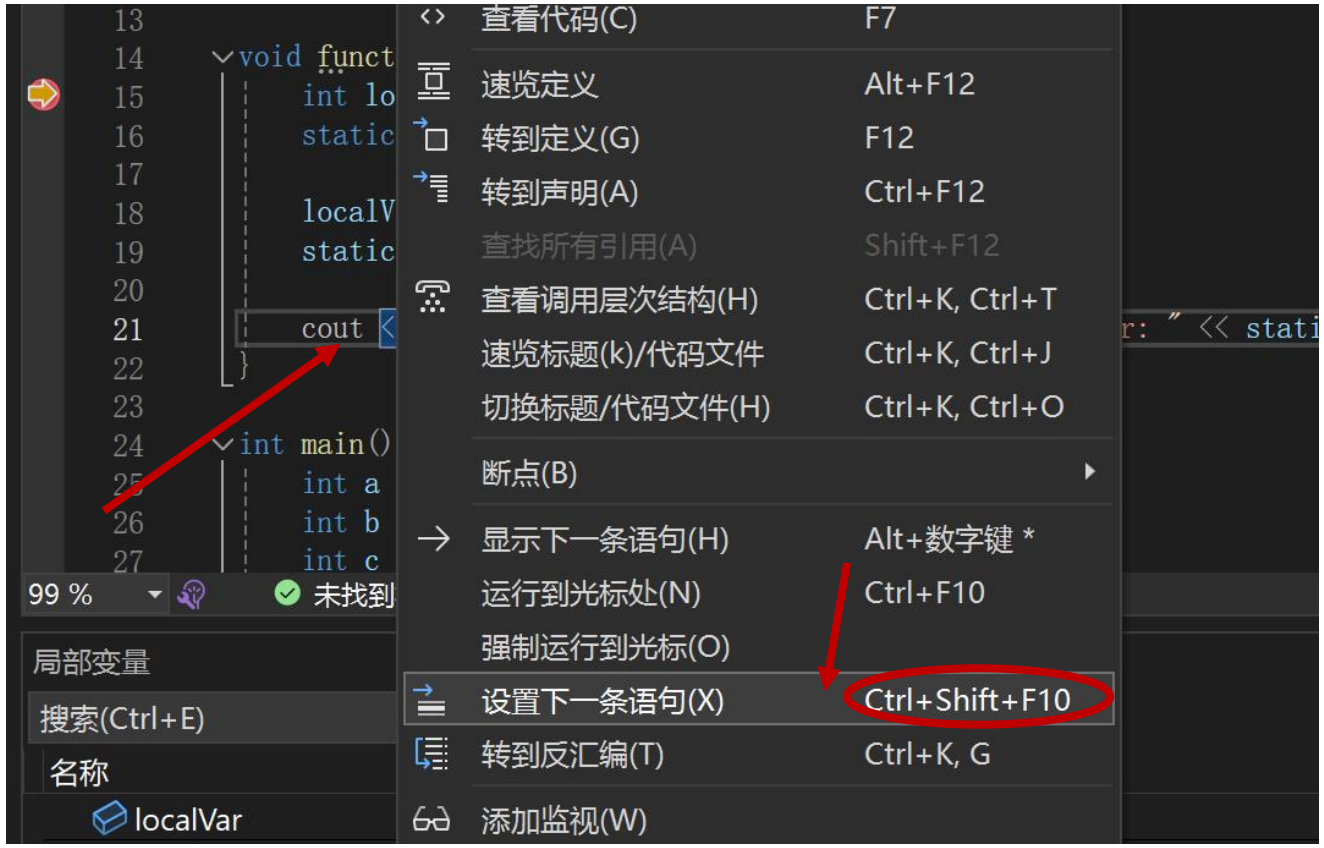
- E.g.:从function的第一行跳到最后一行前
- 方法四：鼠标悬停于目标代码上，将会出现绿三角图标，单击绿三角图标即可



3.3跳到指定位置（本页涉及1.3，1.4，1.5,1.6）

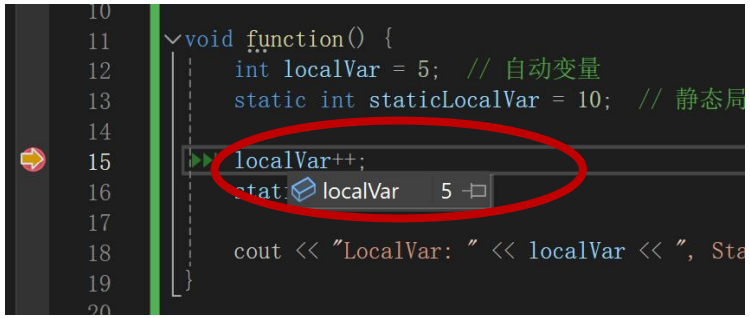
- E.g.:从function的第一行跳到最后一行前
- 方法四：右键单击目标行的任意位置，选择“设置下一条语句”
- 方法五：点击目标行使光标位于目标行，快捷键ctrl+shift+F10

注意：此方法会直接跳到目标行，不会运行当前行和目标行之间本该运行的内容！

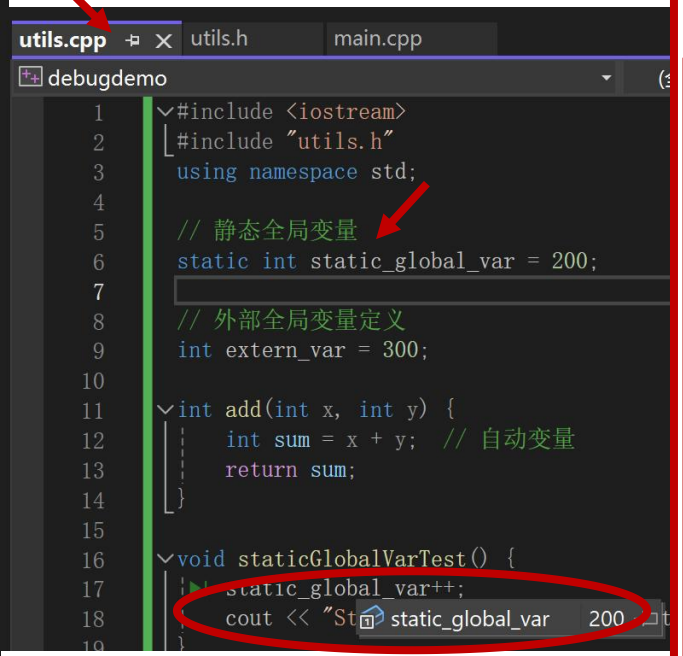


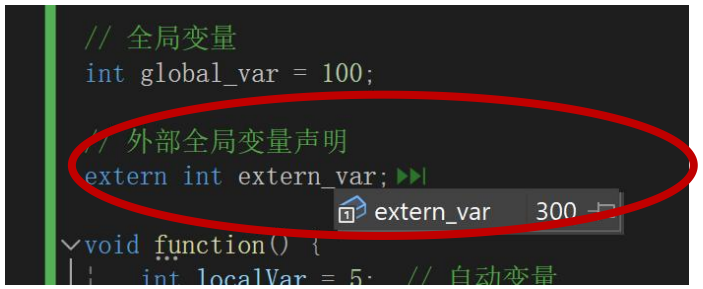
4.1查看变量：数据提示（本页涉及2.1,2.2,2.3,2.4，3.1）

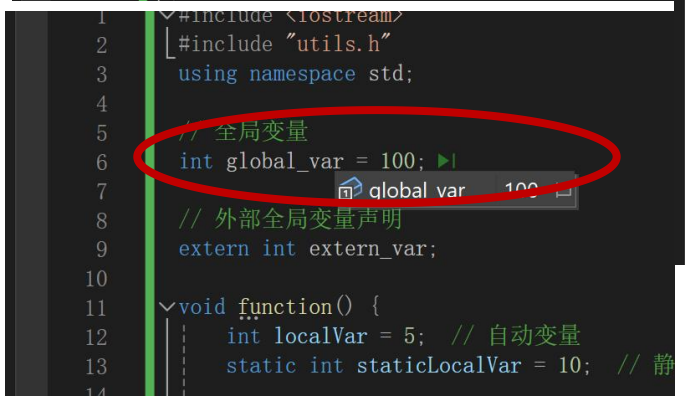
4.1.1调试暂停时，将鼠标悬停在**当前可访问**的变量上，即可查看它的值

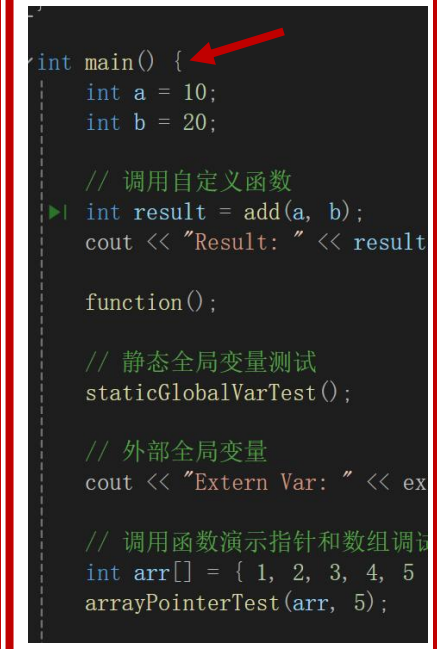












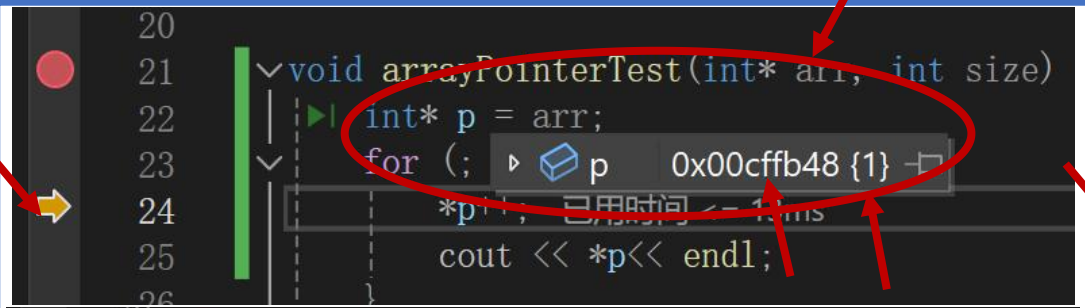
当箭头在function内，可访问**自动变量**localVar，**形参**，**静态局部**staticLocalVar,**[静态]全局变量**(包括**本源文件**中的golabal_var,**外部全局**extern_var，**另一个源文件**中的静态全局static_global_var)

当箭头在function内，不可访问main中的局部变量，故鼠标悬停在a，b上时不会显示它们的值

4.1查看变量：数据提示（本页涉及3.2,3.3，3.4，3.6）

• 4.1.2鼠标悬停法（数据提示法）也可以查看指针变量和数组变量

实参是一维数组名，形参是指针，其实和形参是一维数组没有区别，因为本质都是传递一维数组的地址



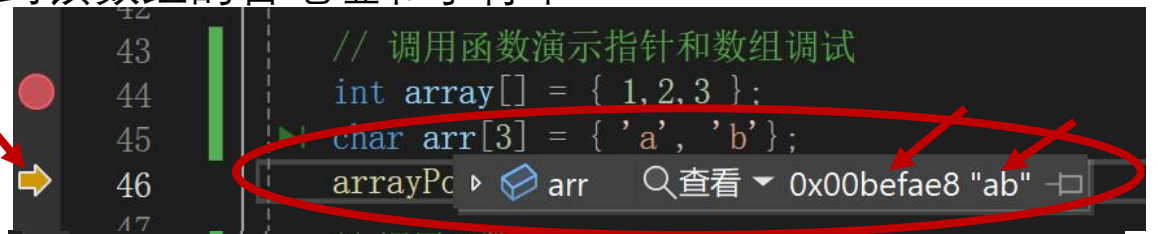


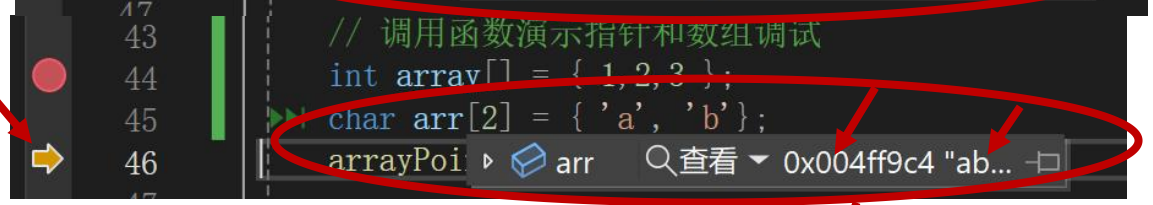
4.1.2.1当箭头在arrayPointerTest内，可访问指针变量p（可以是形参，也可以是函数内的），查看p和*p，p指向数字数组与指向简单数字，仅显示数组首元素的地址，p指向字符数组时，显示规则同右）



4.1.2.2当数组可访问时，鼠标悬停在数字型数组上，可以看到该数组的首地址和数组内的元素

4.1.2.3当数组可访问时，鼠标悬停在字符型数组上，可以看到该数组的首地址和字符串







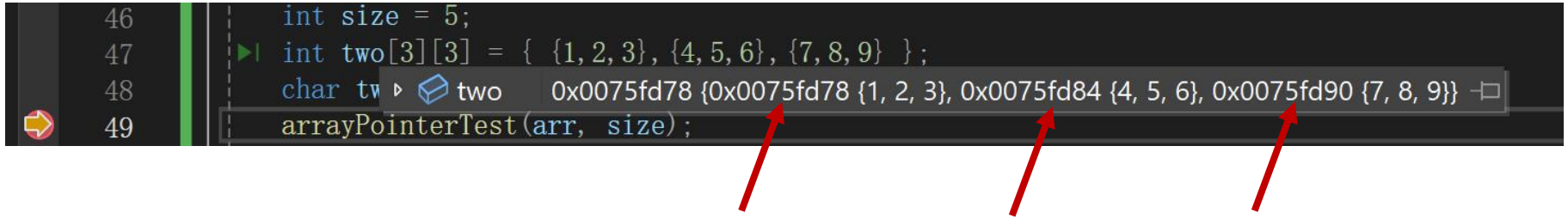
若无\0，则显示字符数组内用户定义的值以及“…”；若有\0（无论显式隐式），则显示字符数组内的值，不显示\0，无“…”

4.1查看变量：数据提示（本页涉及3.5）

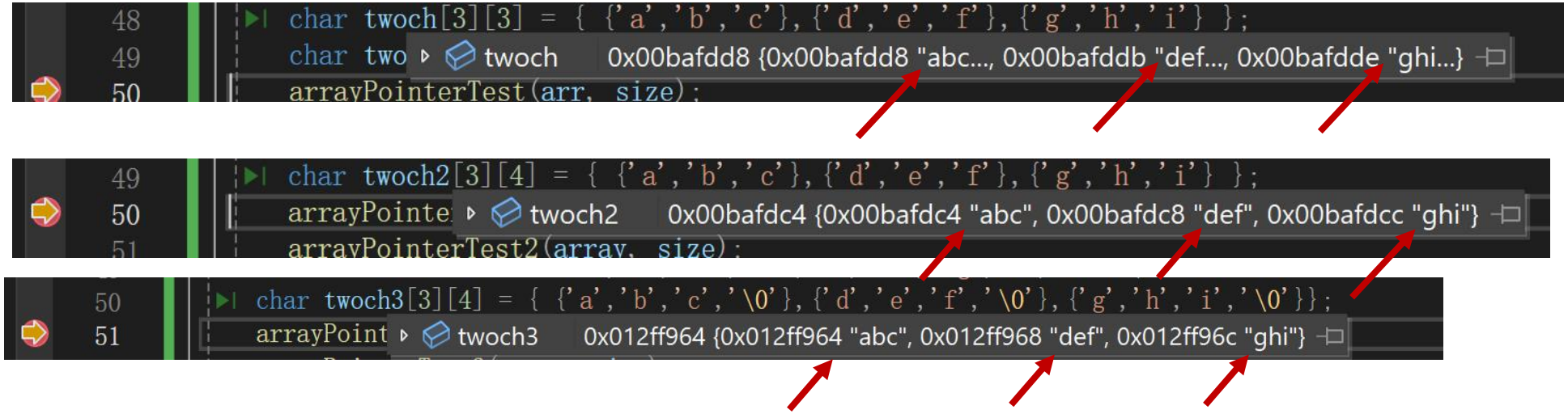
- 4.1.2鼠标悬停法（数据提示法）也可以查看指针变量和数组变量

二维数组相当于以一维数组为元素的一维数组

4.1.2.4数组可访问时，查看二维数字型数组，会显示二维数组首地址，也会在每一个一维数组元素前显示其首地址

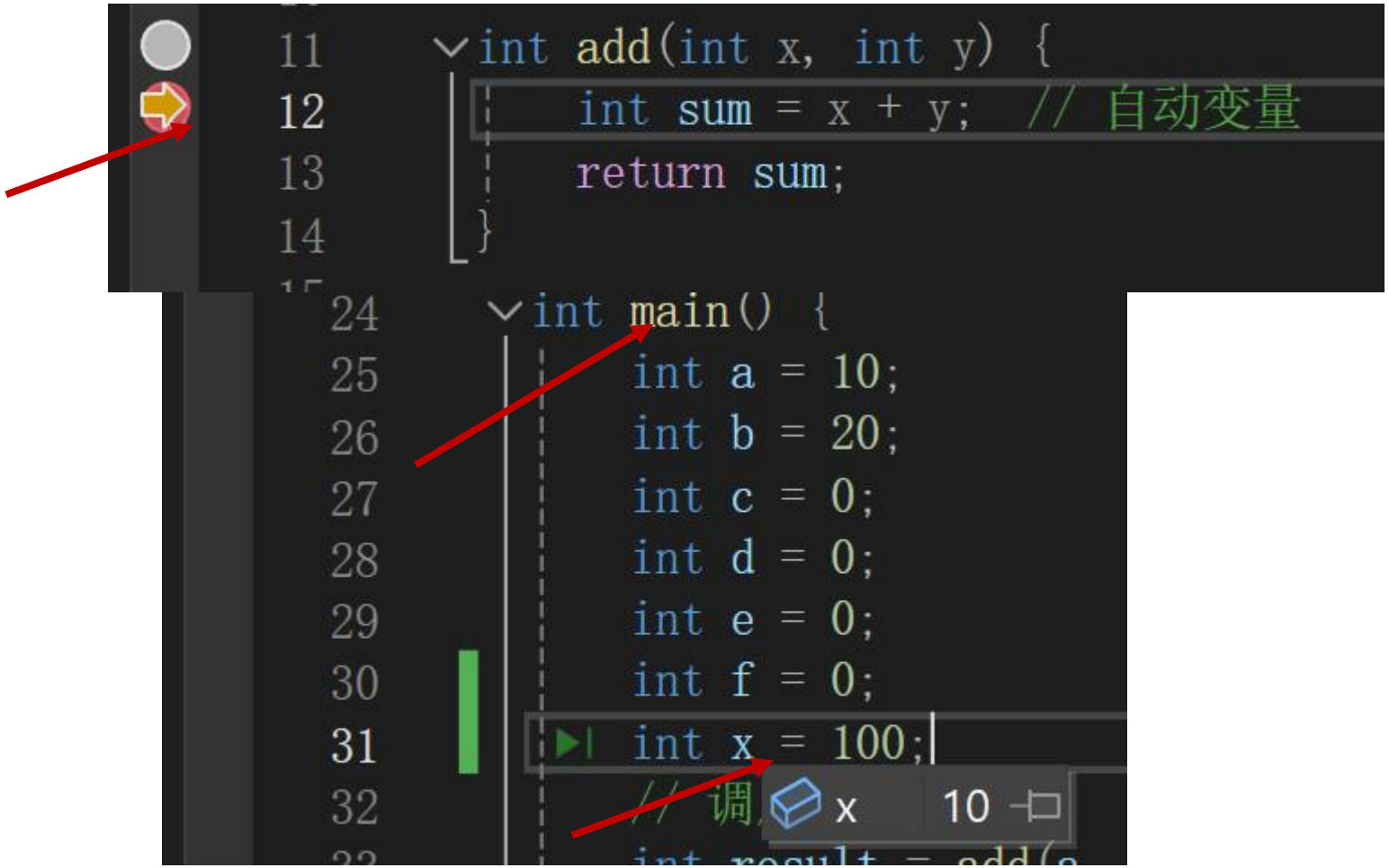


4.1.2.5数组可访问时，查看二维字符型数组，会显示二维数组首地址，也会在每一个一维数组元素前显示其首地址。每一个一维数组的元素如何显示，规则同4.1.2.3



4.1查看变量：数据提示

- 4.1.3鼠标悬停在不可访问的变量上，若变量与可访问变量同名，则可以查看，但实际查看的还是可访问变量



黄色箭头在add函数内，鼠标悬停在main中的x上，显示的x是add函数内的x

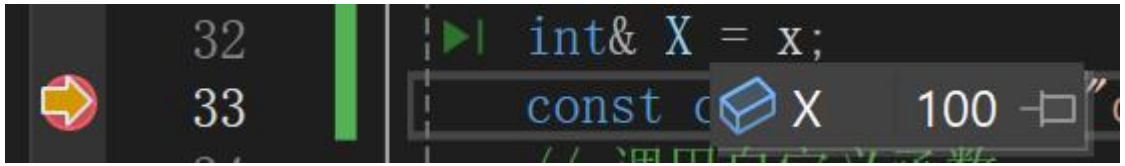
鼠标悬停在一个变量上，只是看这个变量的名字背后的那个量是否可访问，与该变量在程序中的位置无关，因此同名时会发生混淆

4.1查看变量：数据提示（本页涉及3.7，3.8）

- 4.1.5鼠标悬停于指向字符串常量的指针变量，可以看到无名字符串常量的地址



- 4.1.6鼠标悬停于引用，就如同悬停于被引用的那个变量一样。



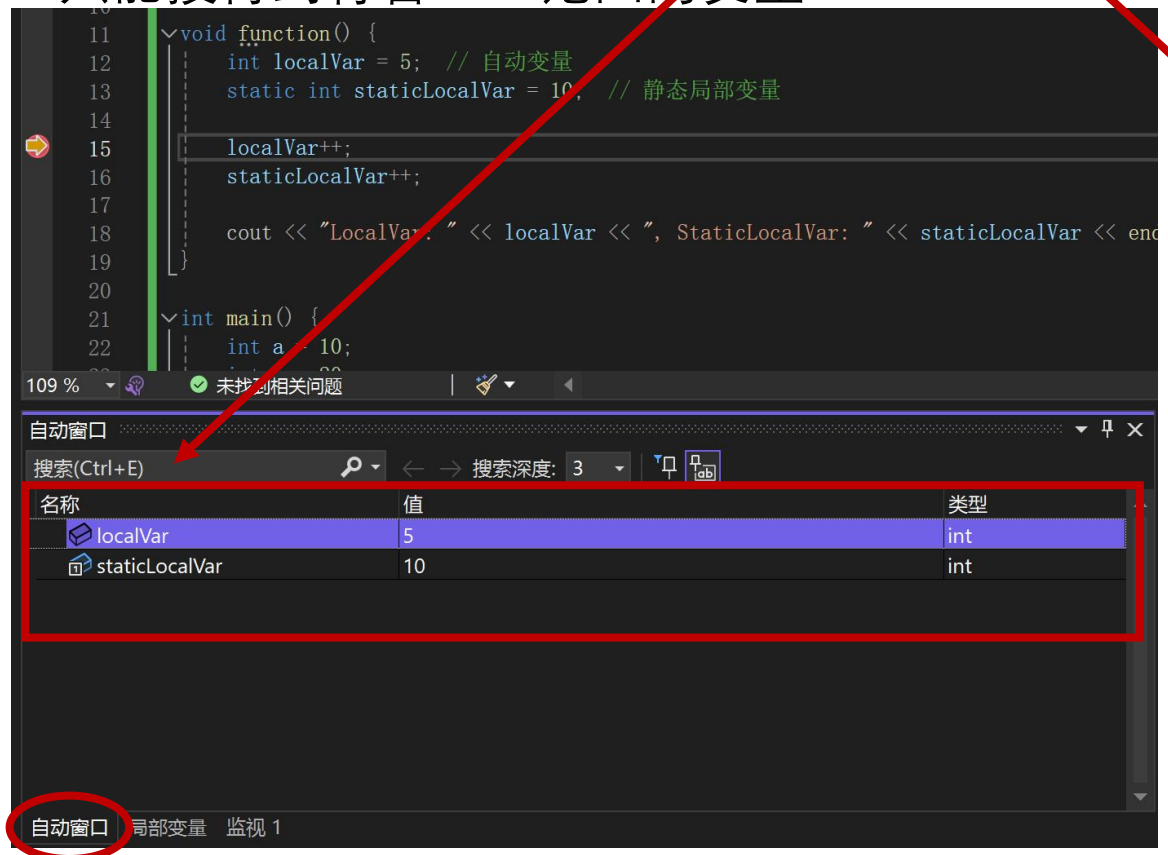
4.2查看变量：自动窗口（本页涉及2.1，2.2，3.1）

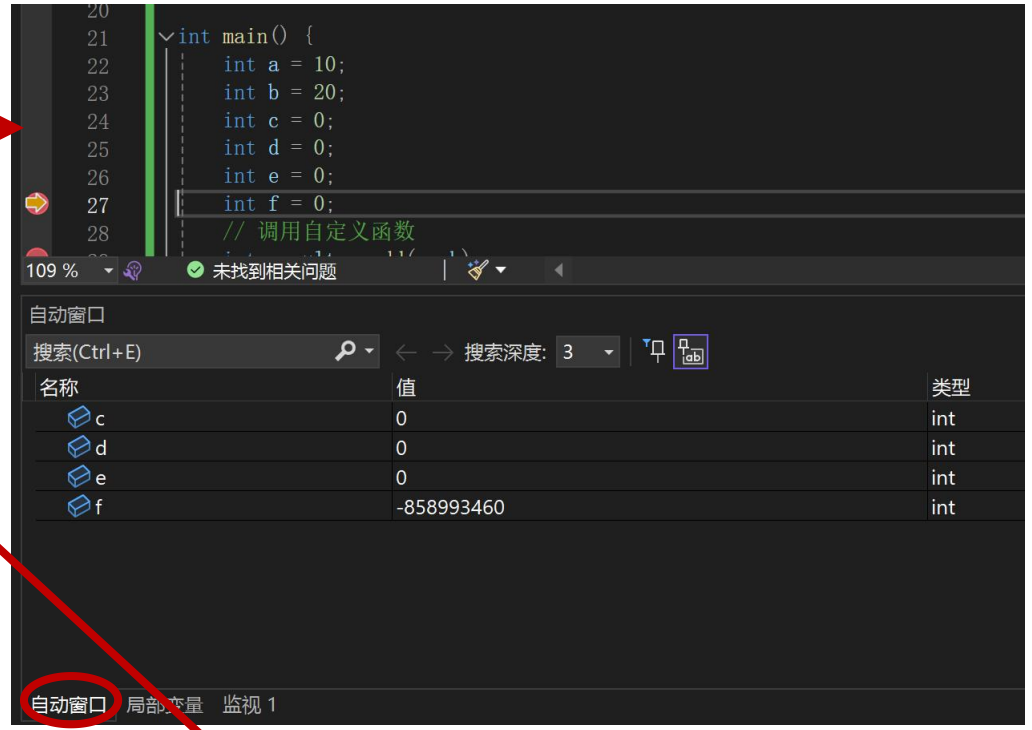
4.2.1单击左下角“自动窗口”，即可查看当前行及前三行的变量

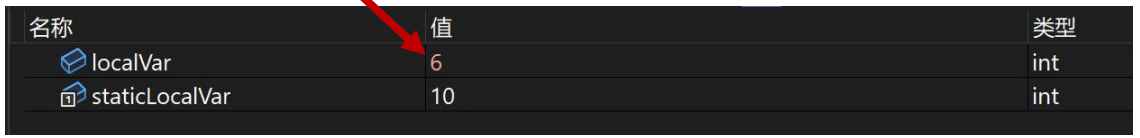
4.2.2从左到右依次显示“名称”、“值”、“类型”

4.2.3若调试时变量的值变化，则会标红

4.2.4若找不到变量，可以搜索（Ctrl+E），只能搜得到符合4.2.1范围的变量

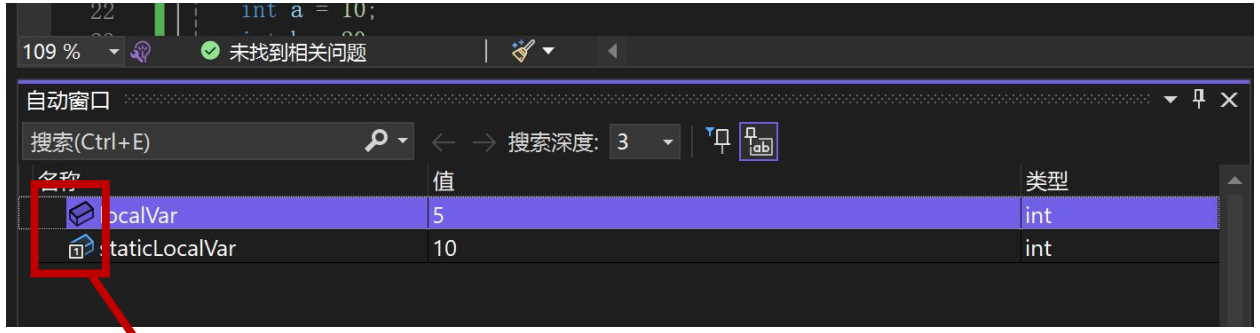






F11后，
localVar++

4.2查看变量：自动窗口（本页涉及2.1，2.2，3.1）



不同的量有不同的图标，在此给出表格（csdn下载）

图标	说明	图标	说明
{}	命名空间		方法或函数
	类		运算符
	接口		属性
	结构		字段或变量
	Union		Event
	Enum		常量
	TypeDef		枚举项
	模块		映射项
	扩展方法		外部声明
	委托		错误
	异常	<T>	模板
	映射		未知
	类型转发		

https://blog.csdn.net/wwlcsdn000

4.2查看变量：自动窗口（本页涉及3.3,3.5）

```
51    char arr[5] = { 'a', 'b','c','d','\0'};
52    int size = 5;
53    int array[5] = { 1,2,3 ,4,5 };
54    char twoch[3][4] = { {'a','b','c','a'},{'d','e','f','a'},{'g','h','i','a'}
55    char twoch2[3][4] = { {'a','b','c'},{'d','e','f'},{'g','h','i'} };
56    char twoch3[3][4] = { {'a','b','c','\0'},{'d','e','f','\0'},{'g','h','i','
57    arrayPointerTest(arr, size);
58    arrayPointerTest2(array, size);
```

117 % 未找到相关问题

自动窗口

搜索(Ctrl+E) 搜索深度: 3

名称	值	类型
arr	0x00cffd34 "abcd" 查看	char[5]
size	5	int
twoch	0x00cffcf8 {0x00cffcf8 "烫烫..., 0x00cffcfc "烫烫..., 0x00cf...	char[3][4]

自动窗口

搜索(Ctrl+E) 搜索深度: 3

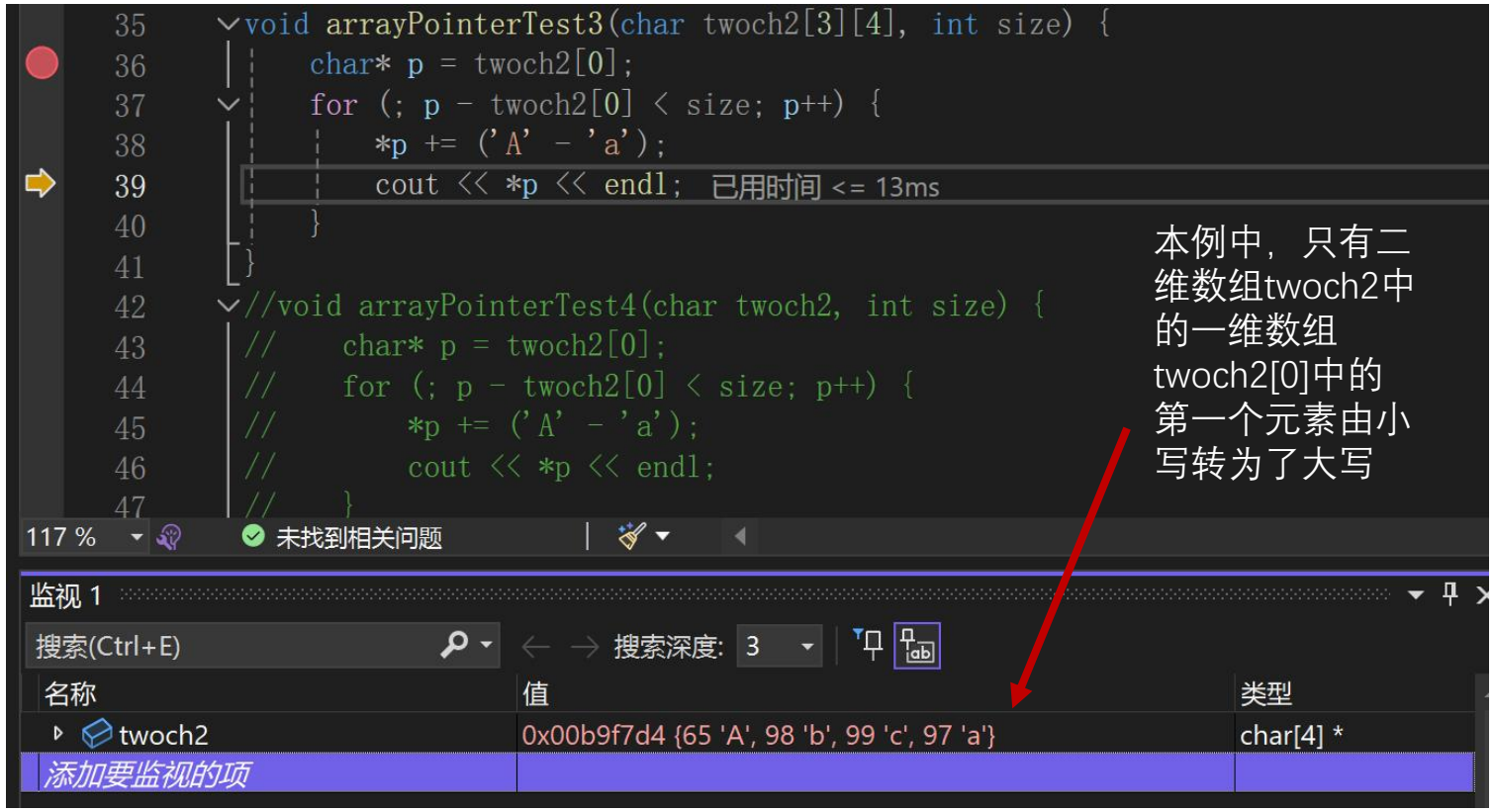
名称	值	类型
arr	0x00cffd34 "abcd" 查看	char[5]
[0]	97 'a'	char
[1]	98 'b'	char
[2]	99 'c'	char
[3]	100 'd'	char
[4]	0 '\0'	char
size	5	int
twoch	0x00cffcf8 {0x00cffcf8 "烫烫..., 0x00cffcfc "烫烫..., 0x00cf...	char[3][4]
[0]	0x00cffcf8 "烫烫...	char[4]
[1]	0x00cffcfc "烫烫...	char[4]
[2]	0x00cffd00 "烫烫...	char[4]

twoch	0x00cffcf8 {0x00cffcf8 "烫烫..., 0x00cffcfc "烫烫..., 0x00cf...	char[3][4]
[0]	0x00cffcf8 "烫烫...	char[4]
[0]	-52 '?'	char
[1]	-52 '?'	char
[2]	-52 '?'	char
[3]	-52 '?'	char
[1]	0x00cffcfc "烫烫...	char[4]
[2]	0x00cffd00 "烫烫...	char[4]

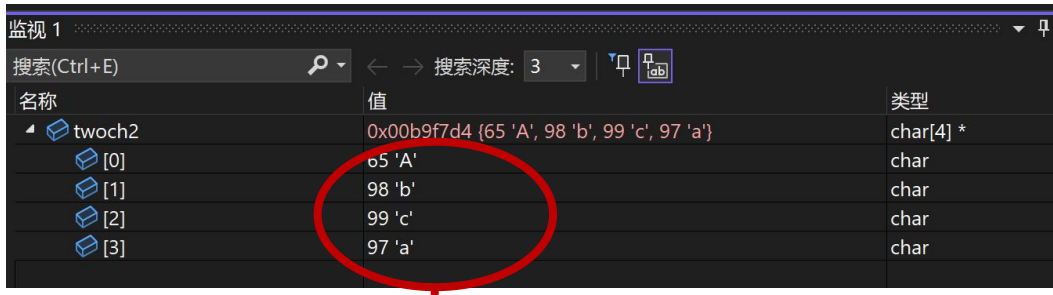
4.2.5数组：点击左边的小三角，可以查看数组内元素。若为二维/多维数组，则看到的是数组元素，可以进一步点开三角查看内部元素

4.2查看变量：自动窗口（本页涉及3.3,3.5）

4.2.5数组：调试过程中，数组中即使只有一个变量变化，整个数组也会标红，若为二维/多维数组，则只会标红有变化的那个一维数组



本例中，只有二维数组twoch2中的一维数组twoch2[0]中的第一个元素由小写转为了大写



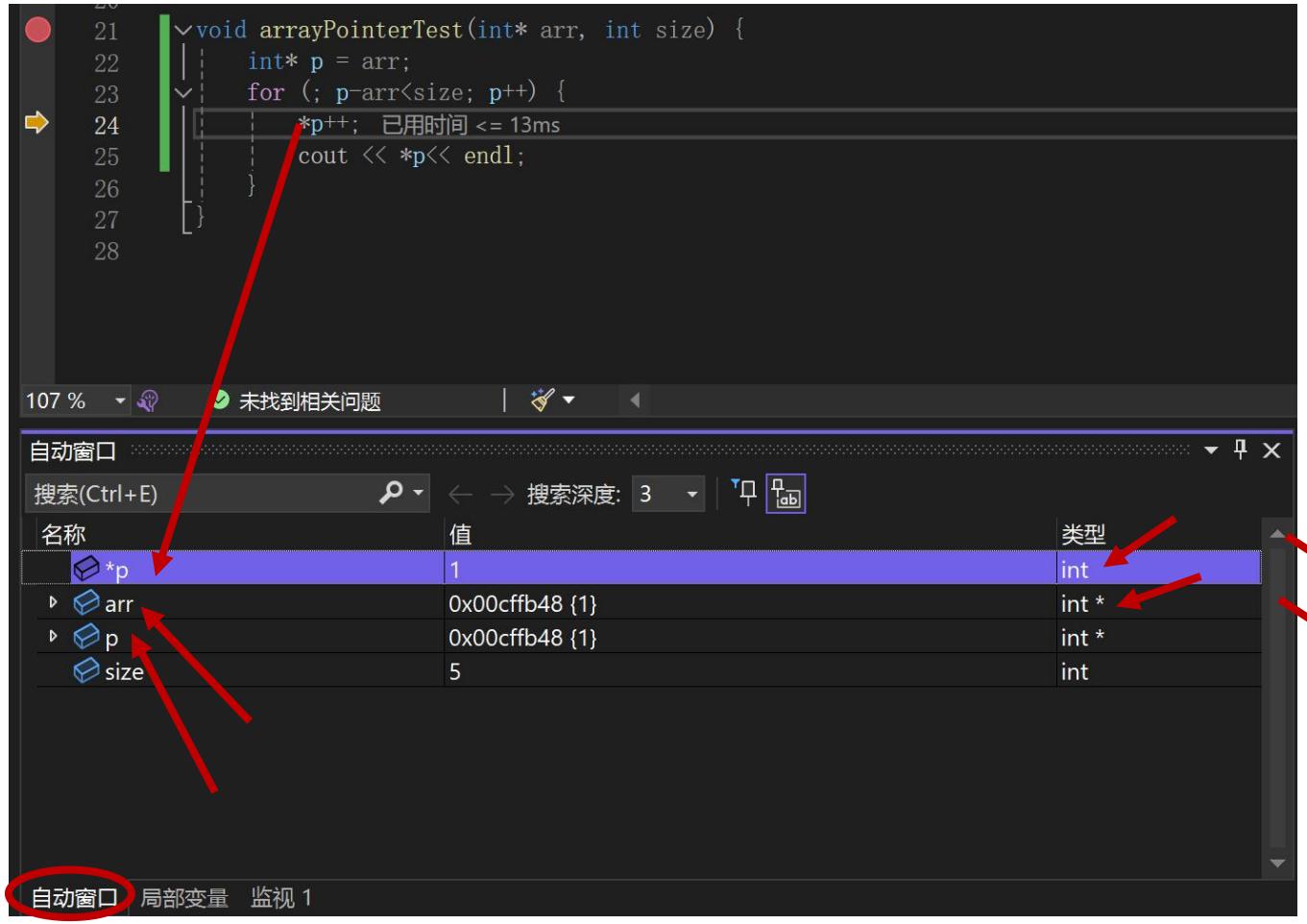
按小三角展开后，每个元素均未标红
看不出是具体是哪个元素变化了

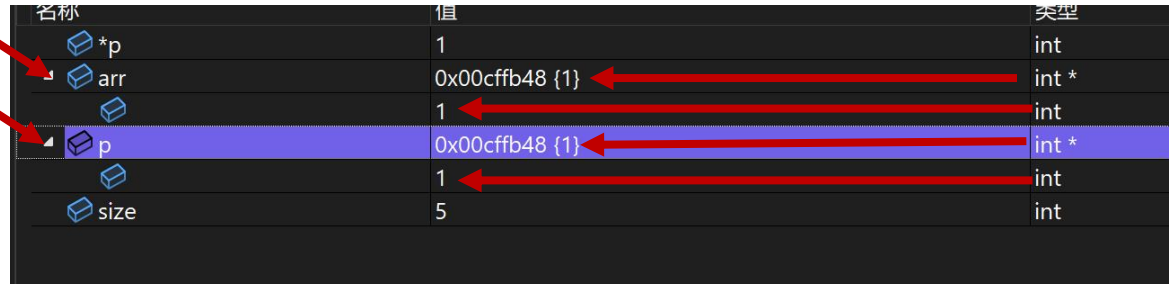
4.2查看变量：自动窗口（本页涉及3.2）

4.2.6查看指针变量：指向简单变量

若源代码有“*”+“指针变量名”的形式出现，则可以直接看到*p，类型为基类型

若有指针变量/地址直接出现，则可以看到该指针变量的值（地址），类型为基类型+*，单击扩展箭头，可以看到指针变量所指的地址上的变量的值，类型为基类型

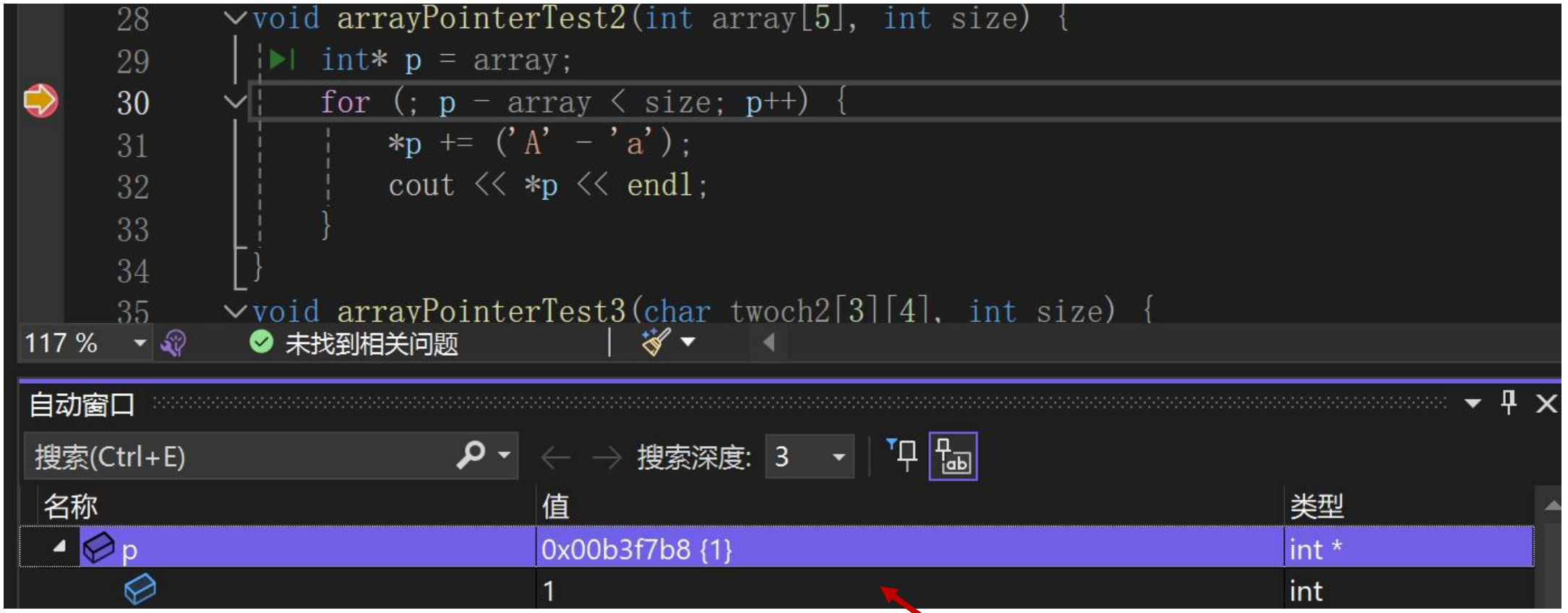




4.2查看变量：自动窗口（本页涉及3.4）

4.2.7查看指针变量：指向数组

指针指向数字型数组，则与指针指向数组中的第一个简单变量无异（因为数组的名字相当于数组首地址）

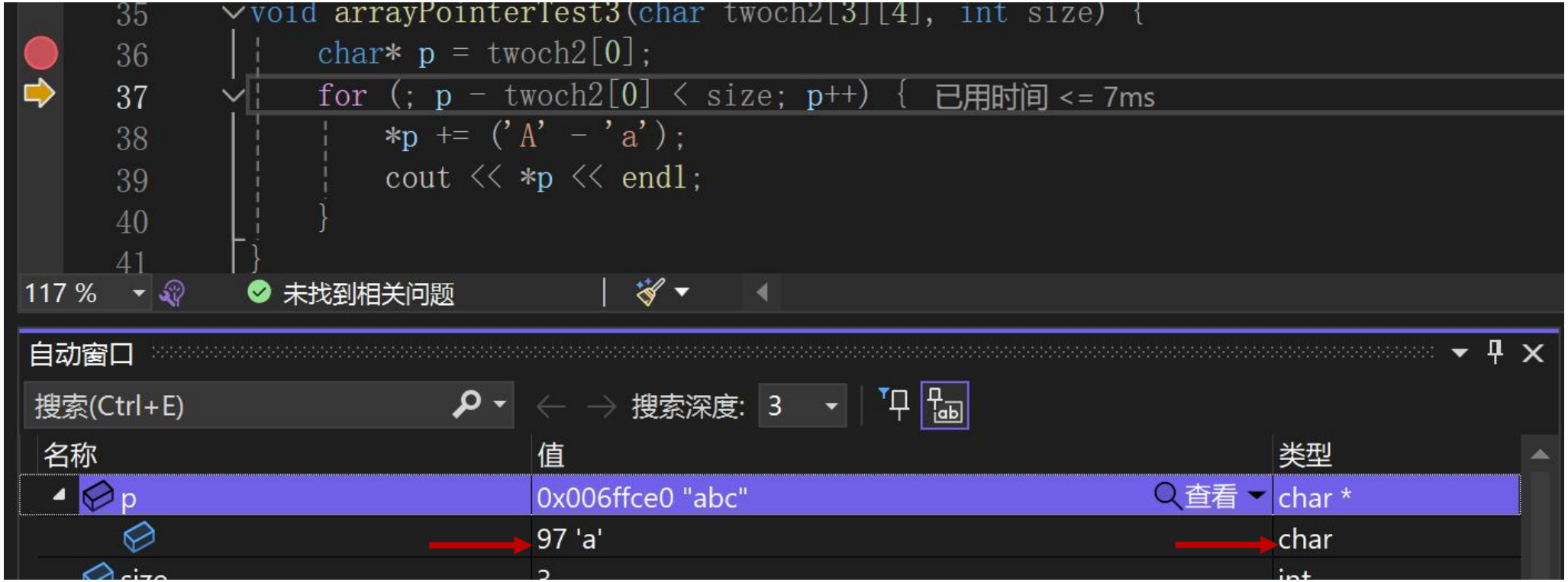


4.2查看变量：自动窗口（本页涉及3.4，3.5）

4.2.7查看指针变量：指向数组

指针指向一维字符型数组/二维字符型数组带一个下标名（等于一维数组），值那一栏的显示与数据提示法中鼠标悬停的显示规则相同

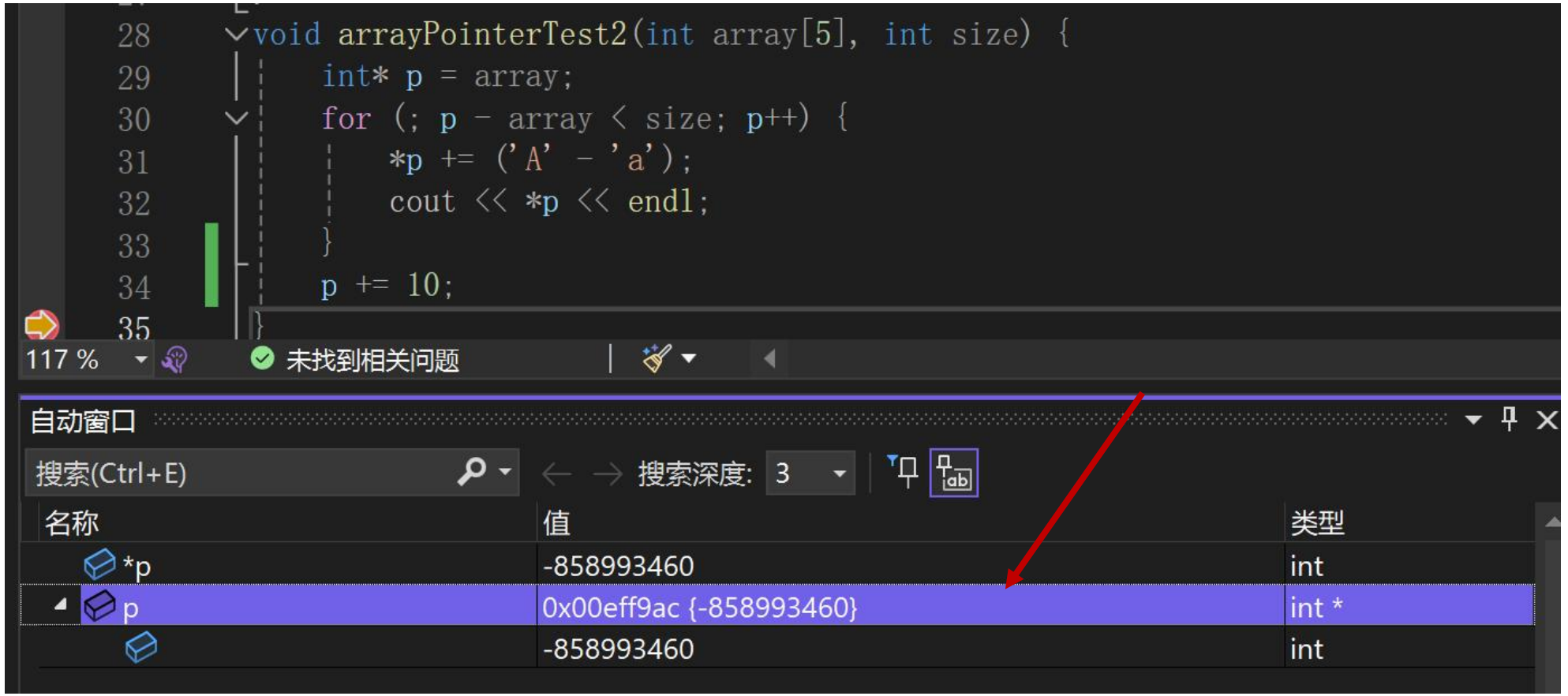
点开小三角，指针所指向的数据还是单个简单变量，并非整个数组



4.2查看变量：自动窗口（本页涉及3.9）

4.2.8查看指针变量：指针越界

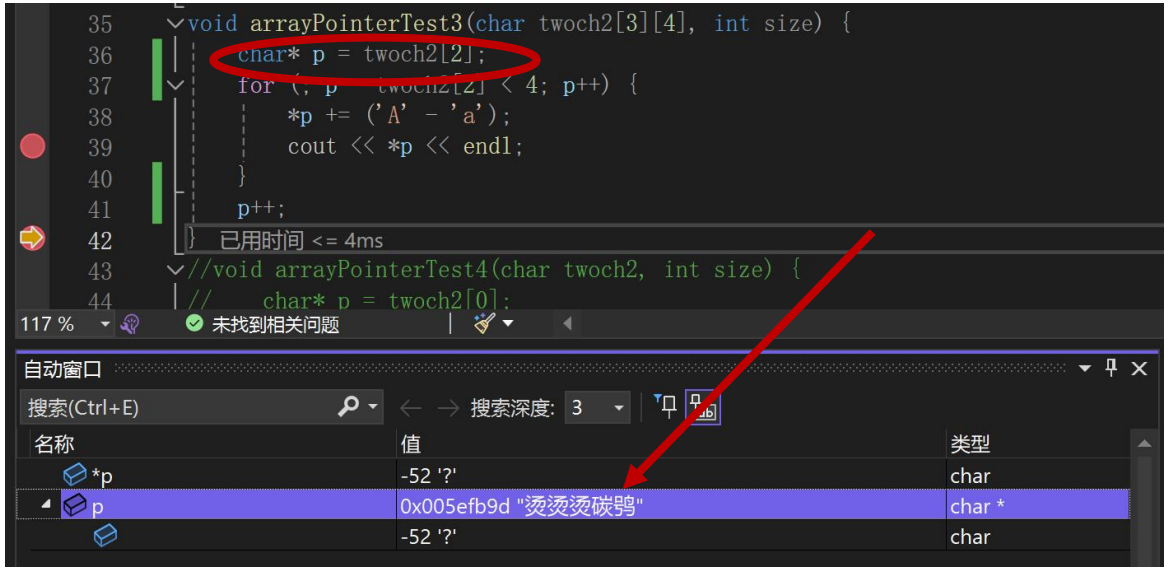
指向数字型数组的指针越界，相当于指向了一个大小内容均未知数字（非数组）（因为数组的名字本质是首元素地址）

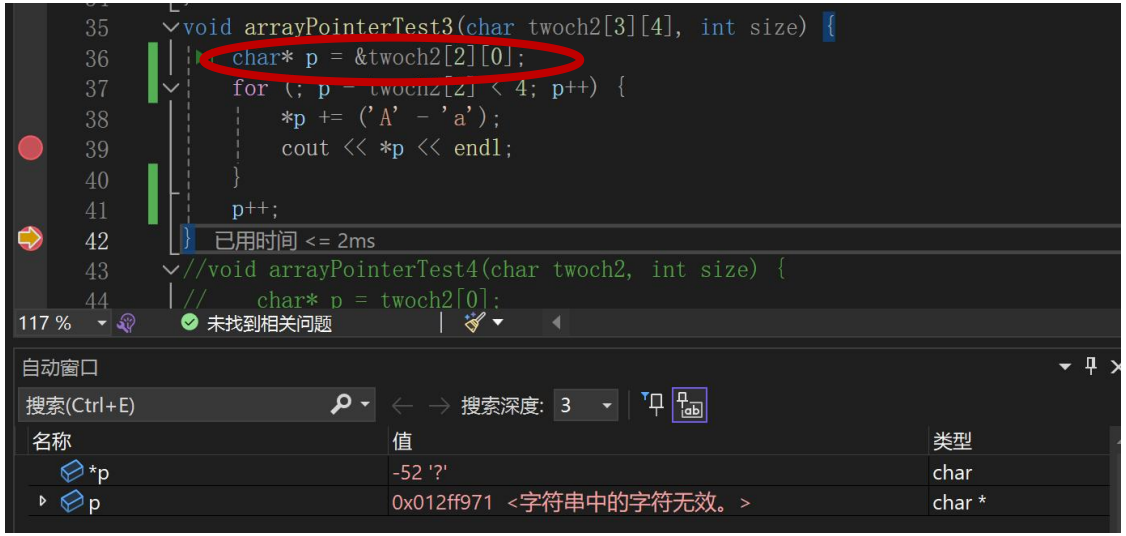


4.2查看变量：自动窗口（本页涉及3.9）

4.2.8查看指针变量：指针越界

指向字符数组或字符变量的指针越界，相当于指向了一个大小内容均未知的字符数组（因为字符数组的名字本质是首元素地址）

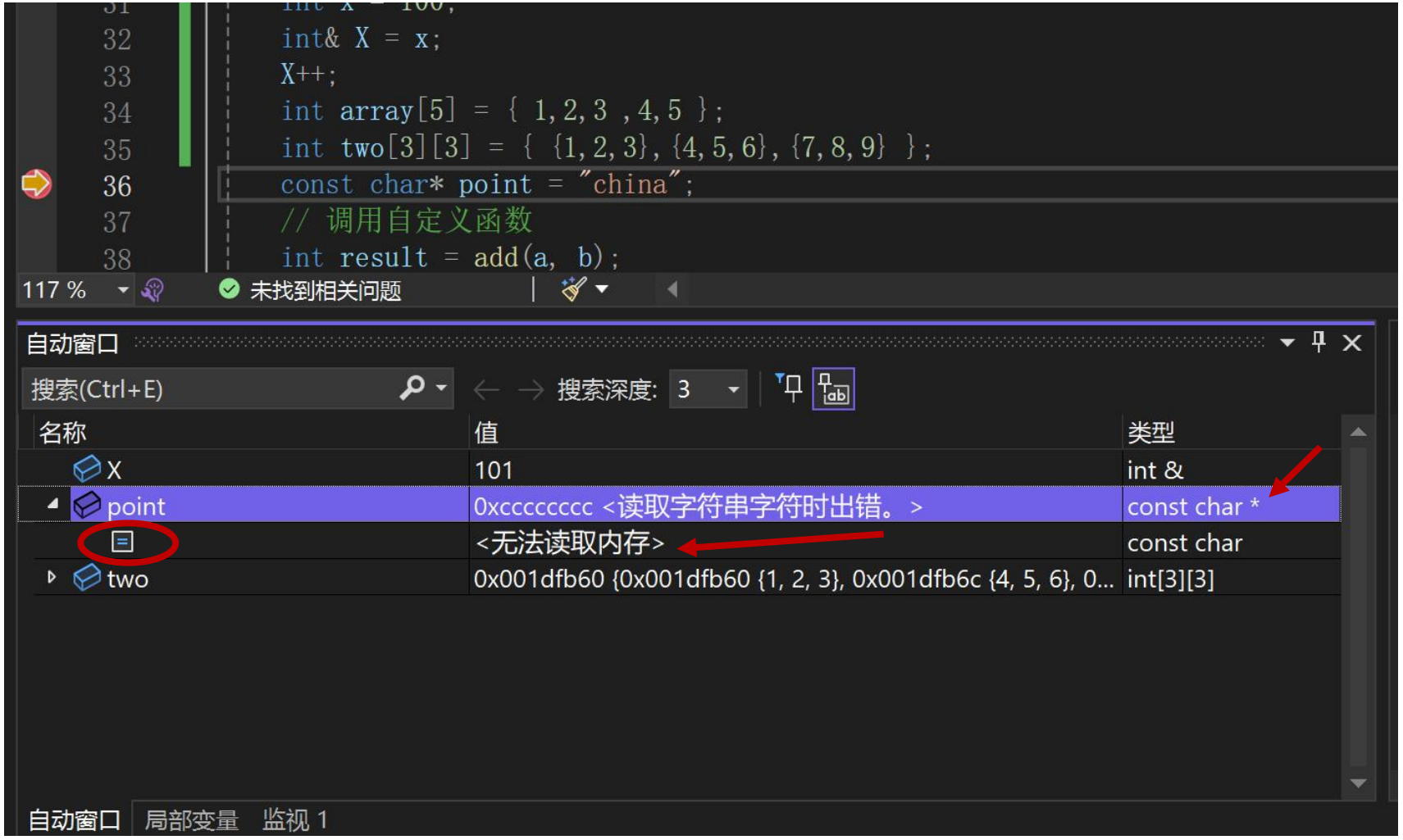




4.2查看变量：自动窗口（本页涉及3.7,3.8）

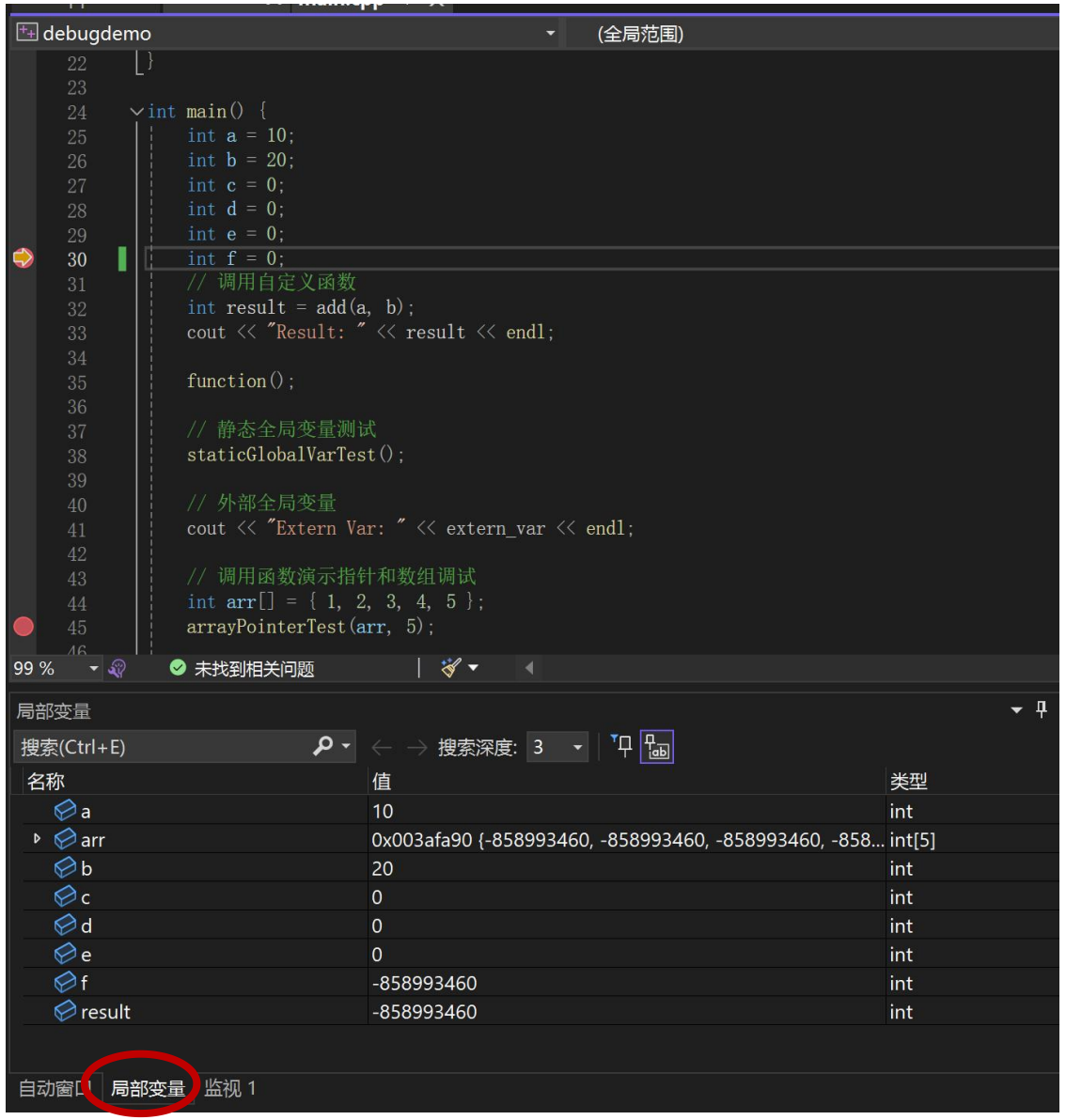
4.2.9查看引用：类型为：被引用的类型+“&”，值为被引用的值

4.2.10查看指向字符常量的指针变量：值的显示与数据提示法看到的一样，是地址+“读取字符串字符时出错”，类型为const char*，展开后图标表示常量，类型为const char，值显示“无法读取内存”



此处为何没有显示array?

4.3查看变量：局部变量窗口（本页涉及2.1，2.2，3.1,3.2）



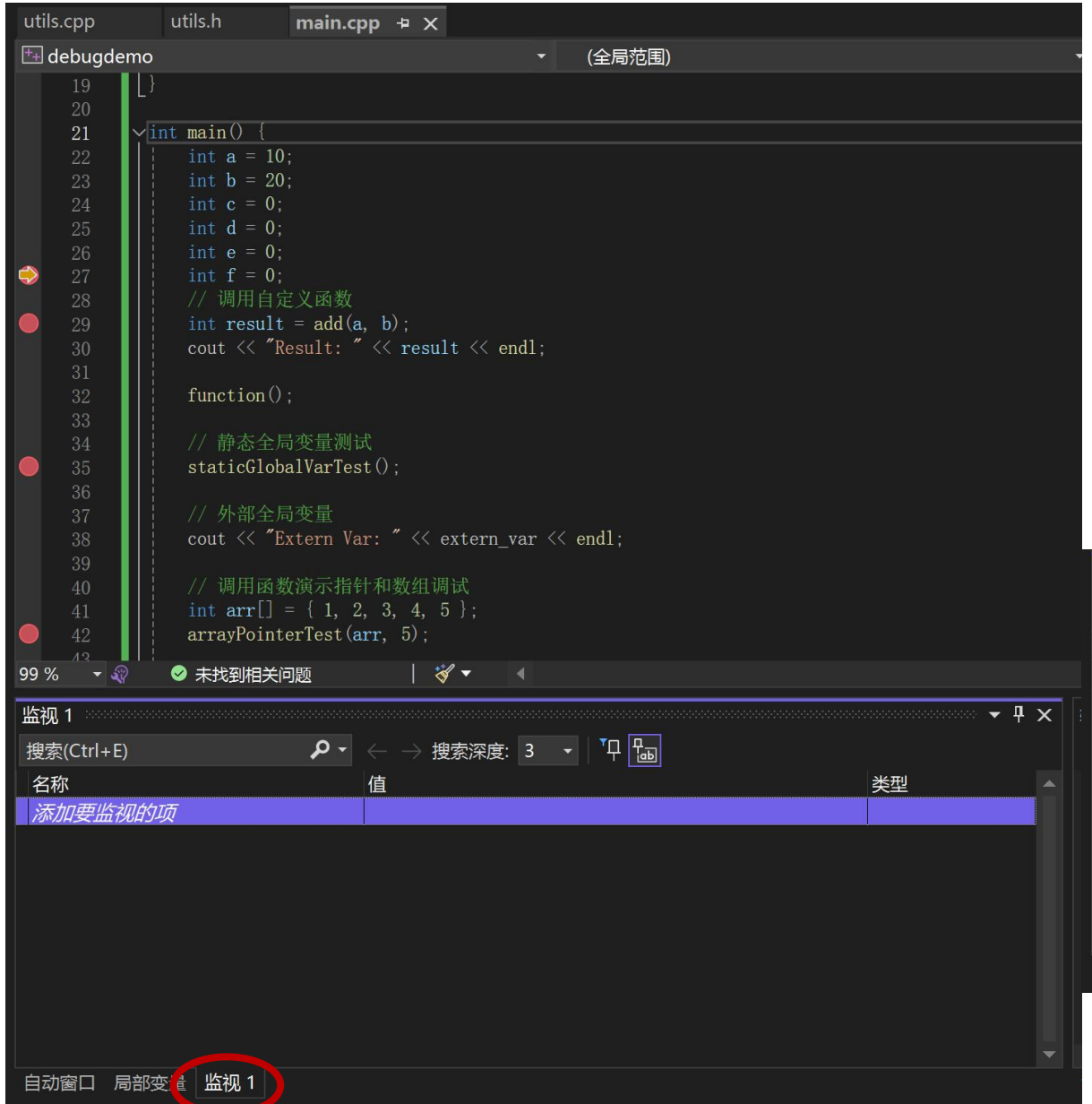
4.3.1单击左下角“局部变量窗口”，即可查看局部范围内定义的变量

包括：当前函数的形参，局部变量，静态局部变量

不包括：静态全局变量，全局变量，外部全局变量

4.3.2除了变量不同，其余用法与自动窗口相同

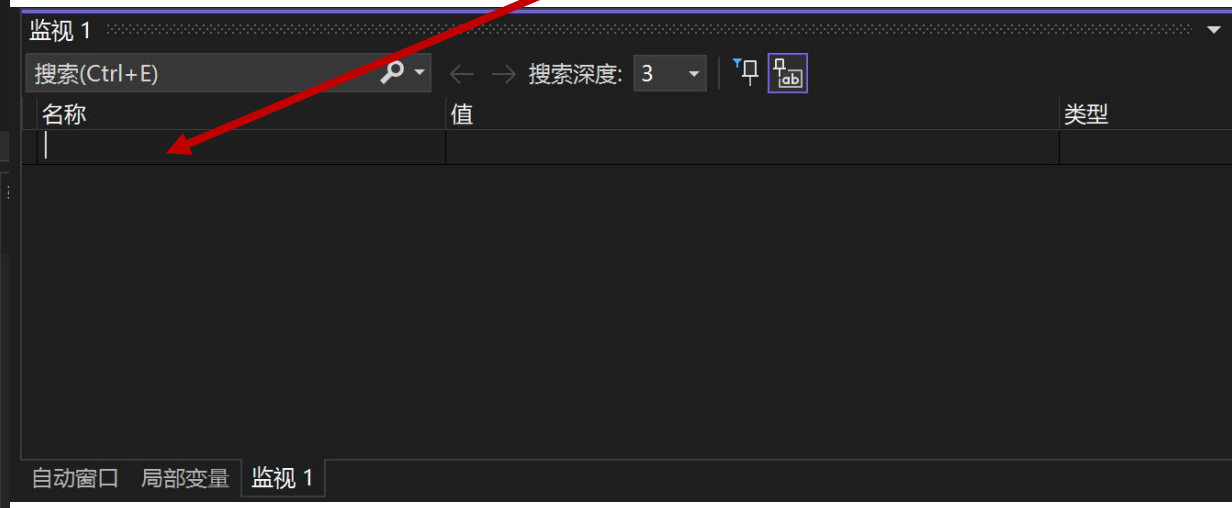
4.4查看变量：监视窗口（本页涉及2.1，2.2，2.3,2.4，3.1）



4.4.1单击左下角“监视1”（也可以选择监视2,3等），即可查看指定的变量

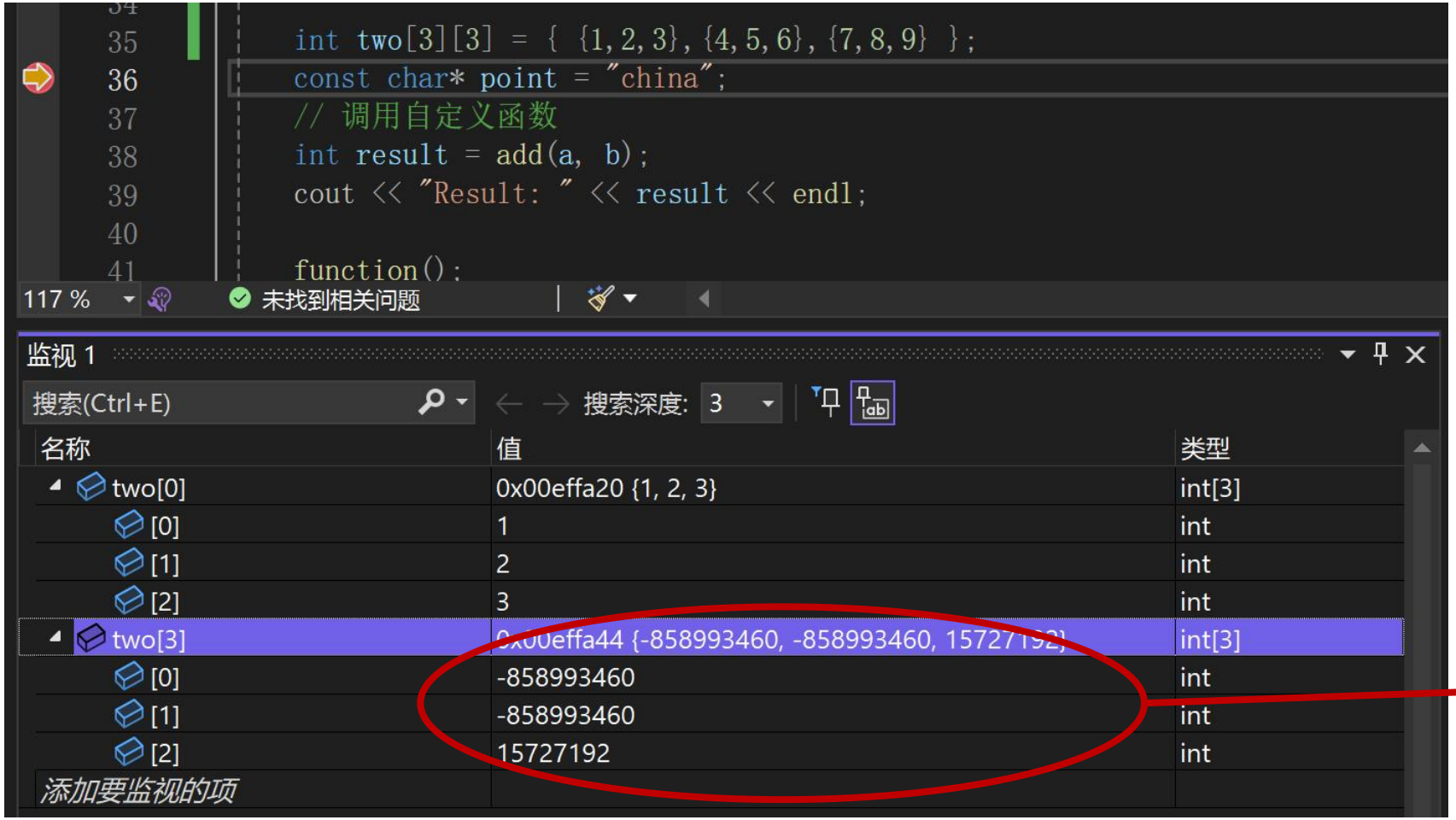
4.4.2如何添加想要监视的变量？

左键单击空的一行，键入想要监视的变量名；也可以选中代码中的变量/表达式，直接拖入监视窗口



4.4查看变量：监视窗口（本页涉及3.5，3.9）

4.4.2若添加的变量是二维数组带一个下标（不带下标见前面的自动窗口），则相当于一个一维数组



越界会显示随机值（数组名 + 下标本质是地址）

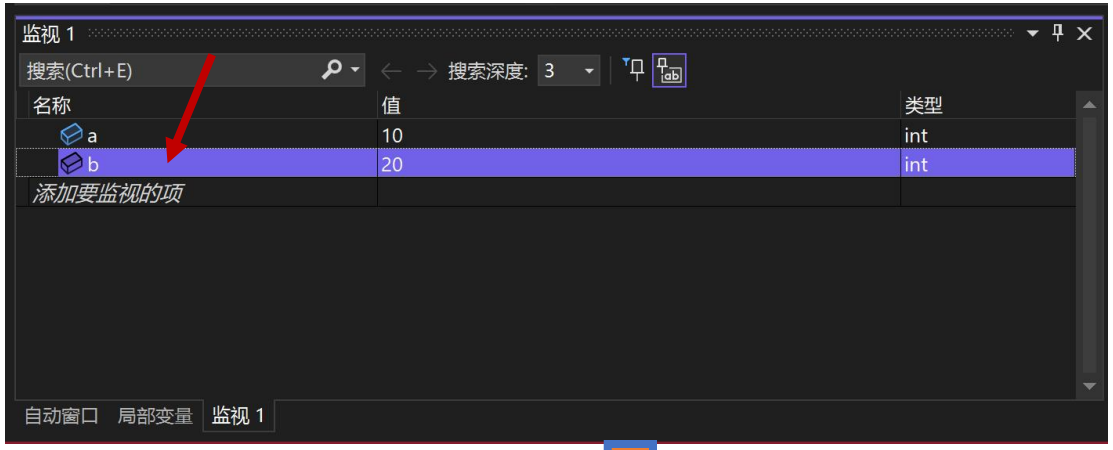
4.4查看变量：监视窗口（本页涉及2.1，2.2，2.3,2.4，3.1,3.2）

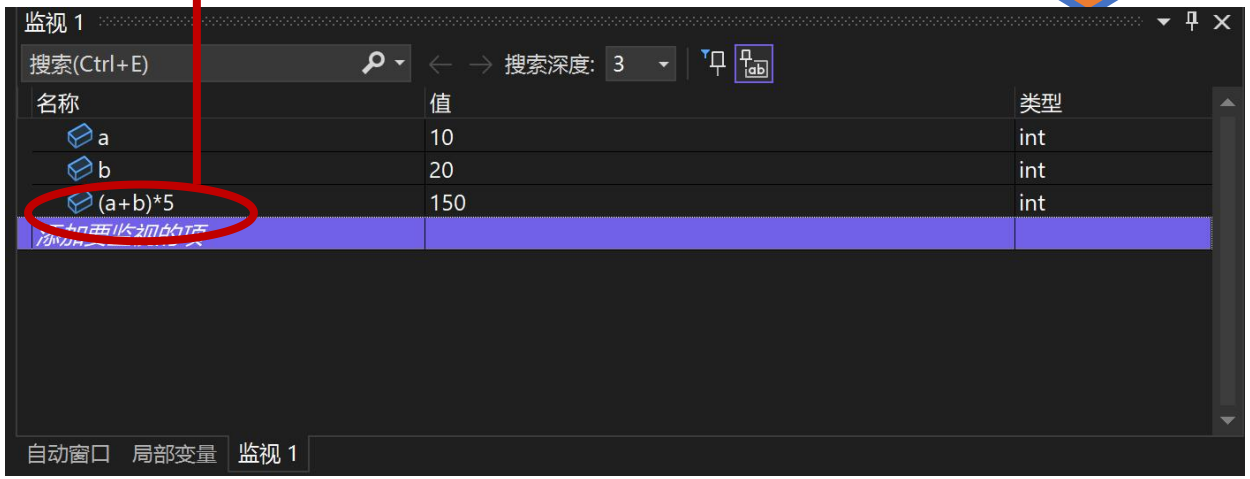
4.4.3换一个变量监视：左键单击那个变量，键盘键入想改为监视的变量

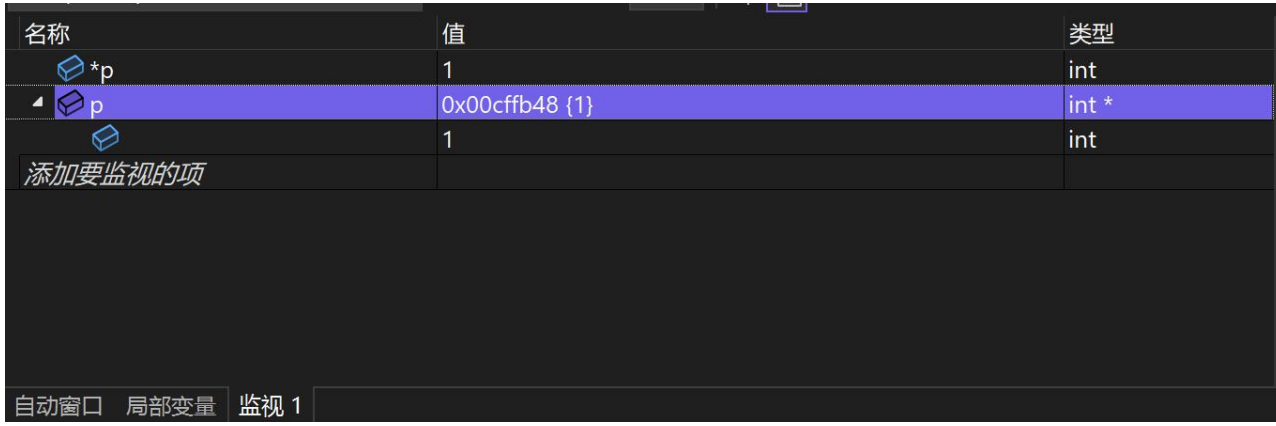
4.4.4也可以监视表达式（必须符合语法规范），键入表达式即可；也可以监视指针变量，输入*+指针变量，会监视指针所指的变量，输入指针变量本身，则监视指针变量本身

注：监视的变量可以不在作用域内

由b改为监视（a+b）*5







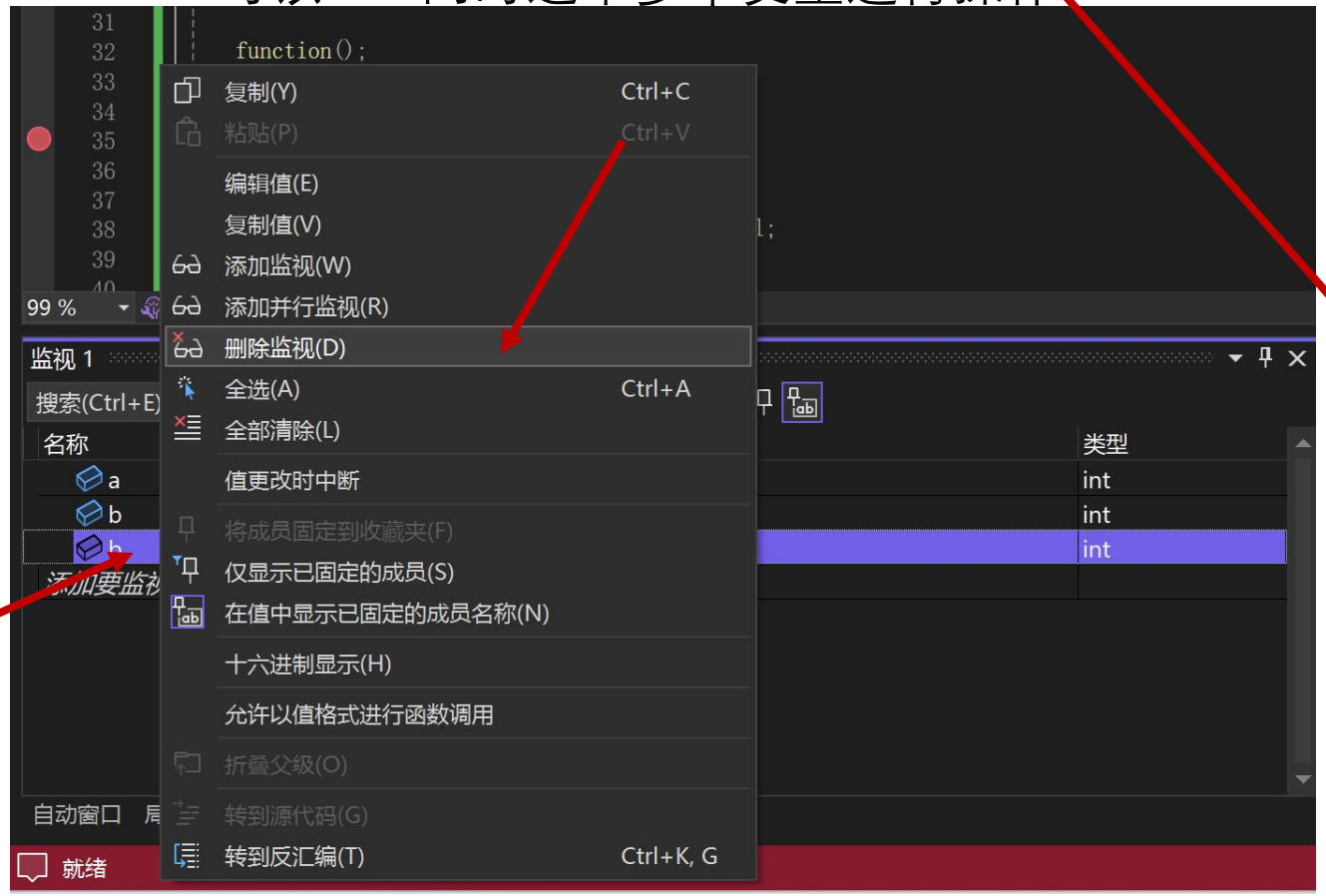
4.4查看变量：监视窗口（本页涉及2.1，2.2，2.3,2.4，3.1）

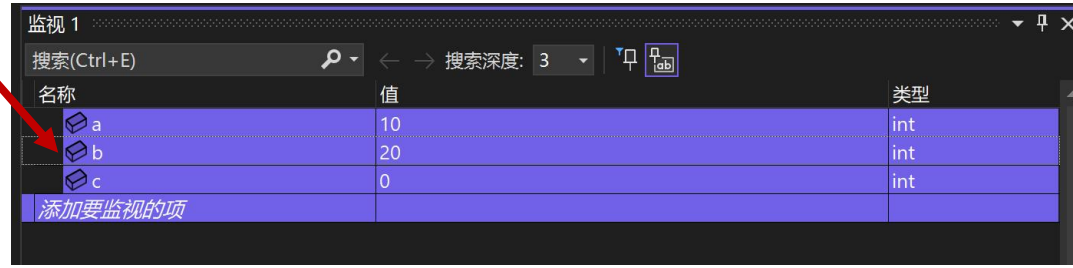
4.4.5如何删除监视？

方法一：右键单击想要删除的变量，选择“删除监视”

方法二：左键点击变量让其变为紫色，按delete键

4.4.6可以shift同时选中多个变量进行操作





4.4查看变量：监视窗口（本页涉及2.1，2.2，2.3,2.4，3.1）

4.4.7监视中添加同名变量遵循“低层屏蔽高层”原则
本例中，main中定义的global_var屏蔽了同名的全局变量

```
21  int main() {
22      int a = 10;
23      int b = 20;
24      int c = 0;
25      int d = 0;
26      int e = 0;
27      int f = 0;
28      int global_var = 99;
29      // 调用自定义函数
30      int result = add(a, b);  已用时间 <= 2ms
31      cout << "Result: " << result << endl;
32
33      function();
34
35      // 静态全局变量测试
36      staticGlobalVarTest();
```

99 % 未找到相关问题

监视 1

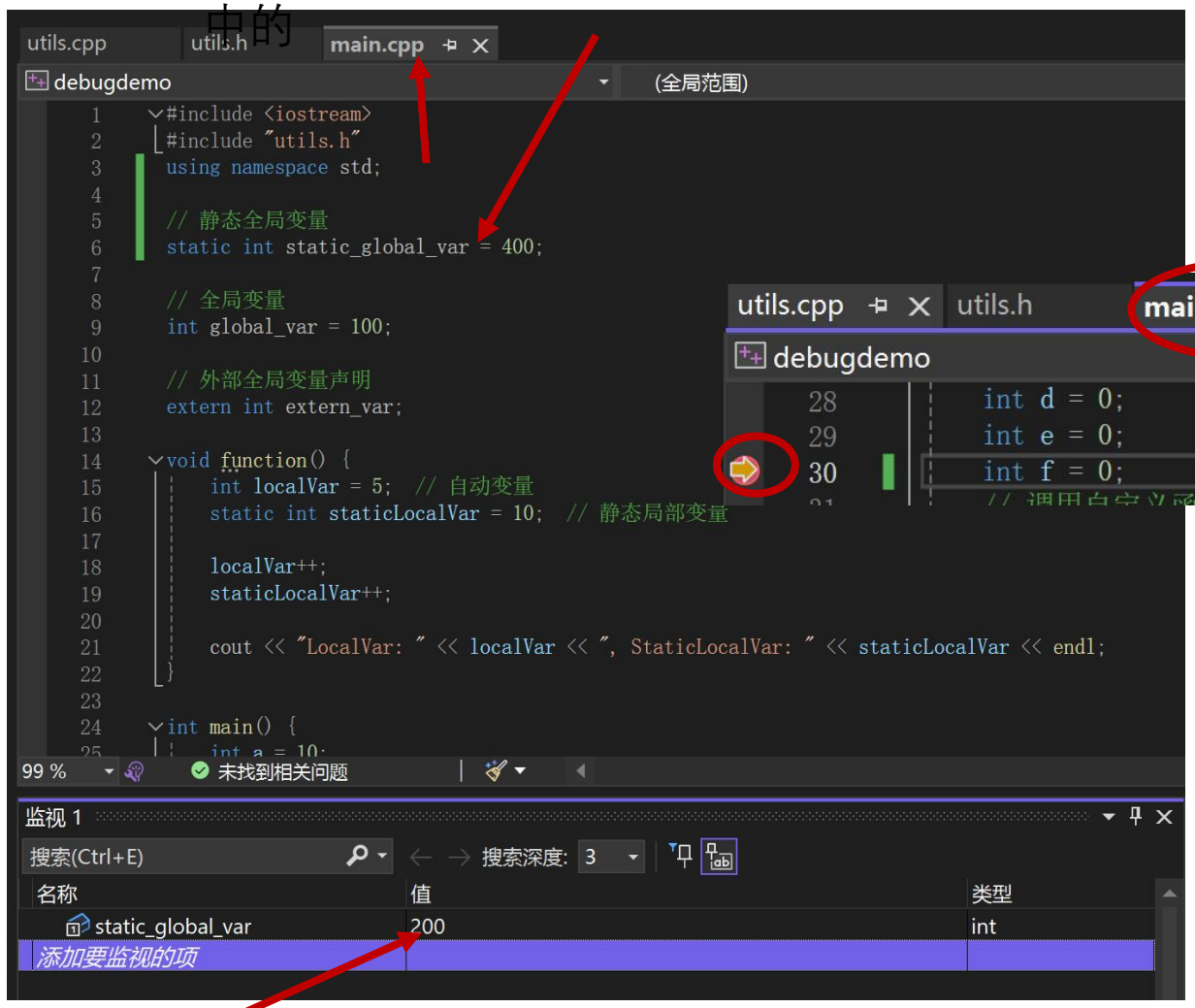
搜索(Ctrl+E) 搜索深度: 3

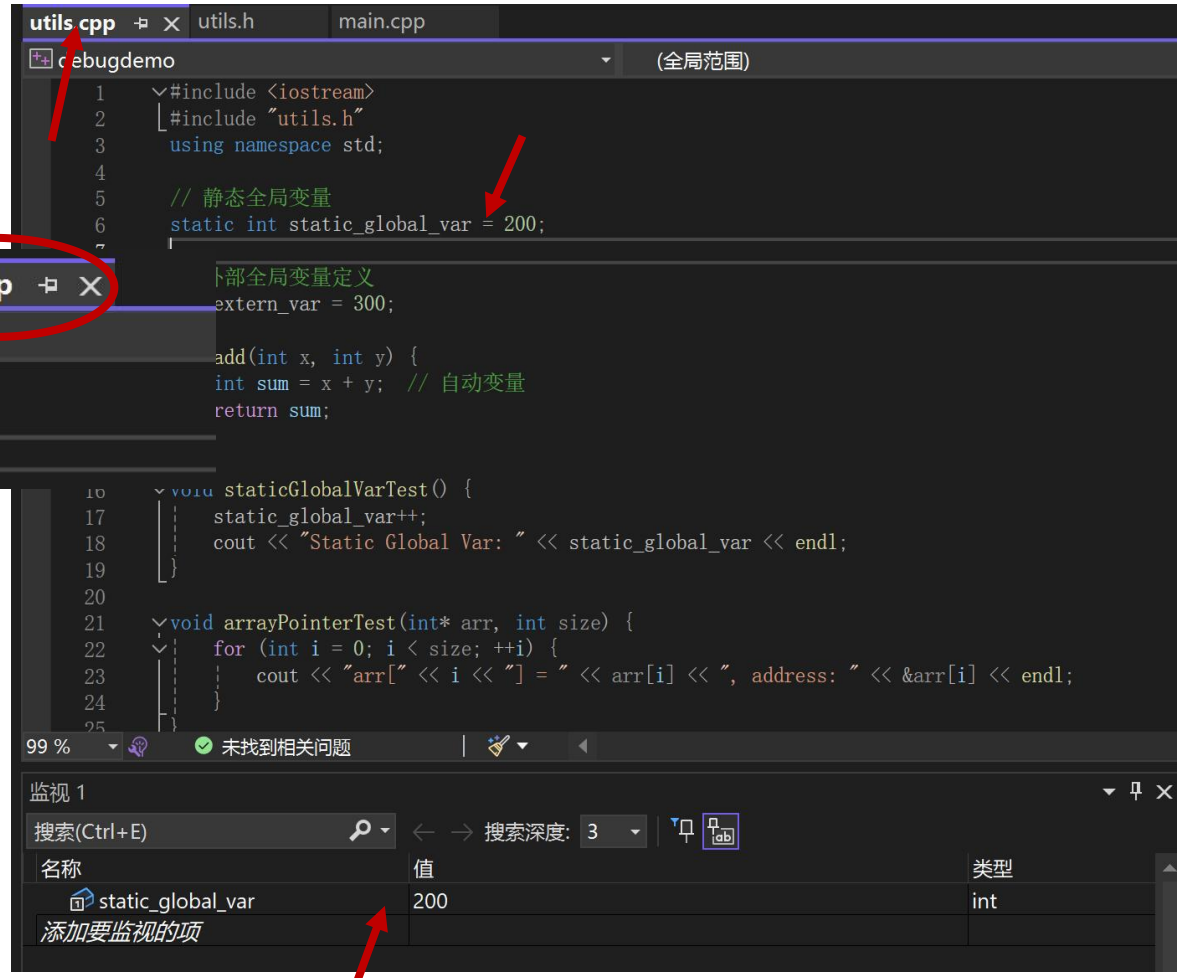
名称	值	类型
a	10	int
b	20	int
c	0	int
global_var	99	int
添加要监视的项		

```
4
5   // 全局变量
6   int global_var = 100;
7
```

4.4查看变量：监视窗口（本页涉及2.1，2.2，2.3,2.4，3.1）

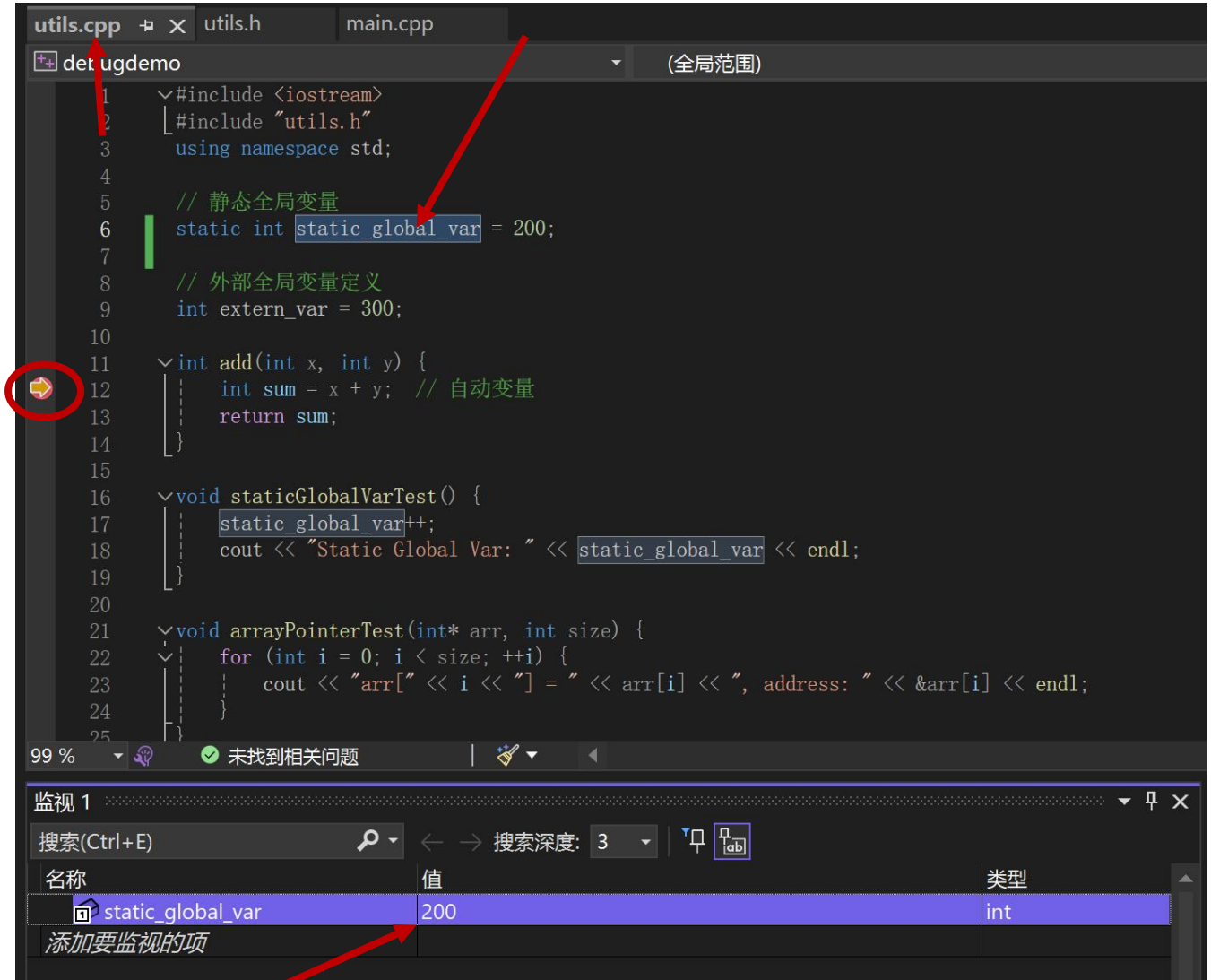
4.4.8监视中添加的同名变量为静态局部变量时，结果可能不尽人意
静态局部变量只在本源程序内有效。本例中，main.cpp和utils.cpp中定义的静态全局变量static_global_var同名,执行箭头位于main.cpp中，监视窗口里的static_global_var却全是utils.cpp中的

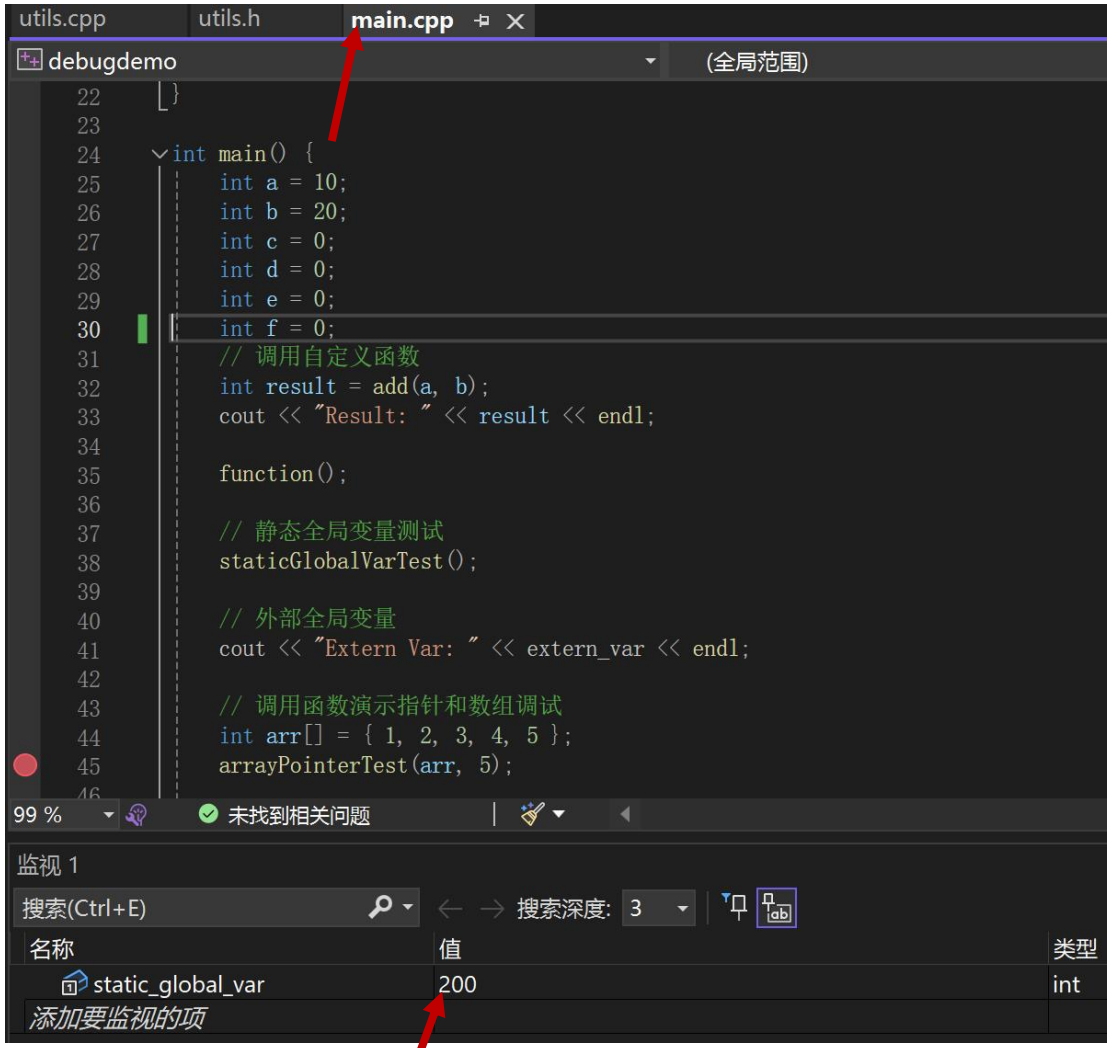




4.4查看变量：监视窗口（本页涉及2.1，2.2，2.3,2.4，3.1）

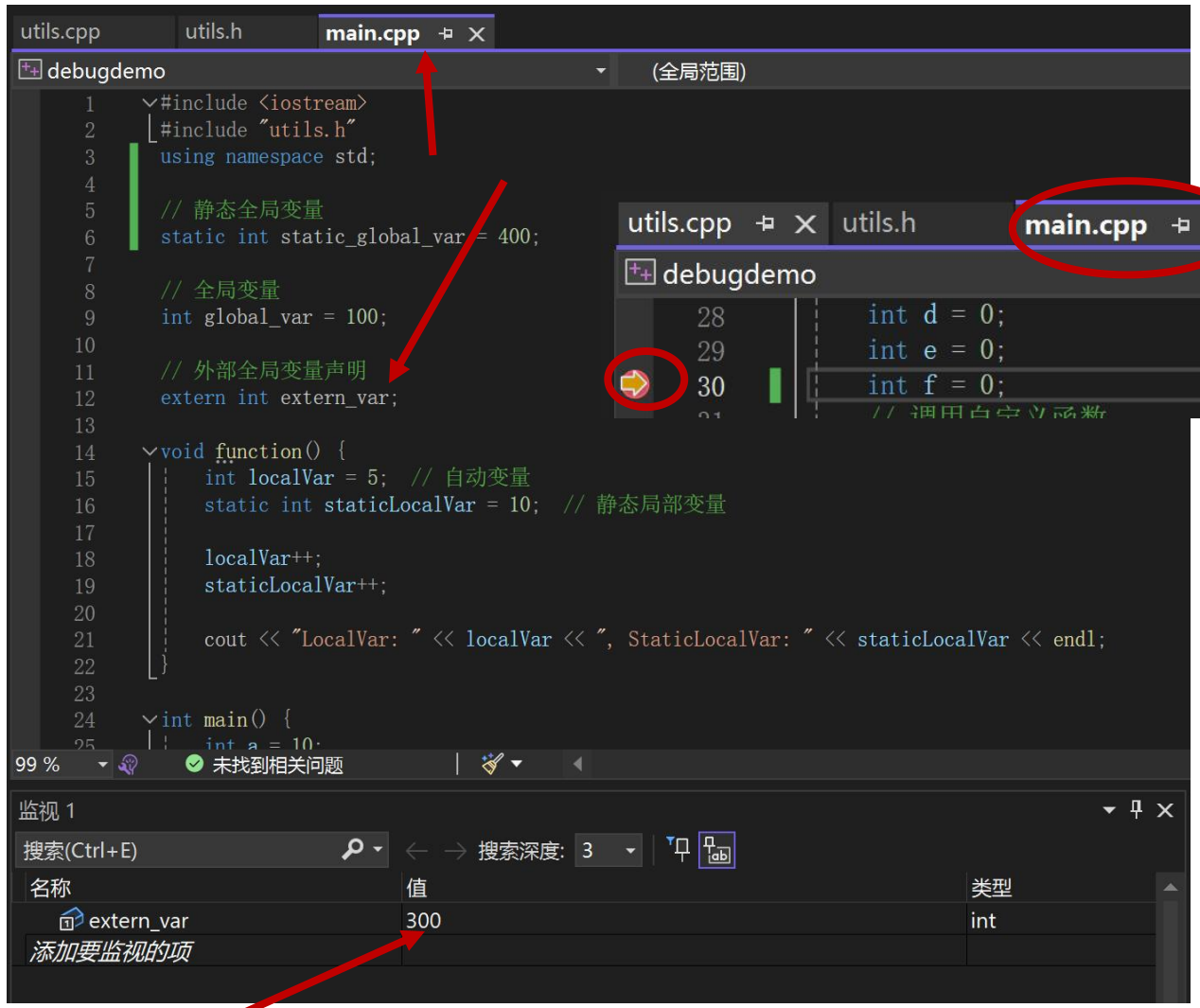
4.4.8监视中添加的同名变量为静态局部变量时，结果可能不尽人意
本例中，main.cpp和utils.cpp中定义的静态全局变量static_global_var同名,执行箭头位于utils.cpp中，监视窗口里的static_global_var也全是utils.cpp中的

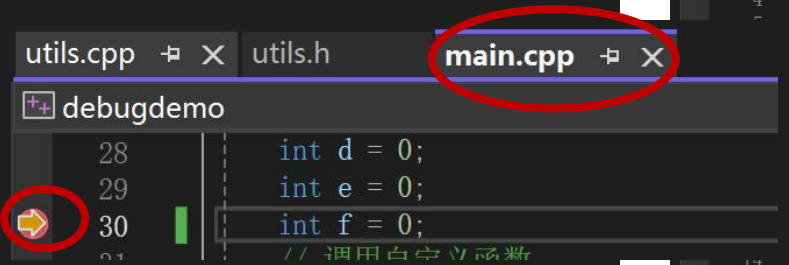


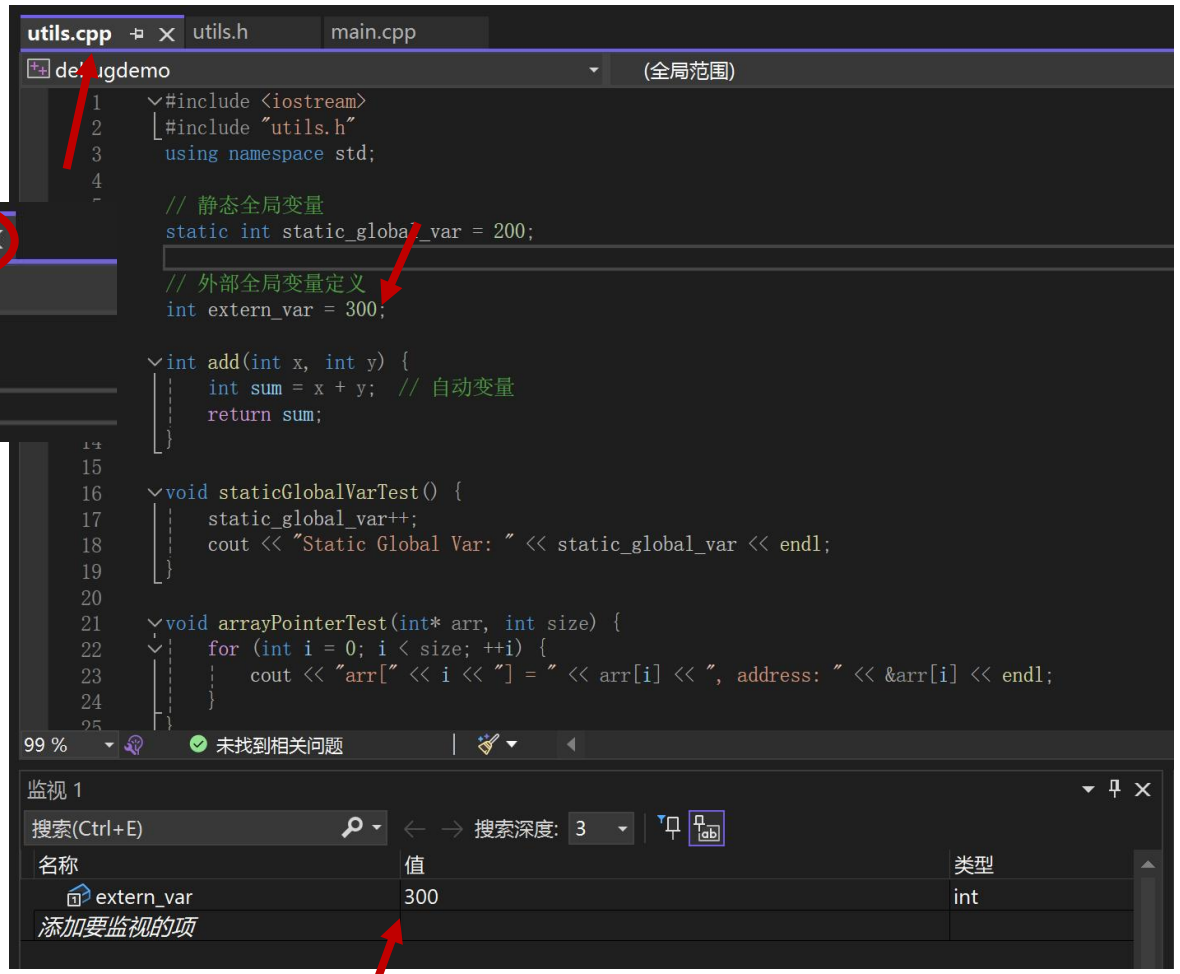


4.4查看变量：监视窗口（本页涉及2.1，2.2，2.3,2.4，3.1）

4.4.9监视中若添加extern声明的全局变量
本例中，utils.cpp中定义的全局变量extern_var在main.cpp内被声明,执行箭头位于main.cpp中，
监视窗口里的extern_var是utils.cpp中的300







4.4查看变量：监视窗口（本页涉及2.1，2.2，2.3,2.4，3.1，3.2）

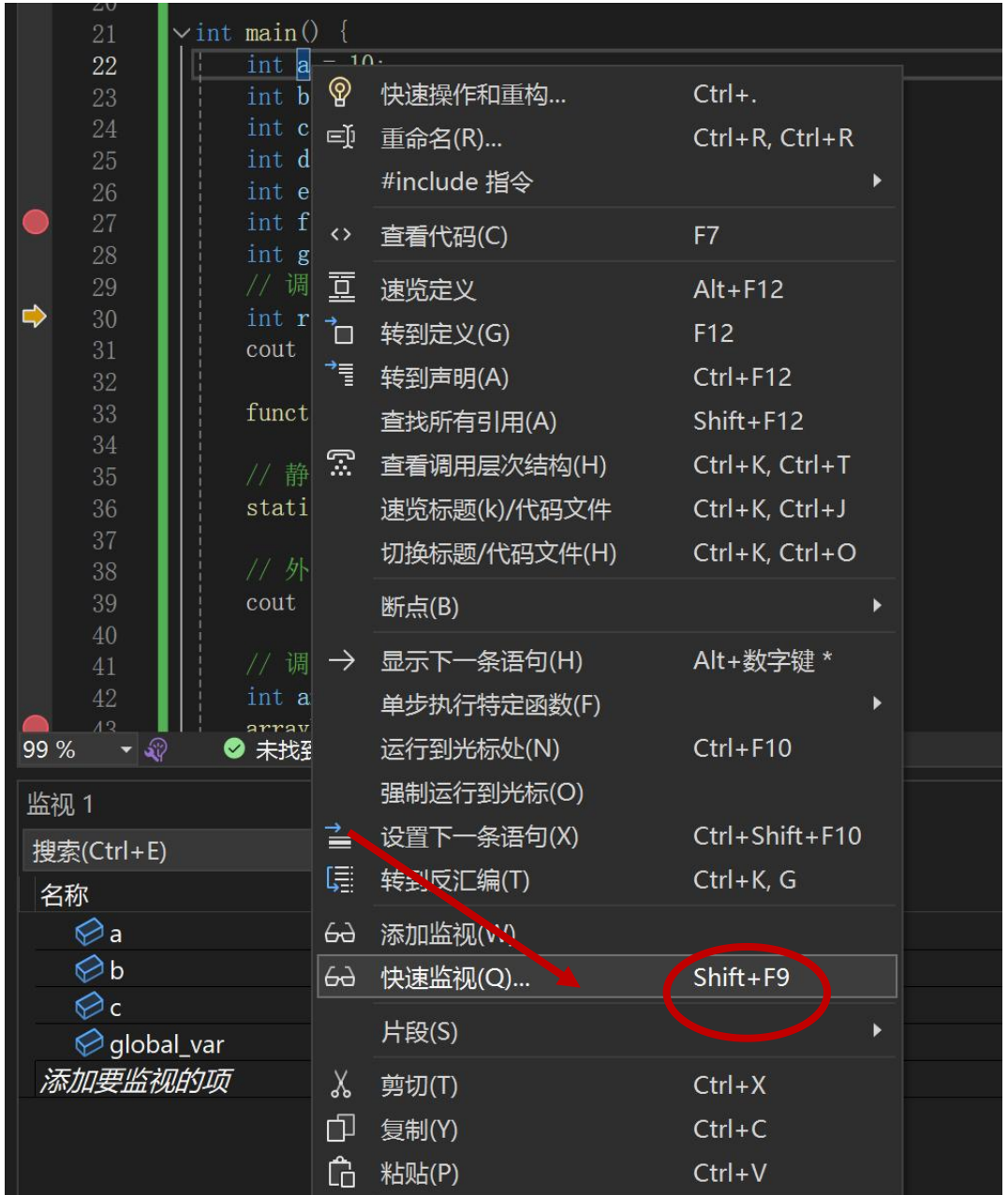
- 4.4.10除了添加的变量不同外，监视窗口的其余用法与自动窗口相同

4.5查看变量：快速监视（本页涉及2.1，2.2，3.1）

4.5.1如何打开快速监视窗口

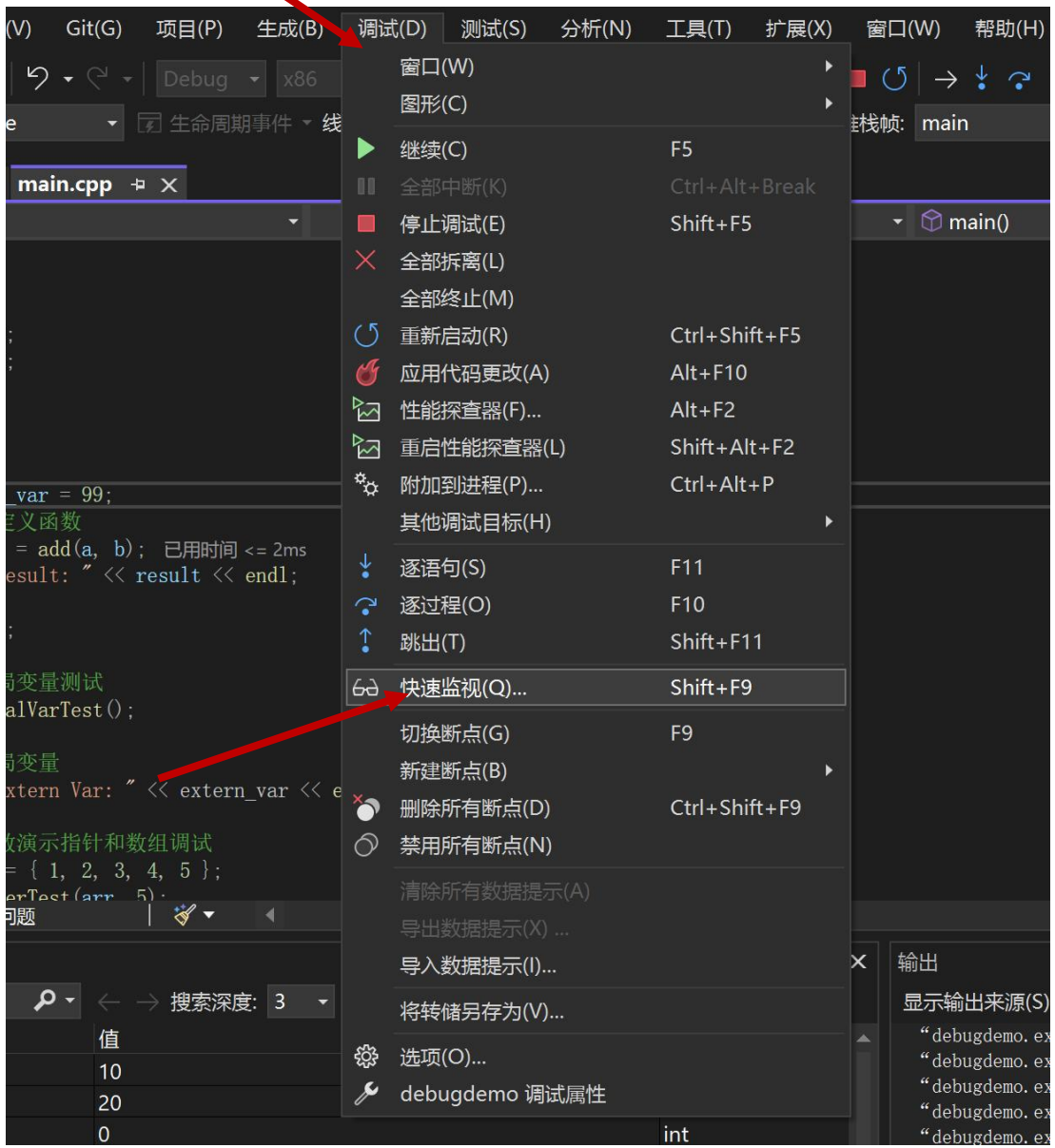
方法一：右键单击代码任意处，选择“快速监视

方法二：快捷键shift+F9



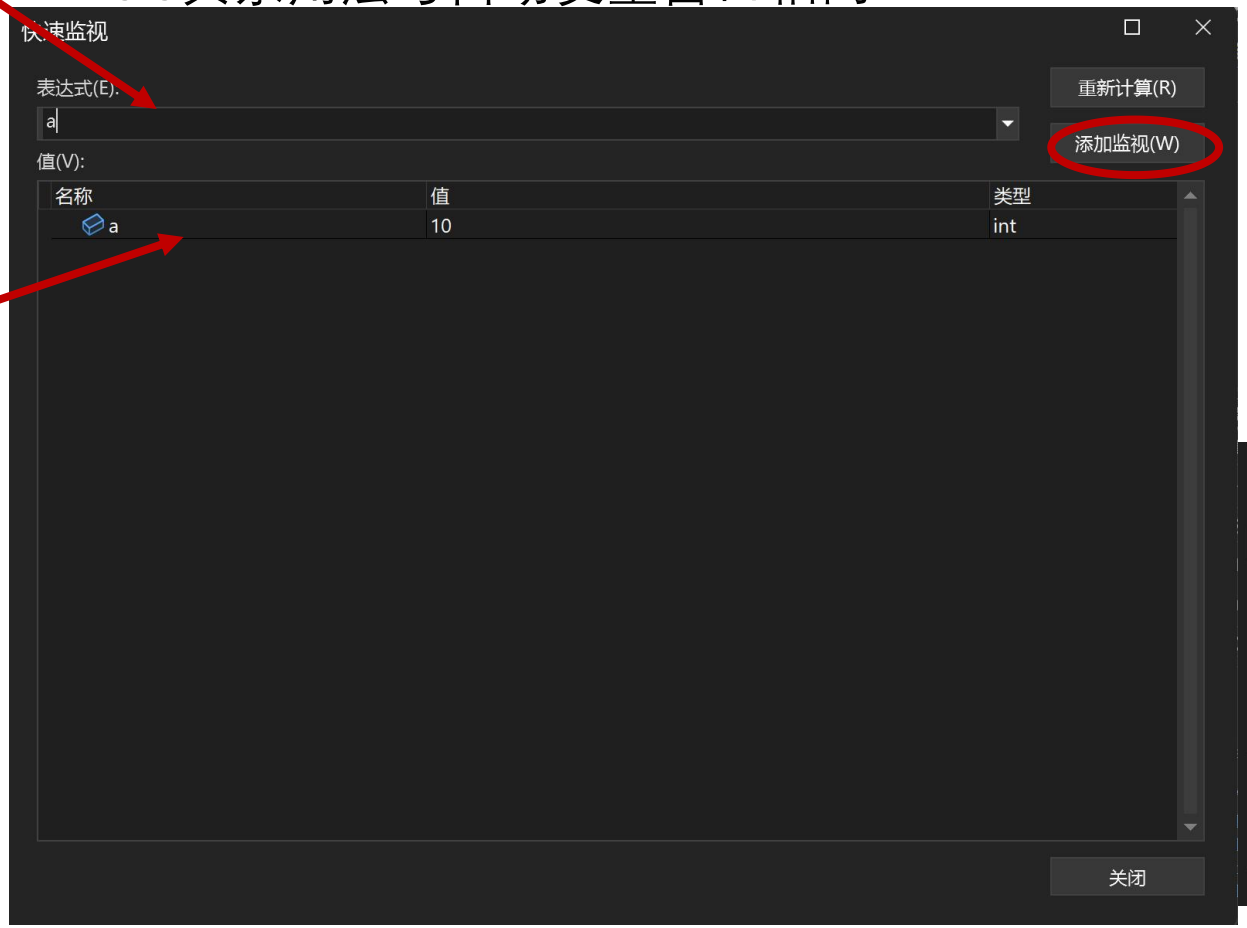
4.5查看变量：快速监视（本页涉及2.1，2.2，3.1）

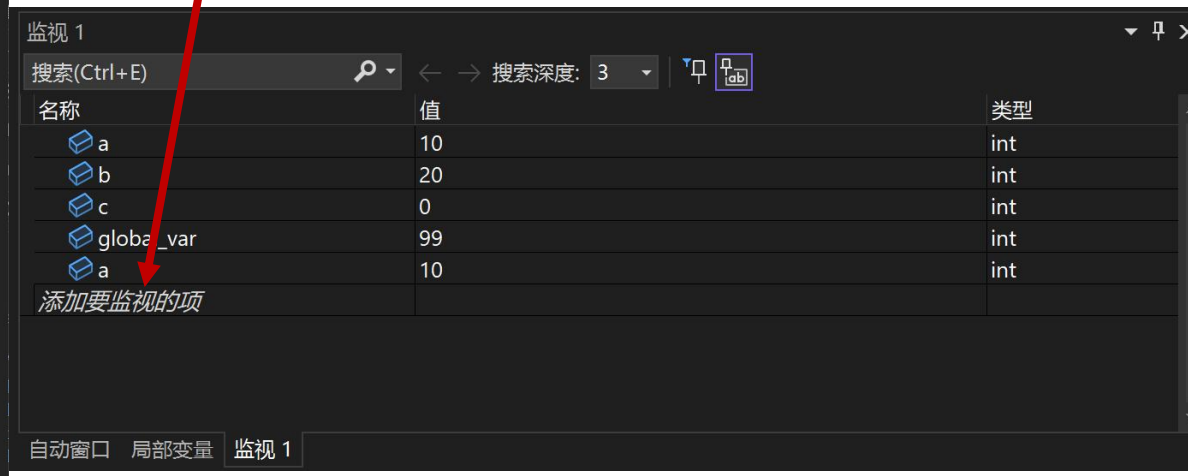
4.5.1如何打开快速监视窗口
方法三：调试>快速监视



4.5查看变量：快速监视（本页涉及2.1，2.2，3.1）

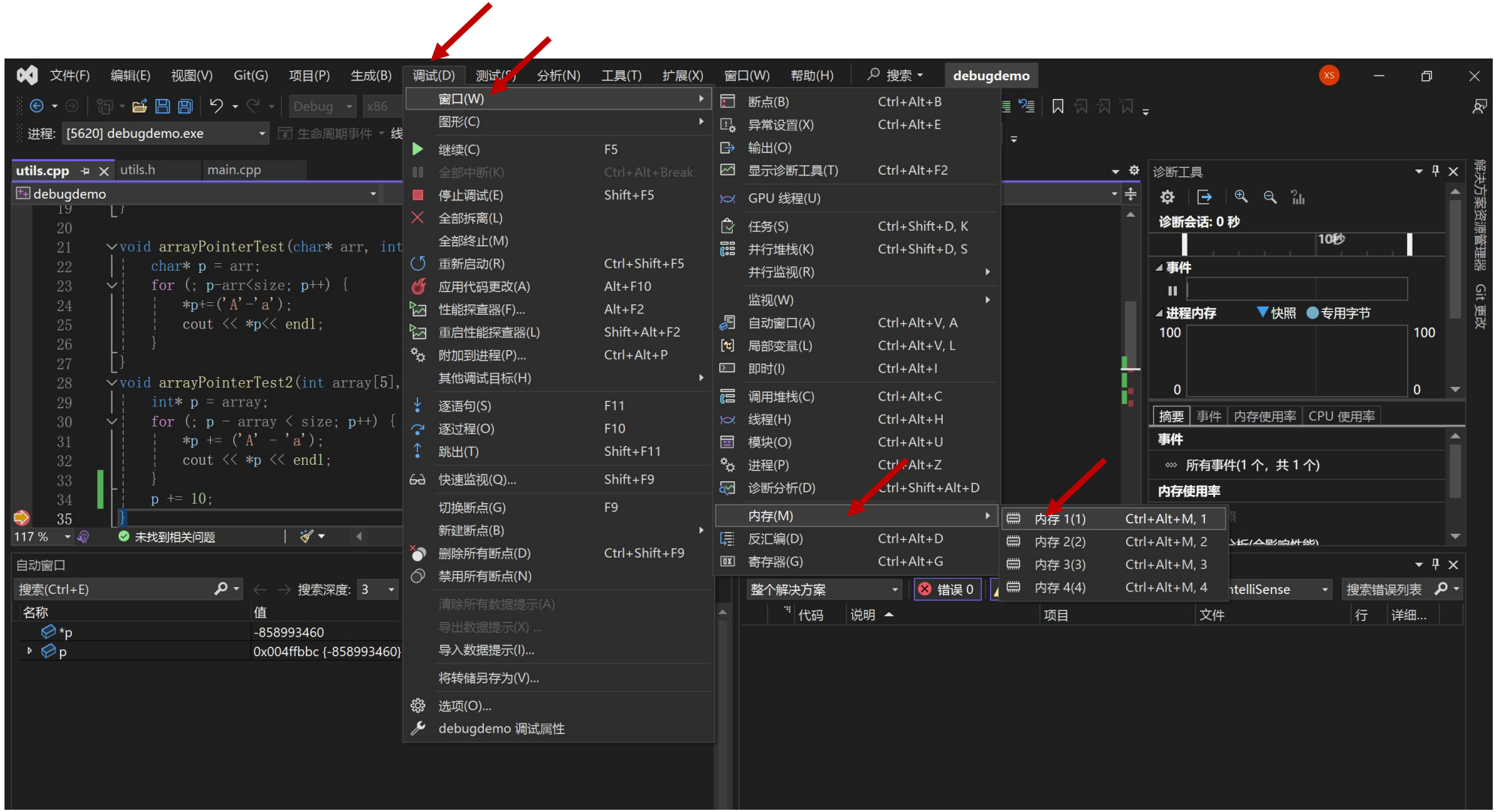
4.5.2在弹出的快速监视窗口中，键入想要监视的变量/表达式，一次只能监视一个，再次键入会覆盖上一个（也可以监视数组，规则与之前相同，监视数组则可监视不止一个简单变量）
4.5.3可以单击“添加监视”，本次快速监视的变量/表达式会被添加到监视窗口中
4.5.4若不执行4.5.3，则关闭快速监视窗口时，监视过的变量不会出现在任何窗口
4.5.5其余用法与自动变量窗口相同





4.6查看变量：查看内存

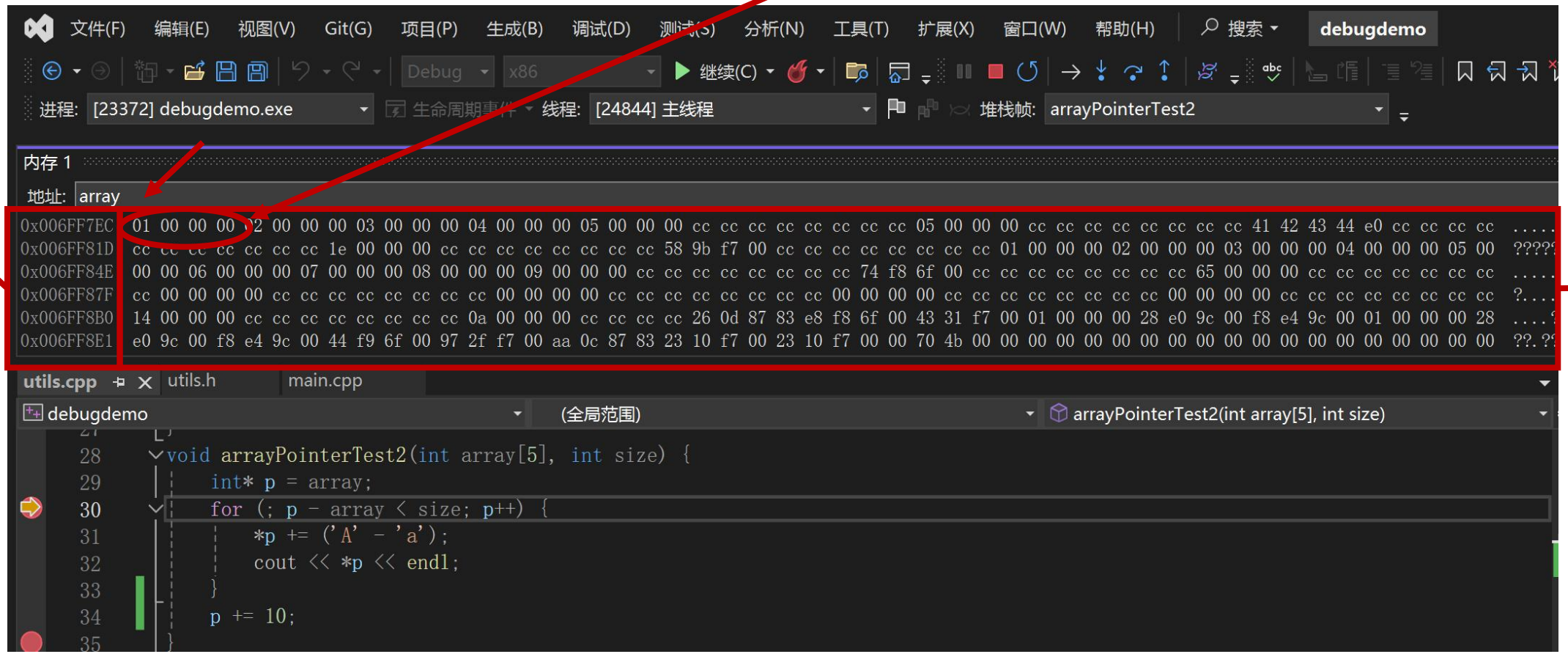
- 4.6.1打开内存窗口：调试开始后，依次点击调试>窗口>内存>任意内存窗口



4.6查看变量：查看内存（本页涉及2.1-3.9）

- 4.6.2输入地址：在弹出的内存窗口处直接输入十六进制地址/变量名，即可查看该变量处以及往后的地址上的值。默认显示格式为一个数字一字节，数字为16进制。
- 本例中，array为int型，因此四个数字表示四个字节，表示array中的一个元素

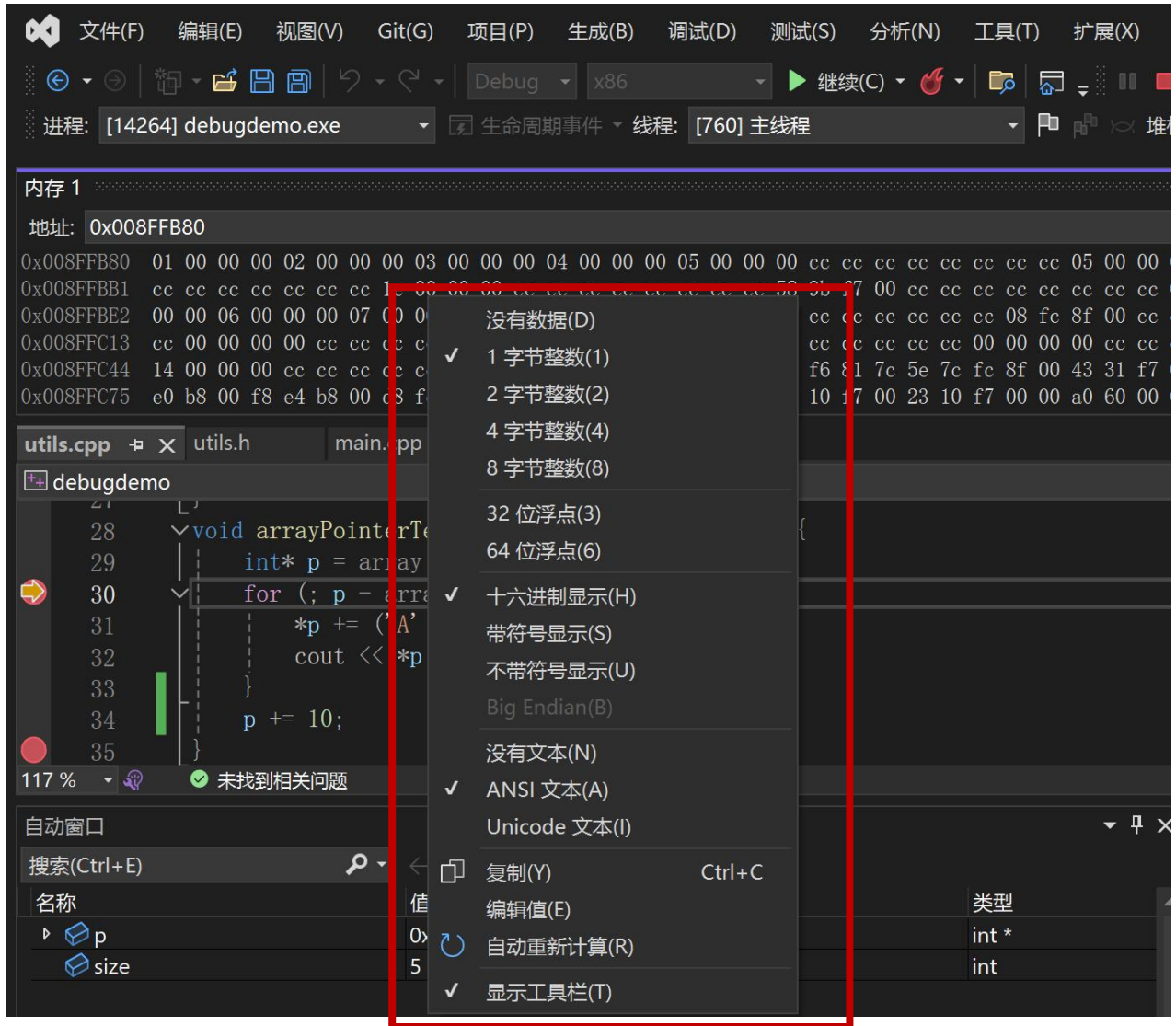
地址



地址上的值，每四个数字表示一个值，每过四个数字地址加1。后面的问号表示省略

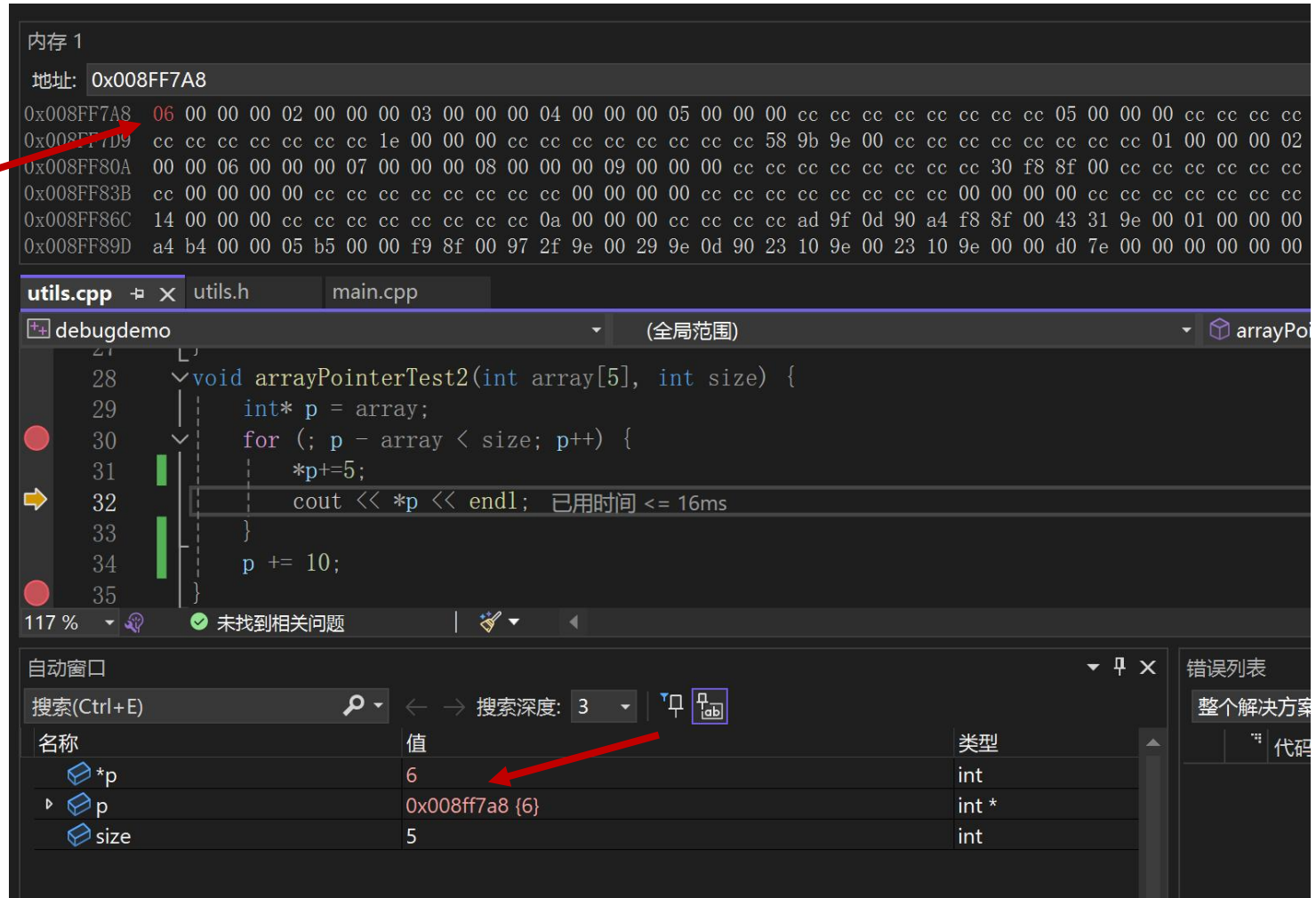
4.6查看变量：查看内存

- 4.6.3改变显示格式：右键内存区域，可以在弹出的窗口中改变格式



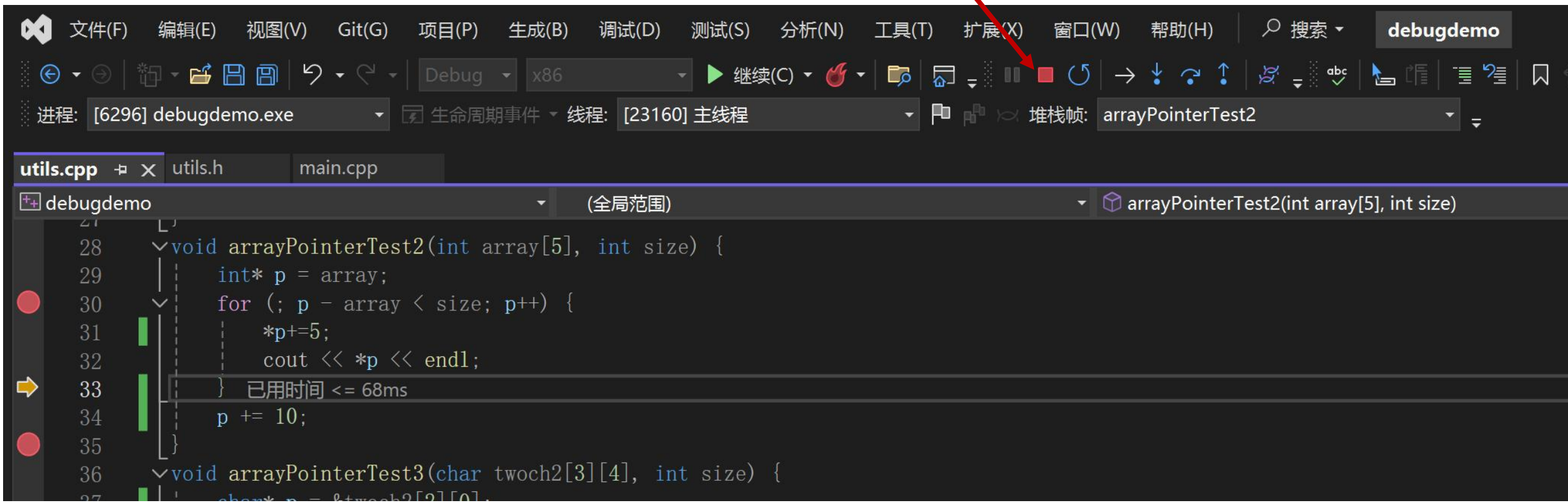
4.6查看变量：查看内存（本页涉及2.1-3.9）

- 4.6.4查看变量变化：调试过程中，若地址上的值变化，则会标红
- 本例中，array中的元素++，相应变化处标红



5结束调试（本页涉及1.1）

- 方法一：点击红色方块



5结束调试（本页涉及1.1）

- 方法二：调试>停止调试
- 方法三：快捷键shift f5

